



Y3997262



中国科学院大学

University of Chinese Academy of Sciences

博士学位论文

大规模并行深度学习训练的自动优化研究

作者姓名: 卢忠哲

指导教师: 孙凝晖 研究员

中国科学院计算技术研究所

学位类别: 工学博士

学科专业: 计算机系统结构

培养单位: 中国科学院计算技术研究所

2022 年 6 月



中国科学院大学

University of Chinese Academy of Sciences

博士学位论文

大规模并行深度学习训练的自动优化研究

作者姓名: 户忠哲

指导教师: 孙凝晖 研究员

中国科学院计算技术研究所

学位类别: 工学博士

学科专业: 计算机系统结构

培养单位: 中国科学院计算技术研究所

2022 年 6 月

Research on Automatic Optimization of Large-scale Parallel
Deep Learning Training

A dissertation submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Doctor of Engineering
in Computer System Architecture

By

Hu Zhongzhe

Supervisor: Professor Sun Ninghui

Institute of Computing Technology, Chinese Academy of Sciences

June, 2022

中国科学院大学
学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：

日 期：

中国科学院大学
学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以按照学术研究公开原则和保护知识产权的原则公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：卢忠哲

日 期：2022.6.15

导师签名：

日 期：

2022.6.15

摘 要

随着深度学习技术的快速发展。深度学习预训练呈现出三个特点：海量的预训练数据（PB 级别），超大参数量的预训练模型（TB 级别），超大规模分布式计算机机群（E 级别）。这三个特点也带来了三个挑战：❶ 数据并行下，随着 batch size 的增大，预训练得到的模型精度却损失的越来越多，模型无法收敛，从而严重影响了并行加速带来的收益；❷ 如何让 GPU 显存可以容纳更大的模型和样本数据，即如何在预训练过程中优化设备内存的使用。尤其是在基于动态计算图的深度学习框架上的优化挑战更大；❸ 如何在超大规模机群上做并行加速训练，即在大量计算加速设备上探索更多数据维度的并行性。目前大部分神经网络的网络层大多使用相同的或单一的并行策略，随着计算设备规模的增加，有些层可能更倾向于复杂的并行化策略，对所有网络应用单一的并行化策略通常会导致次优的运行时性能。

本文围绕上述三个问题进行研究，主要贡献如下：

1. **VBS 优化器**用于数据并行下大 batch size 训练时避免模型精度丢失，该优化器包括分阶段可变 batch size 优化策略和超参自动调优器两个部分。首先，本文分析了 SGD 优化器不同更新阶段对训练过程的影响，发现早期阶段使用的 batch size 几乎完全决定了神经网络的最终模型精度。基于这一发现，本文提出了一种用于分布式卷积神经网络（CNN）训练的可变 batch size 训练策略。接着，基于 BatchNorm 层的特点，我们认为存在一个统一的有效学习率，并给出了其形式化表示。进而引导我们设计了一个超参自动调优引擎。这个自动调优引擎的效果显著，它可以通过理论指导自动调整有效学习率中的超参数。实验结果表明在不丢失模型精度的情况下，VBS 能够在 ResNet-50 训练中成功地将 batch size 扩展到 120K；在只有轻微模型精度损失的情况下，将 batch size 扩展到 128K；并且在其他模型上也表现出优秀的泛化能力。

2. **MegTaiChi 内存管理模块**是一个用于 DNN 训练的基于动态图张量的自动内存管理优化模块，它实现了张量自动划分、自动驱逐以及重新生成的协同优化。MegTaiChi 的关键特性是它根据运行时跟踪的动态张量访问模式自动做出内存管理决策。这种设计的动机是观察到在训练迭代期间对张量的访问模式是

有规律的。基于识别出的模式，MegTaiChi 利用总内存优化空间实现启发式、自适应和细粒度的内存管理，MegTaiChi 同时也利用了动态计算图下静态神经网络的结构特点重新对张量做内存重排管理从而在支持更大 batch size 和模型的情况下避免了动态计算图下带来的内存碎片问题。实验结果表明，与 DTR[1] 相比，MegTaiChi 可以将 ResNet-50 的内存占用减少高达 11%，GL-base 减少 10.5%。对于具有代表性的 DNN 的训练，MegTaiChi 比 MegEngine 和 Sublinear[2] 平均最大容纳 batch size 分别高出 5 $\times$ 和 2.4 $\times$ 。对于百万规模的人脸识别应用，与 256 块 GPU 上的最优经验并行策略相比，MegTaiChi 实现了 1.8 $\times$ 的加速。

3. **MAP 自动并行模块**是一个用于自动生成并行策略的模块，MAP 借助深度学习框架提供的数据和设备的全局视角，用于简化分布式并行训练。在 MAP 的全局视角下，机群被抽象为一台“超级计算设备”。并描述了“超级计算设备”全局视角下的数据与机群中真实的物理设备上的数据的映射关系。我们采用和传统的数据和模型并行性不同的观点，并将算子的并行化方法重新表达为一种符号层面的抽象，指定逻辑数据张量和相应物理张量之间的映射。通过利用对数据，计算设备和算子的抽象，我们对整个神经网络构造一个并行策略搜索空间，通过构建整个神经网络对应的计算图的性能模型，并利用贪心搜索算法，搜索出每个算子的并行策略，使得搜索出的整个神经网络模型的并行策略能够接近甚至超越手动设置的策略。

关键词：并行训练，大 batch size 训练，显存优化，计算图，自动并行

Abstract

With the rapid development of deep learning technology. Deep learning pre-training presents three characteristics: massive pre-training data (Petabyte level), very large number of pre-training models (Terabyte level), and large-scale distributed computer fleet (Eflops level). These three features also bring three challenges: ❶ With the increase of batchsize in data parallelism, the accuracy of the pre-trained model is lost more and more, and the model cannot converge, which seriously affects the benefit of parallel acceleration. ❷ How to make GPU memory available for larger models and sample data, both in terms of how to optimize device memory usage during pre-training. In particular, optimization challenges are greater on deep learning frameworks based on dynamic computational graphs. ❸ How to do parallel training on a very large fleet of machines, i.e., exploring parallelism in more data dimensions on a large number of computationally accelerated devices. Most of the current network layers of neural networks mostly use the same or a single parallelization strategy. As the size of computing devices increases, some layers may prefer complex parallelization strategies, and applying a single parallelization strategy to all networks usually leads to sub-optimal runtime performance.

This paper focuses on the above three issues and the main contributions are as follows:

1. **VBS optimizer** is used to avoid the loss of model accuracy when training with large batch size under data parallelism, and it consists of two parts: a variable batch size strategy and a hyperparameter auto-tuner . First, we analyze the impact of different update phases of the SGD optimizer on the training process, and find that the batch size used in the early phase almost completely determines the final model accuracy of the neural network. Based on this finding, we propose a variable batch size training strategy for distributed convolutional neural network (CNN) training. Then, based on the characteristics of the BatchNorm layer, we argue that there exists a uniform effective learning rate and give its formal representation. This leads us to design a hyperparameter auto-

tuning engine. This auto-tuning engine is remarkable in that it can automatically adjust the hyperparameters in the effective learning rate by theoretical guidance. Experimental results show that VBS is able to successfully scale the batch size to 120K in ResNet-50 training without loss of model accuracy; scale the batch size to 128K with only slight loss of model accuracy; and show excellent generalization on other models as well.

2. **MegTaiChi memory management module** is a dynamic graph tensor-based automatic memory management module for DNN training that implements cooperative optimization for automatic tensor partitioning, automatic eviction, and regeneration. The key feature of MegTaiChi is that it automatically makes memory management decisions based on the dynamic tensor access patterns tracked at runtime. This design is motivated by the observation that access patterns to the tensor are regular during training iterations. Based on the identified patterns, MegTaiChi utilizes the total memory optimization space to achieve heuristic, adaptive and fine-grained memory management, while MegTaiChi also takes advantage of the structural features of static neural networks under dynamic computation graphs to rearrange the tensor memory to avoid the memory fragmentation caused by dynamic computation graphs while supporting larger batch sizes and models. The problem of memory fragmentation under dynamic computation graphs is avoided while supporting larger batch sizes and models. The experimental results show that MegTaiChi can reduce the memory consumption of ResNet-50 by up to 11% and GL-base by 10.5% compared to DTR. For training representative DNNs, MegTaiChi can accommodate on average 5 $\times$ and 2.4 $\times$ larger batch sizes than MegEngine and Sublinear, respectively. For a million-scale face recognition application, MegTaiChi achieves a 1.8 $\times$ speedup compared to the optimal empirical parallel strategy on 256 GPUs.

3. **MAP automatic parallelism module** is a module for automatically generating parallel policies. MAP simplifies distributed parallel training with a global view of data and devices provided by the deep learning framework. In the global view of MAP, a fleet of machines is abstracted as a "supercomputing device". We also describe the mapping between the data in the global view of the "supercomputing device" and the data on the real physical devices in the fleet. We take a different view from the traditional

data and model parallelism and re-express the parallelization of operators as a symbolic level abstraction specifying the mapping between the logical data tensor and the corresponding physical tensor. By using the abstraction of data, computational devices and operators, we construct a parallel strategy search space for the entire neural network. By constructing a performance model of the computational graph corresponding to the entire neural network and searching the parallel strategy for each operator using a greedy search algorithm, the parallel strategy searched for the entire neural network model can approach or even surpass the manually tuning strategy.

Keywords: Parallel training, Large batch size training, Memory optimization, Computational graph, Automatic parallelism

目 录

第 1 章 引言	1
1.1 深度学习训练与高性能计算	1
1.1.1 深度学习应用训练数据不断增加	2
1.1.2 深度神经网络模型不断增大	3
1.1.3 深度学习训练并行加速需求增大	3
1.2 并行深度学习训练所面临的挑战	7
1.2.1 大 batch size 训练	7
1.2.2 基于计算图的显存优化	8
1.2.3 并行策略选取	10
1.3 研究动机	11
1.4 本文主要工作和创新点	12
1.5 本文的组织结构	13
第 2 章 背景及相关工作研究概述	15
2.1 高效的大 Batch 训练	15
2.1.1 大 batch size 训练	15
2.2 大模型：显存优化	18
2.2.1 计算图（Computation Graph）	18
2.2.2 运行时库上的显存优化	19
2.2.3 静态计算图上的 Checkpointing	19
2.2.4 张量切分	21
2.2.5 动态计算图上的 Checkpointing	21
2.3 并行策略	23
2.3.1 数据并行	23
2.3.2 模型并行	24
2.3.3 Pipeline 并行	24
2.3.4 自动并行	24
第 3 章 可变 batch size 训练策略 VBS	27
3.1 问题描述	27
3.2 可变 batch size 训练策略	28
3.2.1 CNN 训练行为分析	28
3.2.2 不衰减学习率，增大 batch size	29

3.3 基于有效学习率的超参调优引擎	33
3.3.1 有效学习率	33
3.3.2 基于有效学习率自动调参	37
3.4 实验	38
3.4.1 数据和模型	38
3.4.2 实验环境配置	38
3.4.3 复现 facebook 的实验	39
3.4.4 VBS 策略评估	40
3.4.5 自动调优引擎评估	40
3.4.6 在不同 CNN 网络上的泛化能力评估	42
3.4.7 与 LARS 的比较	42
3.4.8 与 K-FAC 和 GNC 的比较	48
3.4.9 20 分钟训练 ImageNet-1K	48
3.5 相关工作	50
3.6 小结	53
第 4 章 基于动态计算图的显存优化模块 MegTaiChi	55
4.1 问题描述	55
4.1.1 动态计算图下显存优化存在的挑战	56
4.2 内存优化模块架构	58
4.2.1 动态张量切分	60
4.2.2 动态张量驱逐	64
4.2.3 张量内存分配	66
4.2.4 运行时开销	70
4.2.5 API	71
4.3 实验	71
4.3.1 实验配置	71
4.3.2 DTP 评估	72
4.3.3 DTE 和 TMA 评估	74
4.3.4 关于内存使用	77
4.3.5 MegTaiChi 的评估	78
4.4 相关工作	83
4.5 小结	84

第 5 章 自动并行模块 MAP	85
5.1 并行策略自动优化	85
5.1.1 自动并行策略架构	85
5.1.2 分布式训练符号抽象	85
5.1.3 分布式通信自动路由	87
5.1.4 推广到高维	88
5.1.5 性能模型	92
5.1.6 局部贪心算法	98
5.2 点融合的选择优化	102
5.3 传输代理节点	104
5.4 延迟时间及覆盖算法	105
5.5 通信代价	107
5.6 API	109
5.7 实验	110
5.7.1 实验配置	110
5.7.2 ResNet-50 性能测试	111
5.7.3 并行策略-VGG16	112
5.7.4 InsightFace 评估	112
5.7.5 2D SBP 测试	114
5.7.6 对比 Megtron 和 DeepSpeed	115
5.8 相关工作	116
5.9 小结	117
第 6 章 总结与展望	119
6.1 论文总结	119
6.2 未来工作	120
参考文献	123
作者简历及攻读学位期间发表的学术论文与研究成果	131
致谢	133

图形列表

1.1 目前 SOTA 模型训练的浮点数运算量 (以 FLOPs 为衡量单位)。蓝线上的是 CV, NLP 和语音模型, 模型运算量平均每两年翻 15 倍, 红线上的是 Transformer 模型, 模型运算量平均每两年翻 750 倍。而灰线则标志摩尔定律下内存硬件大小的增长, 平均每两年翻 2 倍。[3] ...	2
1.2 SOTA 模型的参数数量的增长趋势, 图中的绿点表示 AI 硬件 (GPU) 的内存大小, 大型的 Transformer 模型以每两年翻 240 倍接近指数级的速率增长, 但是单 GPU 的内存却只是每两年翻 2。[3]	4
1.3 带宽、内存和硬件的计算能力的增长趋势, 从中可以看出带宽增长非常的缓慢 (大约每 20 年增加 30 倍, 而硬件的计算能力则增加了 90,000 倍)。[3]	4
1.4 GPU 逐渐成为深度学习训练主流的加速器。[4]	5
1.5 数据和模型的增大使得训练规模越来越大。[4]	6
1.6 并行策略逐渐多样性。[4]	6
1.7 Batch size 对准确度和性能的影响。[4]	8
1.8 DNN 训练主要的内存开销来源。[3]	9
1.9 静态图上的优化。	9
1.10 一维卷积运算中张量的划分方式。	11
2.1 Flatten 区极小值和 Sharp 区极小值的概念草图。	16
2.2 Facebook 的 ImageNet 训练 top-1 测试精度与 batch size 大小的关系。	17
2.3 神经网络模型的计算图表达。	18
2.4 静态计算图上的 Checkpointing 策略 (重计算)。	20
2.5 静态计算图上的 Checkpointing 策略 (swapping)。	20
2.6 基于不同张量切分策略的并行矩阵乘运算。	21
2.7 动态计算图下的 Checkpointing 策略。	22
2.8 相同的计算结果下不同的并行策略。	23
2.9 手动并行和自动并行。	25
3.1 随着节点数和 batch size 的增加, 训练精度不断降低。	27
3.2 Hessian 矩阵 top-10 特征值随着 batchsize 的增大的变化。	29
3.3 大 batch size 和小 batch size 下的角度更新值。	29
3.4 ResNet-50 四个网络层训练中 $\ w\ $ 的变化。	30
3.5 不衰减学习率, 增大 batch size。	30
3.6 前期使用 8K batch size, 后期使用 16K, 32K。	31

3.7 学习率有一个上界, 过大的学习率使得 loss 发散。	36
3.8 Batch size 分别在第二阶段增大到 16K, 32k, 64K。	39
3.9 VBS 算法在 shufflenet 上的 top-1 测试精度。	43
3.10 VBS 算法在 ResNet-101 上的 top-1 测试精度。	43
3.11 LARS 在 ResNet-50 训练 ImageNet-1K batchsize 16K 上的表现。	47
3.12 LARS 在 ResNet-50 batch size 16K 训练上的手动超参调优。	47
4.1 (a) 和 (b) 显示了 ResNet-18 训练时的性能; MP-R 和 MP-C 分别代表 基于行和列的模型并行; (c) 显示了仅基于一个参数因素对不同张量 驱逐方案的评估; (d) 显示了碎片整理程序对性能的影响。	57
4.2 MegTaiChi 架构。	58
4.3 四种高效的张量切分模式。	61
4.4 动态张量驱逐机制。	64
4.5 一次迭代中张量在 GPU 内存中的生命周期。	66
4.6 六个高效的内存管理规则。	67
4.7 模拟和优化张量在 GPU 内存中的调度过程。	68
4.8 Python API 的使用。	71
4.9 DTP 评估。	74
4.10 在单个 GPU 上评估 DTE, Meg-Capuchine, Pytorch-DTR 和 Meg-DTR 是三个基线。	75
4.11 DTE 在 8 个 GPU 上的评估, 带斜线的条形图表示基线。	76
4.12 每次迭代中的内存使用情况对比。	78
4.13 MegTaiChi 的运行时开销。	79
4.14 MegTaiChi 与 Sublinear 在单个 GPU 上训练 ResNet-50 模型的对比。	80
4.15 GPT 模型上的性能评估对比。	81
4.16 MoE 模型上的性能评估对比。	82
4.17 训练大规模人脸识别模型。	83
5.1 自动并行策略搜索框架。	85
5.2 一个逻辑张量的 1-D 分布式描述。	86
5.3 自动路由机制完成张量从 S(1) 到 B 的物理数据重排。	88
5.4 2D-SBP 中逻辑张量的物理排布。	89
5.5 分布式张量的 2D-SBP 可能的描述种类。	90
5.6 2D-SBP 自动路由将逻辑子图生成物理子图。	90
5.7 2D-SBP 自动路由将逻辑子图生成物理子图。	91
5.8 2D-SBP 自动路由将逻辑子图生成物理子图 (方式一)。	91
5.9 2D-SBP 自动路由将逻辑子图生成物理子图 (方式二)。	92

5.10 三种基本的图消去方式。	94
5.11 计算图如何通过三种消去方式做图消去。	95
5.12 边局部贪心算法。	99
5.13 邻域局部贪心算法。	99
5.14 传输代理节点。	104
5.15 不同进程在通信之前出现了等待时间。	106
5.16 Nsight system 中的等待现象。	106
5.17 神经网络主干和支流之间相互 overlap。	107
5.18 NCCL 通信库中的 Allredcue 和 Redcue-Scatter 的通信特征。	108
5.19 不同通信原语的通信代价。	108
5.20 MAP API 的使用。	110
5.21 MAP 1D-SBP 在 ResNet-50 上的测试。	111
5.22 MAP 1D-SBP 在 VGG-16 上的测试。	112
5.23 MAP 1D-SBP 在 InsightFace 上的性能测试。	113
5.24 InsightFace 在 MAP 下生成的策略。	114
5.25 MAP 在 2D SBP 模式下 Bert 和 GPT2 上的性能表现。	115
5.26 MAP 与 DeepSpeed 以及 Megtron 的对比。	115

表格列表

2.1 大 batch size 训练相关优化问题研究。	16
2.2 内存优化模块相关工作对比 (M 和 A 表示手动和自动)。	23
2.3 并行策略优化相关工作。	24
3.1 Top-1 ResNet-50 测试精度, 使用 multi-steps 策略。	40
3.2 可变 batch size 策略下, Top-1 ResNet-50 测试精度。	41
3.3 ResNet-50 训练 Top-1 的测试精度, 使用 VBS 配合超参自动调优。 ..	41
3.4 Batch size 64K 以内 ShuffleNet-v2-1x 的 Top-1 的测试精度, 使用 VBS 自动调优。	44
3.5 Batch size 64K 以上 ShuffleNet-v2-1x 的 Top-1 的测试精度, 使用 VBS 自动调优。	44
3.6 Batch size 64K 以内 ResNet-101 的 Top-1 的测试精度, 使用 VBS 自动 调优。	45
3.7 Batch size 64K 以上 ResNet-101 的 Top-1 的测试精度, 使用 VBS 自动 调优。	45
3.8 ResNet-50 的 Top-1 的测试精度, poly 学习策略配合 LR scaling 以及 warm-up。	46
3.9 ResNet-50 的 Top-1 测试精度, LARS 优化器配合超参手动调优。 ...	46
3.10 ResNet-50 的 Top-1 的测试精度, 分别对应 K-FAC、GNC 和 VBS。 ..	49
3.11 ResNet-50 的 Top-1 的测试精度在超算系统上, VBS 配合自动调优。 ..	50
4.1 四种并行切分模式下层内通信的模式。	62
4.2 张量切分带来的算子间的集合通信。	62
4.3 前向计算通信算子和对应的反向通信微分算子。	63
4.4 数据重排代价, p 表示设备数量, $ T $ 代表全局张量 T 的大小。	70
4.5 实验平台配置。	72
4.6 评估中用到的方法。	73
4.7 评估中用到的神经网络模型。	73
4.8 Conv-BN-Relu 块在 1 个 GPU 和 8 个 GPU 上的重新计算和交换开销。	76
4.9 碎片整理代价。	77
4.10 不同的方法支持的最大 batchsize。	81
5.1 1D-SBP MUL 算子分布式签名集合。	87

5.2 矩阵乘算子的 2D-SBP 分布式签名集合。	89
5.3 矩阵乘中不同的 SBP 签名通信量不同。	92
5.4 图消去算法的算法复杂度。	99
5.5 贪心算法的优化率。	100
5.6 上游 weight 算子和下游 optimizer 算子之间的代价。	103
5.7 部分算子只支持少量 SBP 可以减小搜索空间。	103
5.8 SBP proxy 策略到下游节点策略的代价。	105
5.9 上游 Op 到下游 Op 所用的通信和代价。	109
5.10 通信代价的函数拟合。	109
5.11 平台配置。	110

符号列表

缩写

DTP	Dynamic Tensor Partition
DTE	Dynamic Tensor Evict
TMA	Tensor Memory Allocation
VBS	Variable Batchsize Strategy
MAP	Multidimensional Automatic Parallelism
SOTA	State-of-the-Art
DCG	Dynamic Computational Graph
SCG	Static Computational Graph

第1章 引言

1.1 深度学习训练与高性能计算

随着高性能计算机的出现和发展,高性能计算(HPC, High performance Computing)已经成为继理论研究和实验科学之后推进人类科学研究的第三大利器[5],早已广泛应用于核模拟、石油勘探、气象预报、航空航天和工程科学等诸多研究领域[6],是当代科技和国力的战略制高点,各行各业都给予高度重视和持续投入[7]。

与此同时,机器学习,尤其是深度学习,正在迅速对我们日常生活产生影响。深度学习的核心是深度神经网络(Deep Neural Network, DNN),这是一种受人脑相互联系的本质启发而构建的结构。经过适当训练,DNN的表达能力的仅通过观察大量数据,就可以为以前认为无法解决的问题提供准确的解决方案。深度学习已经成功地应用于很多领域,从图像分类到语音识别和医学诊断,再到自动驾驶和在复杂游戏中击败人类玩家。

而HPC和AI联合应用也很广泛,例如帮助解决地球气候问题、加速科学发现,及模拟工作流程以更快完成各种任务。深度学习训练是一个高性能计算问题,因为它需要大量计算和数据通信。深度学习需要计算密集型训练,大量计算能力有助于加快训练周期。OpenAI[8]发布的分析表明,自2012年以来,人工智能训练任务中所使用的算力正呈指数级增长,其目前的增长速度为每3.5个月翻一倍(相比之下,摩尔定律是每18个月翻倍)。自2012年以来,人们对于算力的需求增长了超过300,000倍(而如果是摩尔定律的速度,只会有12倍的增长)。在此期间,GPU等异构计算加速器硬件算力的提升一直是人工智能快速发展的重要因素。随着神经网络技术的快速发展,深度学习应用这种在HPC上的负载也逐渐呈现出三种趋势:海量的预训练数据,超大参数量的预训练模型,超大规模分布式计算机机群下的多种异构加速设备。

接下来,本章将详细介绍当前在HPC上,深度学习应用负载的特点,新的负载下带来的新的问题,及软硬件各自发展而引起的性能鸿沟,从而引出本文的研究动机,并针对待解决的问题提出本文的主要工作和创新点,最后详述本文的组织结构安排。

1.1.1 深度学习应用训练数据不断增加

如图 1.1 所示，最近几年，计算机视觉 (CV)，自然语言处理 (NLP) 和语音识别领域最新模型的训练运算量，以大约每两年翻 15 倍数的速度在增长。而 Transformer 类的模型运算量的增长则更为迅速，约为每两年翻 750 倍。而数据量的大小决定了训练出的模型的好坏。即使是像模型大小较小的网络 ResNet-50 也需要 140G 左右的训练数据大小。与此同时，模型大小的增大随之也需要较大的数据集，GPT-3 使用的训练数据集十分庞大，基于网络文本、数据、维基百科等数据，它使用的最大数据集在数据预处理前容量达到了 45TB。

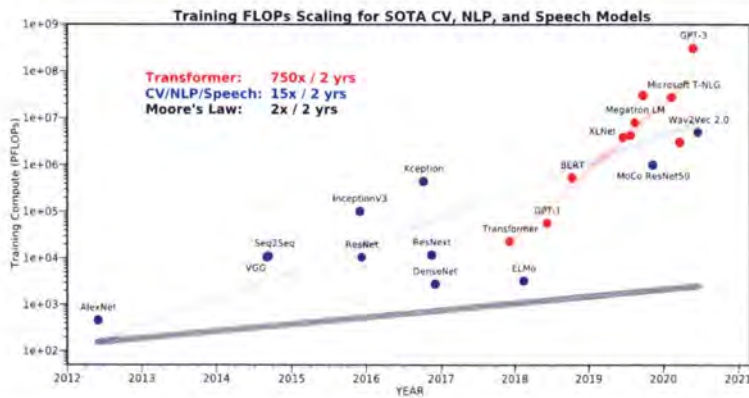


图 1.1 目前 SOTA 模型训练的浮点数运算量 (以 FLOPs 为衡量单位)。蓝线上的是 CV, NLP 和语音模型，模型运算量平均每两年翻 15 倍，红线上的的是 Transformer 模型，模型运算量平均每两年翻 750 倍。而灰线则标志摩尔定律下内存硬件大小的增长，平均每两年翻 2 倍。[3]

Figure 1.1 The current SOTA model training in floating point operations (measured in FLOPs). On the blue line are CV, NLP and speech models, where the model operations are increasing on average by 15 times every two years, and on the red line are Transformer models, where the model computation increases on average by 750 times every two years. The gray line marks the growth of memory hardware size under Moore's Law, which increases on average by 2 times every two years.[3]

为了加快 DNN 训练过程，分布式数据并行计算技术 (SPMD) 是现在深度学习应用模型训练的必用技术，在数据并行下，每块 GPU 持有相同的完整模型副本，每块 GPU 读入不同的 Mini-batch 数据。目前，大多数研究人员选择分布式随机梯度下降 (SGD) 方法进行训练。梯度的聚合和参数更新顺序执行。为了充分利用 GPU 来扩展同步 SGD 的效率，必须确保每个 GPU 在每次参数更新前

都有足够的有效工作负载。例如，将一个具有 8 个 GPU 的计算节点用于分布式 SGD（总 batch size 大小为 256）意味着每个 GPU 在每次更新时只能处理 32 个本地图像的 batch size 大小。如果 batch size 扩大到 512，则每个 GPU 会处理 64 个本地图像。通常，大 batch size 训练是指 batch size 超过 1K 的训练过程。与小 batch size 训练相比，大 batch size 训练可以减少参数更新的次数，从而缩短训练时间。由于遍历一遍完整的数据时梯度的更新次数的减少，大 batch size 的训练也能够避免大量的参数数据移动次数（数据并行下只通信参数的梯度），从而为分布式 SGD 取得了良好的性能。但是，不同于传统的科学计算的 SPMD 并行方式，分布式并行 DNN 训练的大 batch size 带来了一个新的挑战：随着训练节点的扩大，batch size 越来越大，预训练模型的准确性显著下降，过大的 batch size 造成收敛的困难，而过小的 batch size 造成硬件资源利用不足，这将抵消分布式 SGD 带来的并行加速的收益。

1.1.2 深度神经网络模型不断增大

近年来，深度学习在各个领域都取得了空前的成功，成功的另一个关键因素是越来越大的深度神经网络（DNN）获得了很高的准确性。随着深度学习（DL）的模型和数据集大小变得越来越大，用于训练神经网络的内存需求急剧增加。即使最新一代的 Nvidia GPU 最高可支持 80 GB（A100），但在许多情况下，此容量仍是主要瓶颈。例如，对于像 ResNet-200 这样的大型网络，用于训练的本地 batch size 不能大于六个 ImageNet 样本，而在 ResNet-1001 中，本地批处理大小可以降至两个样本。对于需要数成千上万亿参数的模型来说，单参数量就达到了 TB 的量级，而运行时内存的需求量需要更大的内存空间。特别是对于 NLP 和推荐系统相关的模型，即使对模型做切分，这时又会带来通信带宽瓶颈。事实上，芯片内部、芯片间还有 AI 硬件之间的通信，都已成为不少 AI 应用的瓶颈。特别是最近大火的 Transformer 类模型，模型大小平均每两年翻 240 倍（如图 1.2 所示）。类似的，大规模的推荐系统模型，模型大小已经达到了 $O(10)$ TB 的级别。与之相比，AI 硬件上的内存大小仅仅是以每两年翻 2 倍（图 1.3）的速率在增长。

1.1.3 深度学习训练并行加速需求增大

我们继续简要概述用于执行深度学习问题的并行硬件体系结构。它们可以大致分为单机（通常是共享内存）和多机（通常是分布式内存）系统。单机并行：

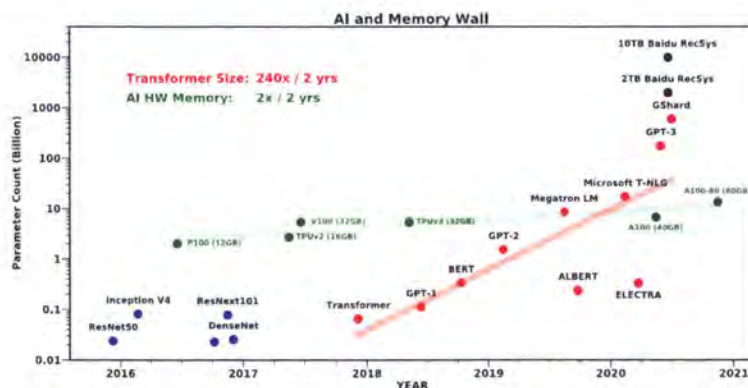


图 1.2 SOTA 模型的参数数量的增长趋势, 图中的绿点表示 AI 硬件 (GPU) 的内存大小, 大型的 Transformer 模型以每两年翻 240 倍接近指数级的速率增长, 但是单 GPU 的内存却只是每两年翻 2。[3]

Figure 1.2 The green dots in the graph indicate the memory size of the AI hardware (GPU), with the large Transformer model grows at an exponential rate of 240 times every two years, but the memory of a single GPU only is doubling every two years.[3]

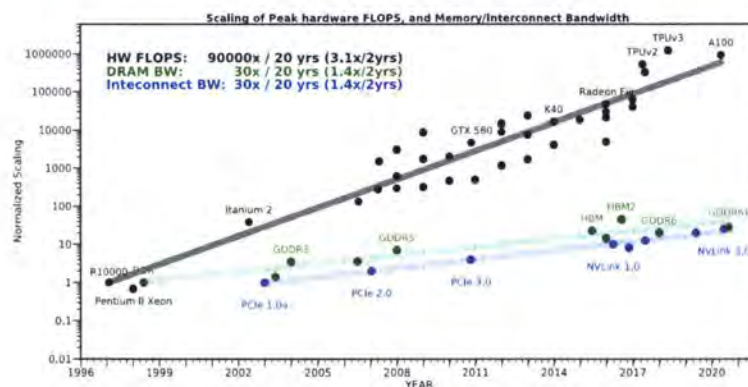


图 1.3 带宽、内存和硬件的计算能力的增长趋势, 从中可以看出带宽增长非常的缓慢 (大约每 20 年增加 30 倍, 而硬件的计算能力则增加了 90,000 倍)。[3]

Figure 1.3 The trend in the growth of bandwidth, memory and hardware computing power shows that bandwidth is growing very slowly (about 30 times every 20 years, while hardware computing power has increased by 90,000 times).[3]

并行性在当今的计算机体系结构中无处不在, 在芯片内部以流水线和乱序执行的形式存在, 并以多核或多插槽系统的形式暴露给程序员。多核系统具有悠久的传统, 可以使用多个进程 (不同的内存域)、多个线程 (共享的内存域) 或两者的混合进行编程。主要区别在于, 多进程并行编程使程序员将数据的分布视为首

要关注点，而多线程编程允许程序员仅对并行性进行推理，而将数据重排留给硬件系统（通常通过硬件缓存一致性协议）。通用 CPU 已针对从事事件驱动的桌面应用程序到数据中心服务器任务（例如，为网页提供服务和执行复杂的业务 workflows）等一般工作负载进行了优化。机器学习任务通常是计算密集型的，这使得它们类似于传统的 HPC 应用程序。因此，大型机器学习工作负载，在诸如通用图形处理单元 (GPU) 或现场可编程门阵列 (FPGA) 等加速系统上表现非常出色，这些系统已经在 HPC 领域使用了十多年。这些设备专注于计算吞吐量，通过专有化其架构来利用 HPC 工作负载中的高数据并行性。正如我们稍后将看到的，大多数学习研究人员使用 GPU 或 FPGA 等加速器进行计算。我们强调加速的主要技术是利用机器学习工作负载中固有的并行性。在 270 篇论文中，165 篇论文展示了实证结果并提供了有关其硬件设置的详细信息。图1.4显示了这些年研究论文摘要中使用的机器架构。从 2013 年开始，我们看到了 GPU 的明显趋势，它在已发表的论文中占主导地位。然而，即使是加速的节点，也不足以应付庞大的计算工作量。图1.5说明了这些工作中采用多机训练的场景越来越多。这表明，从 2015 年开始，带有加速器（如 GPU）的分布式内存架构已经成为当今各种规模的机器学习的默认选择。

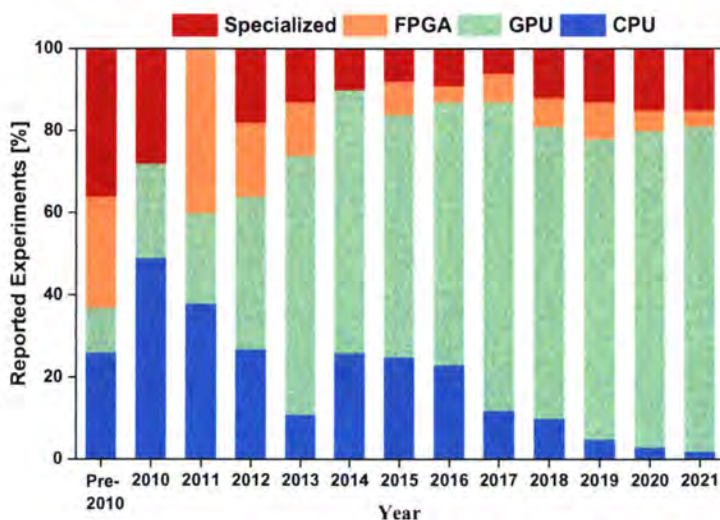


图 1.4 GPU 逐渐成为深度学习训练主流的加速器。[4]

Figure 1.4 GPUs are becoming the mainstream accelerators for deep learning training.[4]

对于多机并行训练。训练大规模模型是一项计算密集型任务。因此，单台机器通常无法在所需的时间范围内完成这项任务。为了进一步加速计算，数据和算

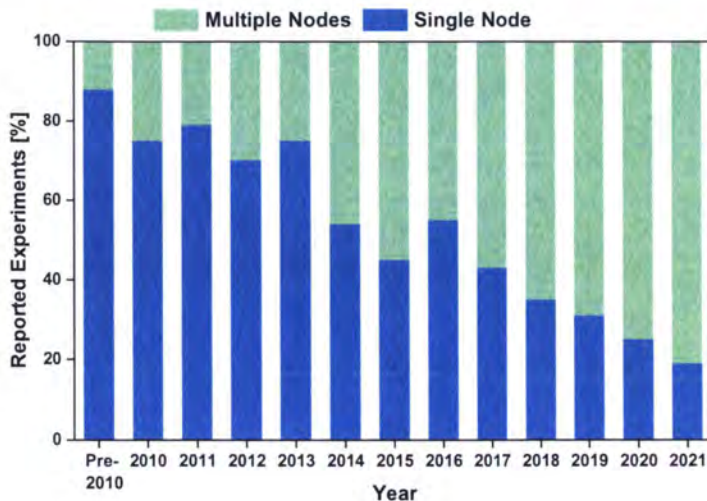


图 1.5 数据和模型的增大使得训练规模越来越大。[4]

Figure 1.5 The increase of data and models makes the training scale larger and larger.[4]

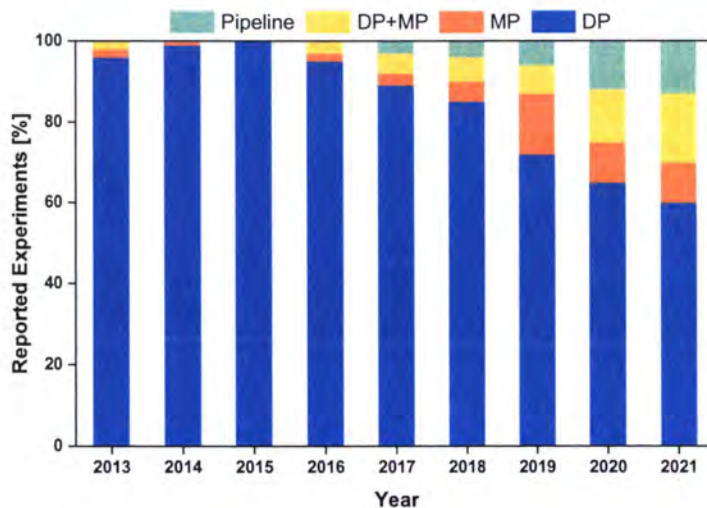


图 1.6 并行策略逐渐多样性。[4]

Figure 1.6 Progressive diversity in parallel strategies.[4]

子可以分布在通过网络连接的多台机器上。互连网络最重要的指标是延迟、带宽、和消息率。不同的网络技术提供不同的性能。例如，现代以太网和 InfiniBand 都提供高带宽，但 InfiniBand 具有明显的低延迟和更高的信息速率。专用 HPC 互连网络可以在所有三个指标中实现更高的性能。然而，机器间的网络通信通常

比机器内通信慢。随着网络模型的增大以及多种多样的神经网络的出现，前文提到的单一的数据并行模式已经无法满足性能的要求，不同的层可能倾向于不同的并行模式，例如卷积层倾向于数据并行，全连接层倾向于模型并行，而由于节点内和节点间的带宽的不同，Pipeline 并行 [9] 更适合用于节点间，模型并行对带宽的需求较高更适合节点内。如图1.6所示，随着训练节点规模的增加，现在训练使用的并行策略也越来越多。

1.2 并行深度学习训练所面临的挑战

1.2.1 大 batch size 训练

在上一节中，我们提到分布式 SGD 可以通过使用 mini-batch 来并发执行。然而，设置 mini-batch 大小本身就是一个复杂的优化空间，因为它同时影响到模型的测试精度（泛化）和硬件效率（利用）。如图1.7所示，mini-batch 不应该太小（区域 A），以便利用损失函数评估中固有的并发性；也不应该太大（区域 C），因为一旦增加到一定程度，模型训练的质量会下降。

我们可以通过将 SGD 与梯度为 L-Lipschitz 的函数 f 的下降定理结合起来，来证明区域 C 的存在：

$$\mathbb{E}_z [f(w^{(t+1)})] \leq f(w^{(t)}) - \eta_t \|\nabla f(w^{(t)})\|^2 + \eta_t^2 \frac{L}{2} \mathbb{E}_z [\|\nabla f_z(w^{(t)})\|^2] \quad (1.1)$$

其中 $z \sim \mathcal{D}$ 和 ∇f_z 是 z 的随机子梯度。这表明一个大的 minibatch（调整后的学习率）可以提高收敛率（负项），但同时也提高了梯度方差和学习率，导致最后一项阻碍了收敛。

人们从实验中一直观察到，使用大 batch size 导致泛化性能差，也就是说，用大 batch size 训练的模型在测试数据上表现很差。其中一个主要原因是，大 batch size 的训练趋向于训练函数的 sharp minima，而 sharp minima 往往泛化效果较差。而小 batch size 则倾向于平坦的最小值，从而导致更好的泛化。小 batch size 所提供的随机性可以帮助权重摆脱 sharp minima 的吸引。另外，用小 batch size 训练的模型被证明能在离起点更远的地方收敛。大批次的模型往往被吸引到离起点最近的最小值，缺乏小 batch size 的探索特性。当使用大 batch size 时，每个 epoch 数据的梯度更新数量会减少。这一点有时可以通过将学习率与 batch size 大小进行缩放来弥补。但仅仅使用较高的学习率会破坏训练的稳定性。另一种方法是延

长模型的训练时间，但这可能导致过度拟合。因此，分布式训练不仅仅是扩展到多个节点的问题。

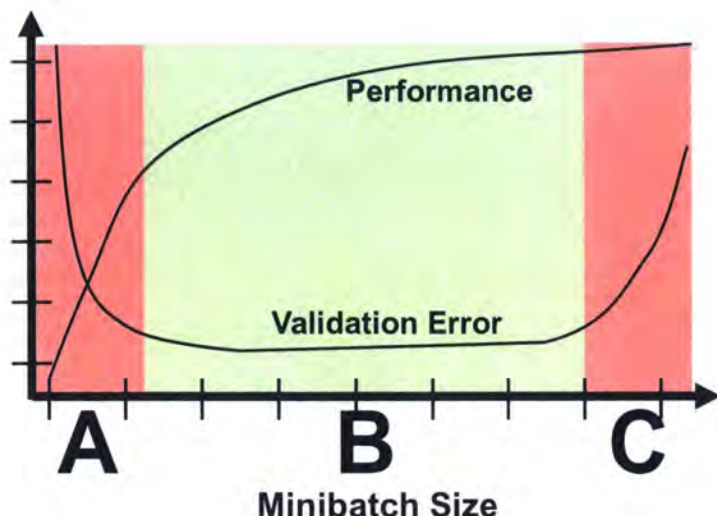


图 1.7 Batch size 对准确度和性能的影响。[4]

Figure 1.7 Minibatch size effect on accuracy and performance.[4]

1.2.2 基于计算图的显存优化

在模型不断增大的情况下，模型将无法放入单个 GPU 中，并且迫使程序员采用复杂的模型划分方法。而训练运行时设备真实的内存消耗是模型大小的多倍。我们首先在模型训练中分析现有系统的全部内存消耗情况，并将其分为三部分如图1.8所示：1) 对于大型模型，模型状态占据了大部分内存，其中包括优化器状态（例如 Adam 优化器的动量和方差，梯度和参数）。2) 剩余的内存被激活值，临时缓冲区和不可用的碎片内存占用，我们统称为剩余状态，3) 算子所需的额外内存空间。其中用于反向计算的中间结果值占用 70% 以上。

Imperative Runtime vs. Declarative Runtime：在深度学习训练框架下，神经网络的定义都是被抽象成计算图的形式，其中包括静态计算图模式和动态计算图模式。对于静态计算图模式 (TensorFlow, MXNet 等) 来说，该模式将计算图的构建和运行分开 (define and run)。在构建阶段，训练框架根据完整的计算流程对原始的计算图（即前面的静态计算图）进行优化和调整得到更省内存和计算量更少的计算图，这个过程称之为“计算图编译”。编译之后图的结构不再改变，也就是所谓的“静态”。在计算阶段，训练框架根据输入数据执行编译好的计算图

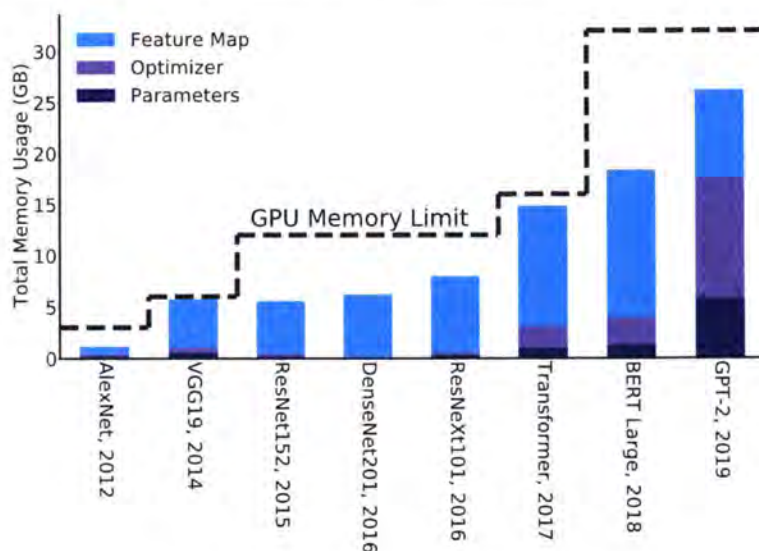


图 1.8 DNN 训练主要的内存开销来源。[3]

Figure 1.8 The main source of memory overhead for DNN training.[3]

得到计算结果。动态计算图 (PyTorch 等), 其核心特点是计算图的构建和计算同时发生 (define by run)。在运行时计算图中定义一个 Tensor 和算子时, 其值就已经被计算且确定了。这种模式在调试模型时较为方便, 能够实时得到中间结果的值。但是, 由于所有节点都需要被保存并且可以被访问, 这导致我们难以对整个计算图进行优化。由于静态图下持有全局信息, 可以提前做一些内存优化, 因此动态图模式下会消耗更多的内存, 优化会更有难度。如图1.9为了存储 x 、 w 、 p 、 b 、 y 五个变量, 动态计算图需要 40 个字节 (假设每个变量占用 8 字节的内存)。在静态图中, 由于我们只需要知道结果 y , 可以让 y 复用中间变量 p 的内存, 实现“原地” (inplace) 修改。这样, 静态计算图所占用的内存就减少为 32 个字节。

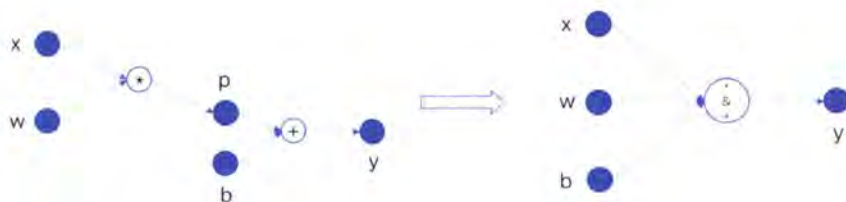


图 1.9 静态图上的优化。

Figure 1.9 Optimization on static computational graph.

1.2.2.1 动态计算图上的显存优化

随着深度学习网络朝着多样化发展，以前的网络都是以静态神经网络架构为主，即神经网络的架构在运行时是固定的，运行时不会发生变化，而现在算法人员设计出了各种各样的神经网络，静态计算图已经无法满足多样化的网络需求，虽然静态计算图可以带来很好的性能，但是对于算法研究工程师来说，静态计算图失去了易用性，debug 模型十分复杂。而动态计算图下可以灵活地设计网络以及方便地调试模型，因此动态计算图框架迅速崛起，由于动态计算图下无法获取全局信息，相比于静态计算图优化起来相对较难。与此同时，动态计算图上的显存优化显得尤为重要。

1.2.3 并行策略选取

先前的工作已经研究了并行化技术，以加快训练速度。最常见的方法是数据并行，它可以在每个设备上保留整个神经网络的参数副本，并为每个设备分配训练数据的子集。另一个常见的方法是模型并行，它将网络参数划分为不相交的算子子集，并在专用设备上训练每个子集。两种方法都将单个并行化策略应用于 DNN 中的所有层。但是，在 DNN 中，不同的层可能更倾向于不同的并行策略，以实现最佳的运行时性能。例如，具有数百万个参数的稠密连接层更倾向使用模型并行性来降低用于同步参数的通信成本，而卷积层通常更倾向使用数据并行性来消除来自上一层的数据重排。此外，某些层可能倾向更复杂的并行策略，例如在多个数据维度的混合并行策略。由于网络模型中不同层的特性不同，因此对所有层应用单个并行策略通常会导致运行时性能欠佳。除了不同网络模型层倾向于不同的并行策略，切分计算图的流水并行与切分算子的层内模型并行同样倾向于不同的硬件拓扑结构，pipeline 并行更适用于节点间通信带宽较低的情形，算子内的模型并行更适用于节点内通信带宽较高的情形。

如图1.10所示，通过定义更多维度的并行空间，在深度学习模型训练中使用更多并行策略，深度学习引擎可以自动在定义的搜索空间中为任意 DNN 查找快速并行化策略。现有方法仅考虑图中多个维度之一或子集。因此在自动并行优化下，每层张量的多维度的切分构成了一个很大的搜索空间，并且引入了大量的集合通信操作，并且不同的设备和网络拓扑需要不同的策略。在这种巨大的搜索空间下找到最优的切分策略或者次优的策略是个很具有挑战的问题。

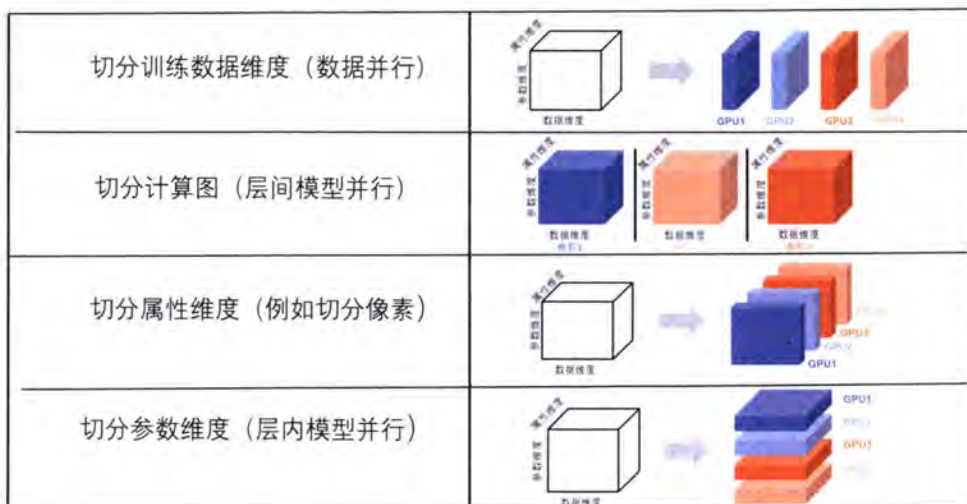


图 1.10 一维卷积运算中张量的划分方式。

Figure 1.10 The strategy to partition the tensor in one-dimensional convolution operation.

1.3 研究动机

随着深度学习数据量的不断扩增，batch size 大小是现代深度学习系统中需要调整的最重要的超参数之一。算法工程师经常希望使用较大的 batch size 来训练他们的模型，因为它可以通过 GPU 的并行性来提高计算速度。然而，众所周知，过大的 batch size 会导致泛化效果不佳（尽管目前还不知道为什么会这样）。对于我们试图优化的凸函数，在较小和较大的 batch size 的好处之间存在着固有的平衡。在极端情况下，使用相当于整个数据集的 batch size 保证收敛到目标函数的全局最优值。然而，这是以较慢的、经验性的策略收敛到该最优值为代价的。另一方面，使用较小的 batch size 已被经验证明是能更快地收敛到“好”的解决方案。这可以直观地解释为，较小的 batch size 规模允许模型“在看到所有数据之前就开始学习”。使用较小 batch size 的缺点是，模型不能保证收敛到全局最优值。它将在全局最优值附近跳动，停留在最优值的某个区域之外，其中停留的位置取决于批次大小与数据集大小的比率。因此，在没有计算限制的情况下，通常建议从小 batch size 开始，获得更快动态训练的好处，并通过训练稳步增加 batch size 大小，也获得保证收敛的好处。而如何尽可能的有效利用小 batch size 和大 batch size 混合训练，仍然是一个值得探索的问题。虽然已有的工作也在一定程度上缓解了大 batch size 的训练，但是引入了额外的超参，使得的手动超参

的调整反而是一种低效的解决方法。

另一方面随着模型的增大，内存墙问题是阻碍大模型研究的一个主要的问题，而随着更加灵活的大模型被研发，基于静态计算图的框架无法灵活表达动态网络，且无法友好地优化动态神经网络的内存。而动态图可以灵活地设计各种类型的模型，基于动态计算图的显存优化极为重要，由于动态计算图会消耗更多的显存，现在已有的动态图显存优化方法主要分为手动管理和自动管理内存中的张量，手动基于经验的管理张量忽略了内存和性能的 trade-off，而自动管理张量一方面是从单卡的角度考虑如何有效的驱逐运行时的张量，没有从多机的角度如何有效的管理张量，并且忽略了对内存碎片的考虑，另一方面，忽略了不同神经网络架构的特点，例如没有充分利用动态计算图下静态神经网络的静态特点。

随着网络模型的增大以及计算设备的增多，高效的大规模模型训练需要调整神经网络中单个算子的数据并行、算子并行和流水线并行方法的粒度。正确地调整并行策略已被证明 [10] 可以在训练性能方面带来数量级的改进，但这取决于强大的机器学习（ML）和系统专业知识。大规模模型的自动化并行将大大加快 ML 的研究和生产，使模型开发者能够快速探索新的模型设计而不考虑底层系统的挑战。不幸的是，它需要搜索一个复杂的策略空间，这个空间随着并行性的维度以及模型和集群的大小而呈指数级增长。例如，当所有的并行技术都被使用时，确定执行计划需要回答一系列相互依存的问题，如创建多少个数据并行副本，沿着哪个数据维度对每个算子进行划分，如何将模型分割成流水并行阶段。不同并行方法的相互作用以及它们对模型和集群设置的强烈依赖，形成了一个可供优化的策略组合空间。最近的自动并行模型训练 [11] 受限于单个空间模型并行方法，或依赖关于模型和集群规格的强假设。

1.4 本文主要工作和创新点

对于大 batch size 训练，我们发现训练过程的早期阶段对 batch size 大小很敏感，并推测不同的训练阶段需要不同的 batch size 大小。我们提出了一种用于大规模分布式 DNN 训练的可变 batch size 策略，并设计了一个新颖的有效学习率理论指导的自动调整引擎，用于可变 batch size 策略以自动调整超参数。我们验证了所提出的策略在 ShuffleNet、ResNet-50 和 ResNet-101 神经网络的大 batch size 训练上的泛化能力。我们在 CNN 训练期间成功地将 batch size 大小扩展到

120K, 而不会损失测试精度, 并且在 2048 块 GPU 128K batch size 上仅损失轻微的精度, 这表明可变 batch size 策略可以很好地支持大规模分布式训练。

MegTaiChi 内存管理模块是一个用于 DNN 训练的基于动态张量的内存管理优化模块, 它首先实现了张量划分和张量驱逐以及重新生成的高效协调。MegTaiChi 的关键特性是它根据运行时跟踪的动态张量访问模式做出内存管理决策。这种设计的动机是观察到在训练迭代期间对张量的访问模式是有规律的。基于识别出的模式, MegTaiChi 利用总内存优化空间, 实现启发式、自适应和细粒度的内存管理。实验结果表明, 与 DTR 相比, MegTaiChi 可以将 ResNet-50 的训练内存占用减少高达 11%, GL-base 减少 10.5%。对于 6 个具有代表性的 DNN 的训练, MegTaiChi 比 MegEngine 和 Sublinear 平均分别比最大 batch size 大小分别高出 5 $\times$ 和 2.4 $\times$ 。对于百万规模的人脸识别应用, 与 256 个 GPU 上的最优经验并行策略相比, MegTaiChi 实现了 1.8 $\times$ 的加速。

MAP 自动并行模块是一个用于自动生成并行策略的模块, MAP 提供一个全局视角的概念, 用于简化分布式训练。简单而言, 在 MAP 的全局视角下, 机群被抽象为一台“超级计算设备”。并描述了“超级计算设备”全局视角下的数据与机群中真实的物理设备上的数据的映射关系。我们采用和传统的数据和模型并行性不同的观点, 并将算子的并行化方法重新表达为一种全局抽象, 指定逻辑数据张量和相应物理张量之间的映射。通过对数据, 计算设备和算子的抽象, 我们对整个神经网络构造一个并行策略搜索空间, 通过构建整个神经网络对应的计算图的性能模型, 并利用贪心搜索算法, 搜索出每个算子的并行策略, 使得搜索出的整个神经网络模型的并行策略能够接近甚至超越手动设置的策略, 并且基本和现有的较优的手动模型并行库性能持平。

1.5 本文的组织结构

本文剩下组织结构如下: 在第 2 章中, 我们介绍了大规模并行深度学习训练的相关研究技术以及相关研究工作, 首先我们介绍了大 batch size 训练, 包括数据并行, 相关工作中已经提出的一些优化策略和技术, 以及现有的一些不足。然后我们介绍了在计算图上的显存优化相关技术以及相关优化工作, 其中包括静态计算图上的显存优化, 动态计算图上的显存优化, 现有的相关工作存在的一些问题与不足。最后我们介绍了目前常见的并行策略, 已有的手动并行策略, 半自

动并行策略以及自动并行技术相关工作和不足之处。

在第 3 章中，我们介绍了大 batch size 训练策略。我们首先分析了大 batch size 下模型训练的行为，包括参数的二范数，角动量的更新在训练初期波动很大。我们的一个猜想是，前期训练并不是如前人工作所说，大 batch 训练不是容易陷入局部最小，而是即使找到一个好的局域最小点，由于震荡的幅度过大，很难很快到达最小点，我们利用了小 batch size 的优点提出前期应该使用小的 batch size，并在此基础上提出了第一个算法：可变 batch size 算法，为了继续增大 batch size，以及在可变 batch size 算法中发现学习率和 batch size 的关系，我们进一步探索了有效学习率，并根据有效学习率设计出自动调参引擎 VBS 超参调优引擎，以便可以在训练中自动调整有效学习率。

在第 4 章中，我们介绍了动态计算图下的显存优化模块 MegTaiChi。我们首先列举了当前动态计算图下显存优化需要解决的问题，然后提出了 MegTaiChi 显存自动优化模块，包括如何规划动态图下多机张量切分的 DTP 组件，如何规划动态图下张量驱逐以及重新生成的 DTE 组件，如何针对静态神经网络做特别张量排布优化的 TMA 组件。通过三个组件在相关实验的联合优化，提高动态计算图下模型训练的性能。最后我们在主流的神经网络上做了相关测试和对比。

在第 5 章中，我们介绍了自动并行策略生成模块 MAP。我们首先借助开源深度框架已有的数据和算子的 SBP 分布式抽象并将其推广到更高的维度，其次根据计算图构建搜索空间以及代价模型，并应用图消去算法削减搜索空间，然后应用贪心搜索算法搜索每个算子的并行策略。我们还提出了相关的通信优化例如用于减少通信的通信代理节点组件，用于更精确的刻画代价模型的等待时间 overlap 算法以及采用更精确的通信代价，最后我们在主流的神经网络上做了相关测试和对比。

在第 6 章中，我们总结了全文，并展望了接下来的工作。在大 batch size 训练方面，我们将进一步深入探索大 batch size 训练的精度丢失原因，以及探索更精确的有效学习率。在动态计算图的显存优化方面，我们将重点解决动态计算图下的显存碎片问题，以及在运行时如何尽量减少打分的开销。在多机下的动态张量切分则是如何更系统性地给张量切分建模，提出更优的切分策略模型。在自动并行策略生成模块，目前我们只考虑了层内并行，即切分算子，未考虑 Pipeline 并行，未来将考虑加入 Pipeline 并行，同时我们会设计更精确的算子计算代价。

第2章 背景及相关工作研究概述

2.1 高效的大 Batch 训练

2.1.1 大 batch size 训练

在数据并行方法中，数据集被分割成 P 部分（一般等于 GPU 个数）存储在每个 GPU 上，每个 GPU 将有一份神经网络的本地拷贝和权重 (w^j)。在数据并行同步中，通信发生在两个地方：本地梯度求和和全局权重的广播。对于第一部分，每个 worker 独立计算本地梯度 ∇w^j ，并将更新内容发送给 master。master 在从 worker 那里得到所有梯度后然后更新 $\tilde{w} \leftarrow \tilde{w} - \eta/P \sum_{j=1}^P \nabla w^j$ 。这里的 η 是优化器的学习率。对于第二部分，master 将 $\tilde{w}$ 广播给所有 worker。

对于通信部分，还有另一种实现方式。1 个 reduce + 1 个 broadcast 操作可以由 1 个 allreduce 操作代替。在这种情况下，每个 worker 独立计算本地梯度 ∇w^j ，然后系统进行全局 allreduce 操作（即 ring allreduce 操作），将梯度之和 $\left(\sum_{j=1}^P \nabla w^j\right)$ 发送给所有节点。之后，每个节点将在本地进行权重更新 $\left(w^j \leftarrow w^j - \eta/P \sum_{j=1}^P \nabla w^j\right)$ 。在本章中，我们使用的是全局 allreduce，而不是 reduce + broadcast 方法。这种同步方法是大规模系统上广泛使用的方法。

将分布式同步 SGD 扩展到更多的加速器有两个挑战。第一个挑战是给每个加速器提供足够的工作负载，这在引言章节中已经讨论过了。第二个挑战是固有的问题即通信问题，即在处理每个局部 batch size 后，所有加速器必须在继续进行下一次迭代的计算之前通过阻塞的方式同步他们的梯度更新。这个问题可以通过重叠计算和通信来部分改善 [12]，但固有的同步开销仍然存在。打破这个同步的通信开销的一个更激进的方法是使用一个完全的异步方法。已经提出了多种异步方法 ([13, 14])。异步方法和同步方法中的通信和更新规则不同。最简单的异步方法是一个 master-worker 方案。在每一步，master 只与一个 worker 进行通信。master 从第 j 个 worker 那里获得梯度 ∇w^j ，更新全局权重，并将全局权重送回第 j 个 worker 那里。worker 的顺序是基于先到先得的策略。master 节点也被称为参数服务器 (parameter server)。参数服务器的想法被 Downpour SGD 方法 [15] 用于现实世界的商业应用中，该方法已经成功地扩展到 16,000 个核心。然而，Downpour 在 1,600 个核心上的性能对于一个全互连网络上的性能并不明

显优于单个 GPU[16]。以上提及的第二个挑战其实相比于传统的科学计算问题都是固有的问题，也有大量的相关研究工作。而数据并行在 DNN 训练上带来的新的问题如引言所说是精度丢失的问题。即过高的数据并行度带来大的 batch size，过大的 batch size 带来训练模型精度的丢失。

表 2.1 大 batch size 训练相关优化问题研究。

Table 2.1 A study of optimization problems related to training with large batch size.

工作	侧重点	技术路线	引用
Keskar2016	泛化问题	大 batch 容易陷入局部最优解	[17]
facebook2017	优化问题	可以保证精度，但是很难找到优化路径	[12]
Lars2017	分层优化	不同的层使用不同的学习率	[18]
LAMB2019	分层优化	不同的层使用不同的学习率	[19]
KFAC2018	二阶方法	利用梯度的二阶信息使用二阶优化器	[20]
Yao2018	二阶方法	利用二阶信息和对抗生成的方法	[21]

针对大 batch size 训练，工业界和学术界主要存在两种观点。第一个观点是大 batch 训练模型精度丢失的问题是一个泛化问题 [17]：如图2.1所示，大 batch 容易陷入局部最优解（即容易陷入 sharp minima），尽管在训练集上模型表现很好，但是很难泛化，在测试集上表现不好，最终导致模型具有高训练精度但是低测试精度。

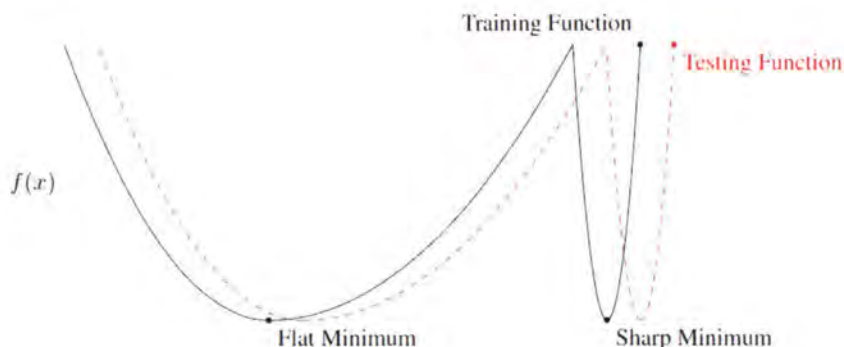


图 2.1 Flatten 区极小值和 Sharp 区极小值的概念草图。

Figure 2.1 Conceptual sketch of the flaten minima and the sharp minima.

第二个观点是大 batch 训练模型精度丢失的问题是一个优化问题 [12]：可以

保证精度，但是很难找到优化路径。如图2.2所示，Facebook 使用了 256 块 GPUs (单卡 batch size 32) 将 resnet50 训练 ImageNet-1K 的 batch size 扩大到 8K。但是当 batch size 扩大到 8K 之后，所使用的优化方法就失效了。

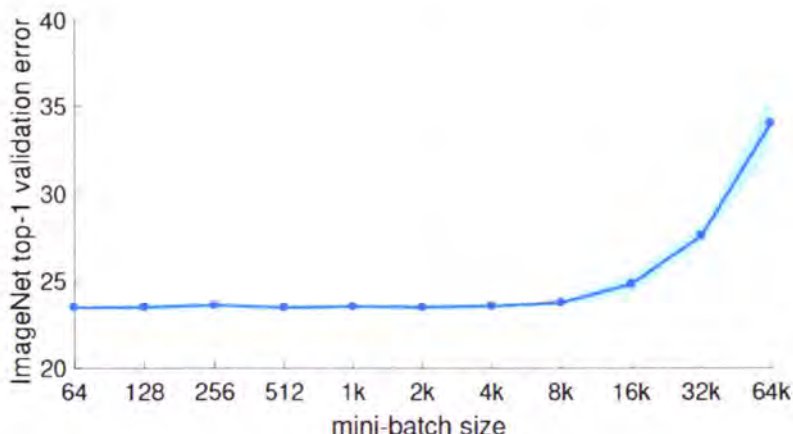


图 2.2 Facebook 的 ImageNet 训练 top-1 测试精度与 batch size 大小的关系。

Figure 2.2 Facebook's ImageNet training top-1 test accuracy versus batch size.

在表格2.1中我们总结了前人的工作，在一阶方法上这些工作主要停留在调整学习率，当增加 batch size 时，我们需要同时增加初始基本学习率 LR ，以防止丢失精度。有两个增加基本 LR 的规则：Sqrt 缩放规则 [17]。当我们将 batch size 增加 k 倍时，应将 LR 增加到 $\sqrt{k}$ 倍，以使得梯度期望值的方差保持恒定。线性比例缩放规则 [12]。当我们将 batch size 增加 k 倍时，应基于 $\nabla l(x, y, w_t) \approx \nabla l(x, y, w_{t+j})$ 的假设将 LR 增加 k 倍，其中 $j < b$ 。预热方案通常使用线性比例尺规则，如果 $k\eta$ 过大，一开始可能会使训练有所发散。因此，人们将初始 LR 设置为较小的值，并在几个时期（例如 5 或 10 个 epoch）内将其逐渐增大为 $k\eta$ ，此方法称为渐进预热 (warmup) 方案。另一种方法称为恒定预热方案，该方法在前几个时期使用恒定的小 LR 。[12] 表明，对于 ResNet-50 训练，渐进预热要比恒定预热好。[22] 表明，无论 batch size 如何， LR 都应有一个上限。我们的实验结果与这一发现相符。[23] 在他们的实验中，当将 batch size 从 1600 增加到 6400 时，也使用了线性缩放 LR 。但是，他们没有显示小 batch size 基线的准确性。而 [18] 提出了 Layer-wise 的自适应学习率 (LARS) 算法。LARS 认为不同的网络层应该使用不同的学习率，而这个学习率应该是自适应的 $\eta = l \times \gamma \times \frac{\|w\|_2}{\|\nabla w\|_2}$ ，LARS 认为神经网络层的参数的二范数越大，那么相对学习率也应该越大，即参数越大的层更

新的步长应该越大，参数较小的网络层更新步长应该越小。虽然 LARS 是自适应的但是也引入了额外的超参数。超参的调整是经验式的，且耗时的。而还有一些工作则采用二阶优化的方法，利用求解二阶梯度信息 Hessian 矩阵设计二阶优化器例如 K-FAC[20]，但是 Hessian 矩阵的求解计算量大，影响了训练速度，而实验表明二阶方法只能保证收敛快，并不能保证模型精度。

现有研究的不足之处：未充分利用小 batch size 的优点，可以在训练初期尽可能的利用小 batch size 训练；Layer-wise 的学习率调参引入额外超参数，需要手动调整超参，可以探索自动调参。

2.2 大模型：显存优化

2.2.1 计算图 (Computation Graph)

我们通过一个简单的数学表达式 $y = (w * x) + b$ 来介绍计算图的基本概念，如下图所示：

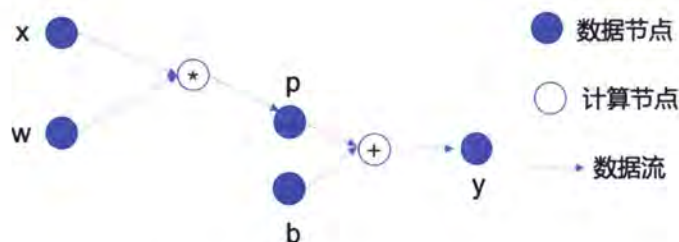


图 2.3 神经网络模型的计算图表达。

Figure 2.3 Neural network model computational graphical representation.

从图2.3中我们可以看到，计算图中存在：数据节点(图中的实心圈)：如输入数据 x 、 w 、 b ，运算得到的中间数据 p ，以及最终的运算输出 y ；计算节点(图中的空心圈)：图中 $+$ 和 $*$ 分别表示计算节点加法和乘法，是施加在数据节点上的运算；边(图中的箭头)：表示数据的流向，体现了数据节点和计算节点之间的依赖关系；图2.3便是一个简单的计算图示例。计算图是一个包含数据节点和计算节点的有向图(可以是有环的，也可以是无环的)，是数学表达式的形象化表示。在深度学习领域，任何复杂的深度神经网络模型本质上都可以用一个计算图表示出来。

对于训练过程中显存的优化也通常是在计算图上或者运行时库规划如何释

放和重新生成中间节点，前人的工作主要通过两种方式节省内存或者扩充内存：利用计算换显存和通信换显存。

2.2.2 运行时库上的显存优化

交换到 Host 内存（虚拟化）：vDNN ++[24] 充当运行时内存管理器，用于虚拟化 DNN 的内存使用情况，以便在 DNN 无法放入 GPU 内存时同时对 GPU 和 CPU 内存中的 DNN 算子进行训练。但是在每一层的末尾，计算的同步和数据的切入/切出在某种程度上会导致 vDNN ++ 效率低下。ooccuDNN 库是 Nvidia DNN 库（cuDNN）的扩展。即使层数超过 GPU 内存容量，它也使用 out-of-core（将数据交换到 CPU 端）方法来应用 cuDNN 兼容的算子。在 ooccuDNN 中，单个计算图层的特征图可以划分为以下三个维度中的任意一个：batch, channel 和 filter。这意味着 ooccuDNN 中的张量交换仅限于单层范围。同时交换到内存的方法对带宽要求很高，尤其是多机训练时。

2.2.3 静态计算图上的 Checkpointing

Checkpointing 即只保存神经网络运行时产生的部分中间值，checkpoint 之间的中间值会被释放掉或者拷贝到 CPU 端。不同于运行时库，重计算 [23] 是一种基于计算图的方法，如图 2.4，它通过在前向计算时释放一些中间值，并在反向传播中从检查点的激活值冗余地重新计算出当前释放的中间值来使较大的模型或者 batch size 容纳到显存中（以增加计算时间为代价）。如图 2.5，swapping 和重计算相似，它通过在前向计算时将中间值拷贝到 CPU 端，在反向计算时通过重新拷贝来使较大的模型或者 batch size 容纳到显存中。从定义上讲，性能是以内存容量为代价的。另外，在特殊的递归方案下，对于 $\mathcal{O}(\sqrt{N})$ (or $\mathcal{O}(\log_2 N)$) 的 N 层模型，梯度检查点在内存消耗上有较低的限制，因此可能不常用。虽然可以训练更大的模型，但仍然受到内存需求的限制。静态图上的 checkpointing 策略是在计算图编译阶段就生成好了一个 plan，即哪些算子的中间结果值需要自动被释放或者拷贝到核外。这个 plan 在运行时是确定的，不会发生改变。

将中间值交换到 CPU 内存端并配合重新计算策略：SuperNeurons[25] 和 Capuchin[26]。SuperNeurons 使用一种简单的方法根据目标层组合 swapping 和重新计算，例如卷积层的激活值被换出，而 BatchNorm 层被重新计算。但是，这并未考虑训练期间的实际执行时间和内存使用情况。Pooch[27] 扩展了一个 DL 框架来

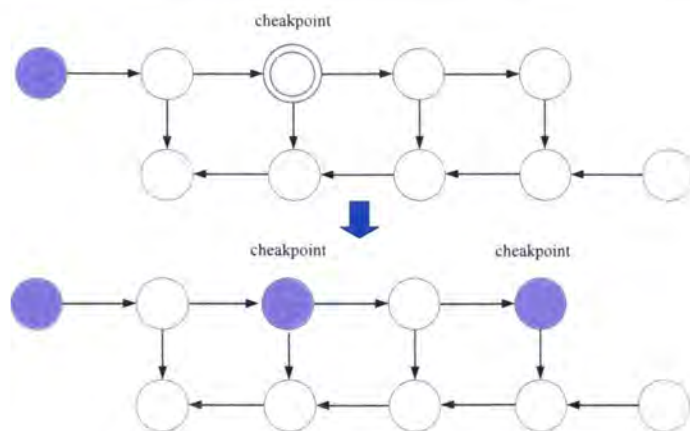


图 2.4 静态计算图上的 Checkpointing 策略 (重计算)。

Figure 2.4 Checkpointing strategy (recomputation) on static computational graphs.

实现 out-of-core (swapping) 的 DNN 训练, 并通过调度的方法混合激活 swapping 和重新计算两种策略。PooCH 根据给定模型中算子的分类来决定调度, 但交换的目标层或重新计算的目标层是根据运行时 profiling 确定的。Capuchin[26] 在运行时动态跟踪提取张量访问模式。它结合了 swapping 和重新计算功能, 并且以 swapping 为主要的优化策略, 因为 Capuchin 认为 swapping 的开销可以被计算重叠。

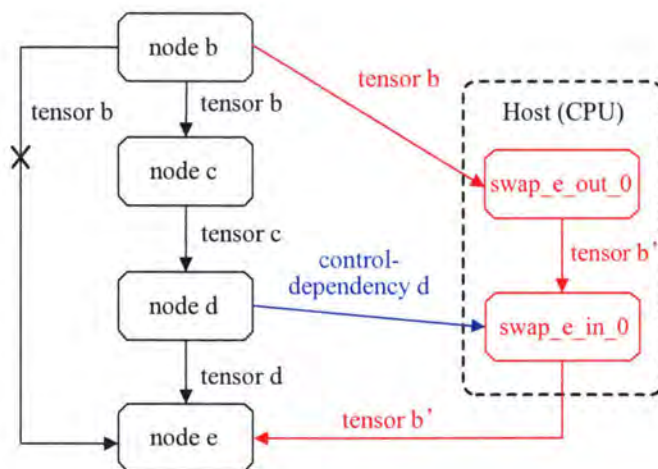


图 2.5 静态计算图上的 Checkpointing 策略 (swapping)。

Figure 2.5 Checkpointing strategy (swapping) on static computational graph.

2.2.4 张量切分

当单个设备内存容量无法容纳整个模型时,在不同的 GPU 上切分模型是避免内存容量限制的方法,因此模型并行正越来越受到关注。通过拆分任何类型的 DNN 的任何维度来显式实现模型并行。模型并行方法是侵入性的,需要根据模型张量的切分方式,逐例更改模型的实现方式。此外,使用模型并行将要使用的 GPU 的最小数量与模型大小联系在一起。例如,威震天 LM 模型 [28] 至少需要内存容量为 16GiB 的 16 个 GPU 才能容下。即使是为减少超大模型的 GPU 数量而进行了高度优化的解决方案,仍然需要数十个或数百个 GPU。模型并行化方法是复杂的,并且在大型模型的最小数量的 GPU 上施加了不小的限制。隐式模型并行方法在 GPU 上划分数据流图。这种纯 DAG 方法的局限性在于扩展到多节点的复杂性,因此 DAG 解决方案到目前为止仅限于单个节点。另外,调度 DAG 是具有 $O(V^2)$ 复杂度的 NP 难题,因此基于该方法的工作显示了大量的时间和资源开销。除此之外,基于张量切分的不同模型并行方式所占用的显存消耗也是不同的如图 2.6 所示,不同的切分方式会引入不同的集合通信,不同的集合通信会消耗不同大小的显存,当前已有的工作都忽略了不同的张量切分方式对显存占用的影响。

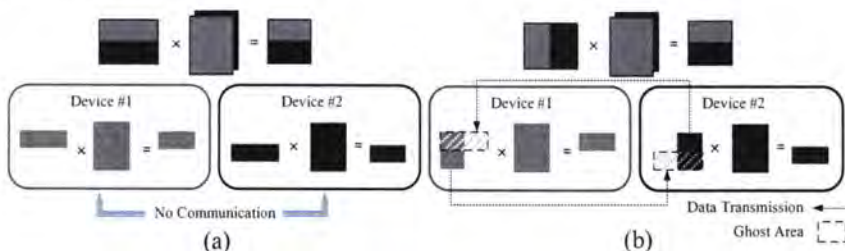


图 2.6 基于不同张量切分策略的并行矩阵乘运算。

Figure 2.6 Parallel matrix multiplication operations based on different tensor cut strategies.

2.2.5 动态计算图上的 Checkpointing

表 2.2 总结了前人在显存优化上的工作,在大模型或者大 batch size 上的显存优化主要基于静态计算图即 offline,而动态计算图上的显存优化则是基于运行时才制定 checkpointing plan 如 2.7 所示是 online 的,当前已有的工作中 DTR[1] 工作提出了在动态计算图上做自动 Checkpointing (Zero[34] 虽然支持动态计算图但是是手动指定的策略,非自动的策略,手动策略对 trade-off 显存和性能是不友好

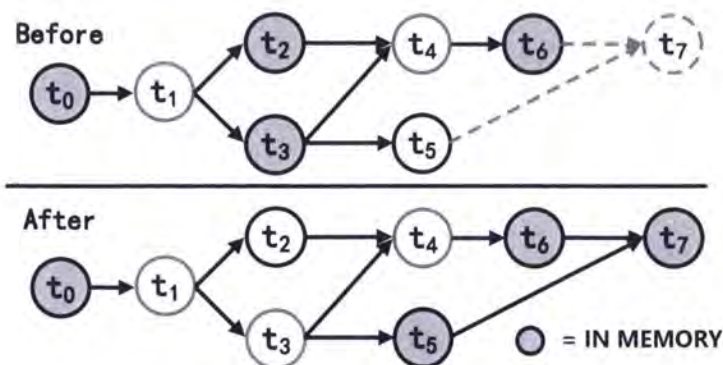


图 2.7 动态计算图下的 Checkpointing 策略。

Figure 2.7 Checkpointing strategy under dynamic computational graphs.

的)。而随着深度学习技术的发展，神经网络呈现出多样性以及灵活性的特点，越来越多的框架引入了动态计算图模式，但是在动态图上的显存优化的工作却很少，并且基于动态图的 DTR 仍存在优化空间。例如，DTR 忽略了内存碎片问题，并且未考虑不同模型的特点做优化。同时，动态图上的优化是相对于静态图优化的难度也很大，而静态图上的优化只支持静态神经网络。动态图仍有很大的优化空间：基于张量切分的大模型训练的相关研究较少（复杂的集合通信，建模困难），缺少动态图下 checkpoint 和基于张量切分联合设计的自动调优框架。

现有研究的不足之处：目前已有的工作大部分只支持静态计算图，而静态计算图上的显存优化无法满足当前的模型发展；动态计算图下的优化不支持多机；动态计算图下容易产生内存碎片；动态计算图下未考虑模型架构的特点。

表 2.2 内存优化模块相关工作对比 (M 和 A 表示手动和自动)。

Table 2.2 The related work to memory optimization (M and A for manual and automatic).

工作	计算图	评估	交换	重计算	多节点	动态模型	模型并行	碎片整理	引用
vDNN++	N/A	online	✓(M)	✗	✗	✗	✗	✗	[24]
SuperNeuron	N/A	online	✓(M)	✗	✗	✗	✗	✗	[25]
SwapAdvisor	static	offline	✓(A)	✗	✗	✗	✗	✗	[29]
Sublinear	static	offline	✗	✓(A)	✓	✗	✗	✗	[30]
Checkmate	static	offline	✗	✓(A)	✗	✗	✗	✗	[31]
Capuchin	static	online	✓(A)	✓(A)	✗	✗	✗	✗	[26]
DTR	dynamic	online	✗	✓(A)	✗	✓	✗	✗	[1]
Pipeline	static	offline	✗	✓(A)	✓	✗	Inter layer	✗	[32]
Flexflow	N/A	None	✗	✗	✓	✗	Intra layer	✗	[33]
Zero(updated)	dynamic	offline	✓(M)	✓(M)	✓(M)	✗	Inter layer	✗	[34]
Gshard	static	none	✗	✗	✓	✗	Intra layer	✗	[35]
KARMA	static	offline	✓(M)	✓(M)	✓	✗	Inter layer	✗	[36]

2.3 并行策略

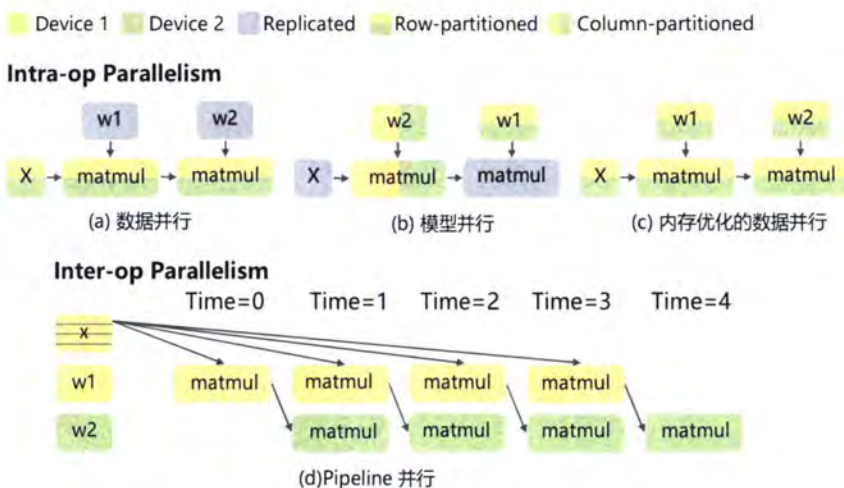


图 2.8 相同的计算结果下不同的并行策略。

Figure 2.8 Different parallel strategies for the same computational result.

2.3.1 数据并行

图2.8(a) 为数据并行, 训练数据被划分到分布式 worker 中, 但模型是复制的。每个 worker 在其独立的数据分块上计算梯度和参数更新, 并在权重更新前与其

他 worker 同步梯度，以便所有 worker 在整个训练中观察到一致的模型参数。

2.3.2 模型并行

当模型太大，无法装入一个设备时，算子内并行是一种有效的模型并行选择。算子内并行是指划分特定算子（下文中缩写为 OP），如图2.8(b) 中的 matmul 算子。沿着非 batch 的轴线切分，在多个设备上并行计算算子内的每一部分。由于输入张量是联合分区的，当一个设备计算其运算分区时，输入张量的所需部分可能不在其本地内存中。因此，需要通过通信从其他设备获取输入数据。当张量被均匀分区时，即 SPMD 并行，所有设备都遵循相同的集合通信模式，如 Allreduce, Allgather, Alltoall。

2.3.3 Pipeline 并行

如图2.8(d) 所示，Pipeline 并行不是对操作进行分区，而是将模型对应的计算图中的不同算子组（称为阶段）放在不同的工作器上，同时，它将训练 batch size 分成若干 macro batch size，并在分布式 worker 上对 macro batch 的前向和后向进行 pipeline 处理。与算子内并行不同，Pipeline 并行使用点对点通信，在不同的 worker 之间传输前向和后向的中间激活值。

表 2.3 并行策略优化相关工作。

Table 2.3 Work related to parallel strategy optimization.

工作	侧重点	技术路线	引用
One weird trick	并行策略	不同的层适用不同的并行策略	[37]
ColocRL	策略搜索优化	构建搜索模型解决算子的放置问题	[38]
FloexFlow	张量切分	抽象并行空间并勾勒了自动并行的基本框架	[39]
Tofu	算子层内并行	提出了一套 DSL，方便开发者描述张量的划分策略	[41]
GSHARD	XLA 图编译器半自动	把计算图 lower 到 IR 层 (中间表示层)，在 IR 层组合切分策略	[35]
GSPMD	XLA 图编译器半自动	把计算图 lower 到 IR 层 (中间表示层)，在 IR 层组合切分策略	[40]
WHALE	XLA 图编译器半自动	把计算图 lower 到 IR 层 (中间表示层)，在 IR 层组合切分策略	[41]

2.3.4 自动并行

深度学习模型的快速发展，DNN 模型变得越来越大，训练起来的计算量也越来越大。因此，现在的标准做法是在分布式异构集群之间并行进行 DNN 训练。尽管 DNN 模型和用于并行化它们的集群越来越复杂，但是当今的深度学习框架用于并行化训练的策略仍然很简单，而且常常不是最优的。最早 [37] 提出不同

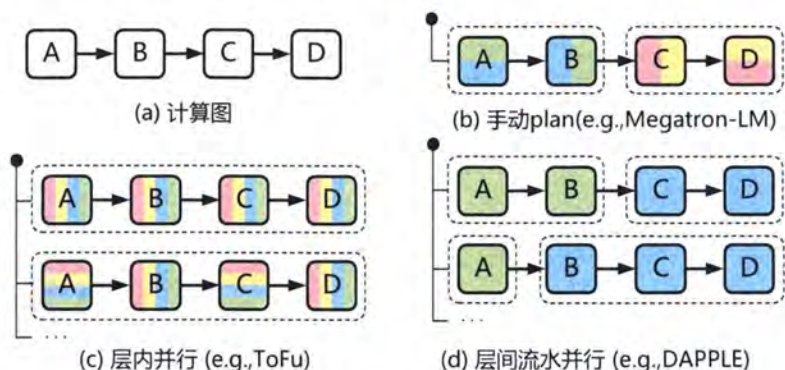


图 2.9 手动并行和自动并行。

Figure 2.9 Manual and automatic parallelism.

的神经网络层应该使用不同的并行策略。最常见的并行化策略是数据并行和模型并行，数据并行放置了整个神经网络的参数副本在每个设备上，以便每个设备处理训练数据的子集，并在迭代结束时在副本之间同步网络参数。数据并行对于具有少量可训练参数（例如卷积层）的计算密集型算子非常有效，但在具有大量参数的算子上却表现出次优并行性能。另一个常见的并行化策略是模型并行，该模型并行将神经网络的模型参数分别分配给指定的加速器。这种方法消除了设备之间的参数同步，但是需要在算子之间进行数据传输。ColocRL 等 [42] 使用强化学习为模型并行性学习有效的算子分配，但仅在算子的维度上探索并行性。OptCNN[39] 使用分层并行机制将 CNN 与线性计算图并行化。OptCNN 使用动态编程共同优化如何并行化每个算子，但并未考虑跨不同算子的并行性。此外，OptCNN 不适用于许多用于语言建模，机器翻译和推荐的 DNN 模型，这些 DNN 往往是 RNN 或其他非线性网络。在 OptCNN 的工作中，介绍了针对 DNN 的并行化策略的综合 SOAP（样本-算子-属性-参数）搜索空间，该搜索空间概括并超越了先前的方法。算子维度描述了 DNN 中不同算子如何并行化。对于单个算子，样本和参数维度大小指示训练样本和模型参数如何在设备之间分布。最后，属性维度定义了样本中不同属性的划分方式（例如，图像的高度和宽度维度）。在 FlexFlow[39] 中定义和使用 SOAP 搜索空间，它是一种深度学习引擎，可以在 SOAP 搜索空间中自动为任意 DNN 查找快速并行化策略。现有方法仅考虑 SOAP 维度之一或子集。例如，数据并行性仅探索样本维度，而 OptCNN 并行化样本，属性和参数维度中的线性 CNN。FlexFlow 考虑在所有 SOAP 维度

上并行化任何 DNN 网络模型（线性或非线性），并探索更全面的搜索空间，其中包括作为特殊情况的现有方法。FlexFlow 最早提出自动并行的基本框架，如表2.3和图2.9所示，之后 Tofu 提出一种 DSL 语言用于并行化任何算子并提供了和 FlexFlow 同样的搜索空间。但是 Tofu 需要在 DSL 里描述 operator 的实现方式，如果能描述 tensor 的划分上更好一些。而 GSHARD, GSPMD 以及 WHALE 则是将算子的切分表达 lower 到 XLA 图编译器层，但是只提供半自动并行，未提供策略搜索。Tofu, GSHARD 以及 WHALE 特别得限定了“partition and reduce”的模式。“partition and reduce”也包含一些缺陷，例如对张量切分之后，不见得立刻去最张量的 reduce, 可以让中间的 partial 结果在系统里继续参与计算，这样反而有可能效率更优。

现有研究的不足之处：目前全自动并行缺乏原生算子级别的切分策略抽象表达；大部分还是半自动并行；搜索空间表达不够全面, 例如缺乏高维的并行表达；缺乏细粒度性能建模以及优化。

在接下来的章节中，我们针对本章提出的三个相关问题的已有工作的不足和缺陷，分别详细的介绍本工作提出的三个解决方案。

第3章 可变 batch size 训练策略 VBS

3.1 问题描述

为了加快 DNN 训练过程，分布式数据并行计算技术是现在深度学习应用模型训练的必用技术，每块 GPU 持有相同的完整模型副本每块 GPU 读入不同的 Mini-batch 数据。目前，大多数研究人员选择分布式随机梯度下降（SGD）方法进行训练。梯度聚合和参数更新顺序执行。近两年来，人们对大 batch size 训练的研究兴趣越来越大，试图增加步长，减少参数更新的次数。一般来说，根据分布式 SGD 算法，batch size 越大，随机梯度就越接近全局梯度。不幸的是，增加 batch size 往往会降低 DNN 的测试精度 ([12, 17, 37]) 总结了 batch size 从 128 增加到 1024 时测试误差的变化（采用了线性缩放规则）。我们在图3.1中的实验显示了大 batch size 训练的一般情况。单卡 batch size 32，8 卡 GPU batch size 大小为 256 的 ResNet-50 的基线测试精度为 75.4%，对于 batch size 为 16K (512GPUs) 和 32K (1024GPUs)，测试精度分别下降到 73.6% 和 70.6%。此外，batch size 大小的增加导致了额外的精度损失。随着训练节点的扩大，既随着 batch size 越来越大，预训练模型的准确性显著下降，过大的 batch size 造成收敛的困难，而过小的 batch size 造成硬件资源利用不足，这将抵消并行 SGD 的好处。

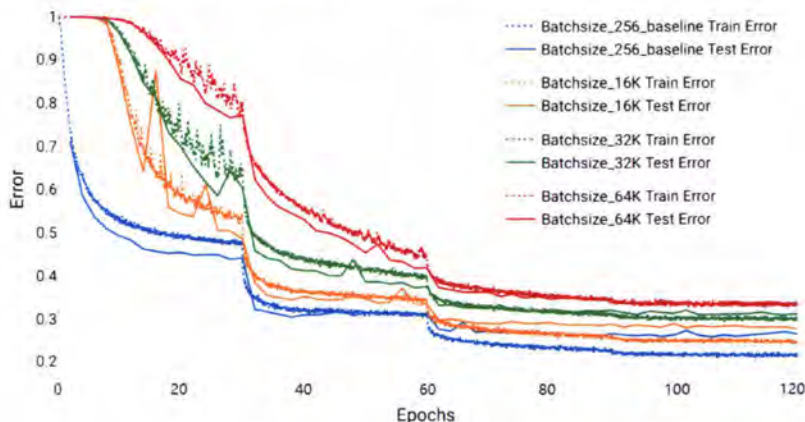


图 3.1 随着节点数和 batch size 的增加，训练精度不断降低。

Figure 3.1 As the number of nodes and batch size increase, the training accuracy decreases.

3.2 可变 batch size 训练策略

3.2.1 CNN 训练行为分析

根据前人的工作，对于 SGD 的收敛来说，在训练过程中的每个迭代步骤中改变学习率是很重要的。然而，对于大 batch size 训练，大多数现有的调整超参数的方法可以保持测试精度不丢失，直到 batch size 增加到 8K。最有说服力的解释之一是：小 batch size SGD 引入了噪声行为，允许从 sharp 的局部最小值 [17] 中逃脱，而大 batch size SGD 具有较少的随机性，导致失去了这种优势。例如，Keskar 推测大 batch size 训练往往会陷入 sharp 的局部最小值。下面本文从基于二阶 Hessian 矩阵特征值的视角 [43] 验证 Keskar 的猜想。图 3.2 显示了 Hessian 矩阵在不同 batch size 下的前 15 个最大特征值。显然，较大的 batch size 具有较大的主特征值以及其他 14 个特征值，这表明随着 batch size 的增加，目标函数变得更加陡峭和尖锐。因此，大 batch size 训练的优化轨迹很难达到平坦的最小值。

近来，相关研究发展了数学理论来理解 SGD 的行为。受 DNN 训练的球形运动动力学 [44] 的启发，本文还将训练过程映射到一个球面上。在这种情况下，超参数搜索被限制在一个球体上，训练步长可以通过更新角度的大小来衡量。图 3.3 分别显示了使用大 batch size 和小 batch size 的角度更新值。很明显，与小 batch size 训练相比，大 batch size 训练涉及更大的角度更新均值和更大的方差，这在训练的早期更新步骤中很明显。它表明，即使能找到正确的平坦最小值，大 batch size 训练也很难快速收敛到损失函数底部最小值。

此外，基于非线性优化的迭代理论，为了可视化早期 epoch 对整体训练的影响，图 3.4 显示了在不同 batch size 的训练过程中不同层的权重范数的变化。在 ResNet-50 训练中，本文观察到对于神经网络中的每一层，batch size 越大，前 30 个 epoch 内的权重范数曲线变化越剧烈，这表明 batch size 显著影响早期的迭代步骤。然而，在 30 个 epoch 之后，每层的权重范数值趋于稳定，并且波动很小，这意味着随着寻找最小值的过程趋于稳定。这种权重范数的变化趋势与球体上角度更新值的变化一致（图 3.3）。与 multi-steps 学习率策略类似，本文将训练过程分为几个阶段，每个阶段由几个 epoch 组成。例如，30 epochs 在图 3.5 中形成一个阶段。本文上面的发现表明，第一阶段不仅为后面的阶段提供了输入，而且还决定了后期搜索过程的主要范围，其中每一层的权重范数没有发生剧烈变化。

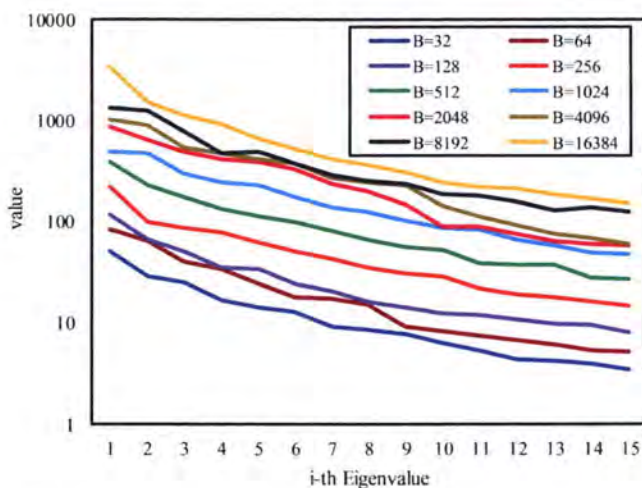


图 3.2 Hessian 矩阵 top-10 特征值随着 batchsize 的增大的变化。

Figure 3.2 The top-10 eigenvalues of hessian matrix with increasing batch size.

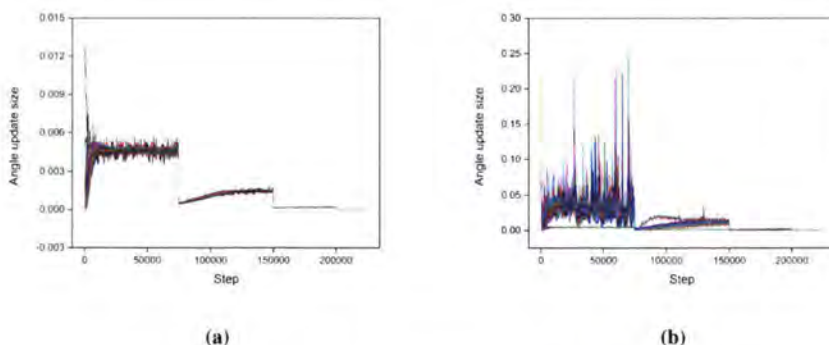


图 3.3 大 batch size 和小 batch size 下的角度更新值。

Figure 3.3 The angle update value when using large batch size and small batch size.

3.2.2 不衰减学习率，增大 batch size

基于上面的分析，在第一阶段对 SGD 使用小 batch size，然后在后期阶段改变 batch size 是合理的。在图 3.6 中，第一阶段的 batch size 固定为 8K，而后期使用更大的 batch size，例如 16K 或 32K。从第二阶段到最后，很明显，当使用第一阶段的相同输入时，不同 batch size 的训练误差曲线几乎是相同的，这再次证明了 batch size 在后期几乎没有影响的事实。此外，随着 batch size 的增加，不同 batch size 测试误差曲线的轮廓几乎保持不变。换句话说，模型达到了相同的最终测试精度，甚至可以优于基线。如图 3.5 和 3.6 所示，第一阶段的小 batch

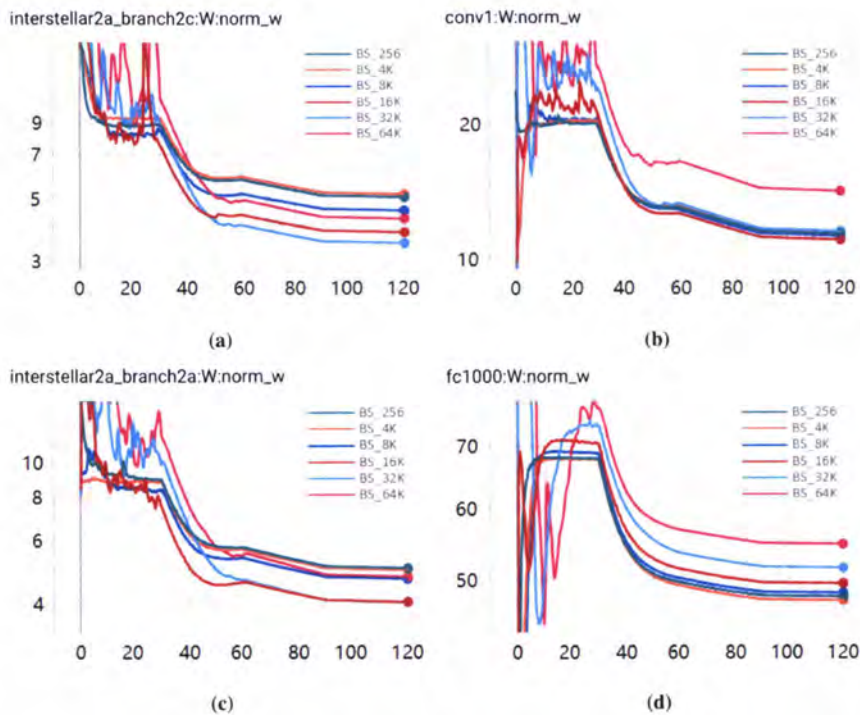


图 3.4 ResNet-50 四个网络层训练中 $\|w\|$ 的变化。

Figure 3.4 $\|w\|$ variation of four network layers in ResNet-50 during the training

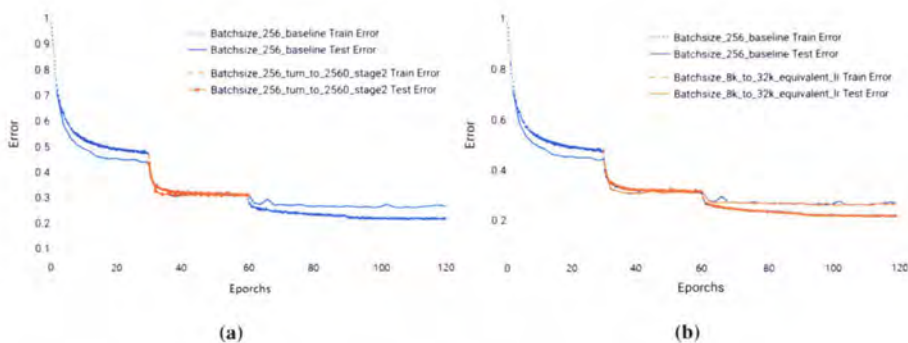


图 3.5 不衰减学习率，增大 batch size。

Figure 3.5 No decay learning rate, increase batch size.

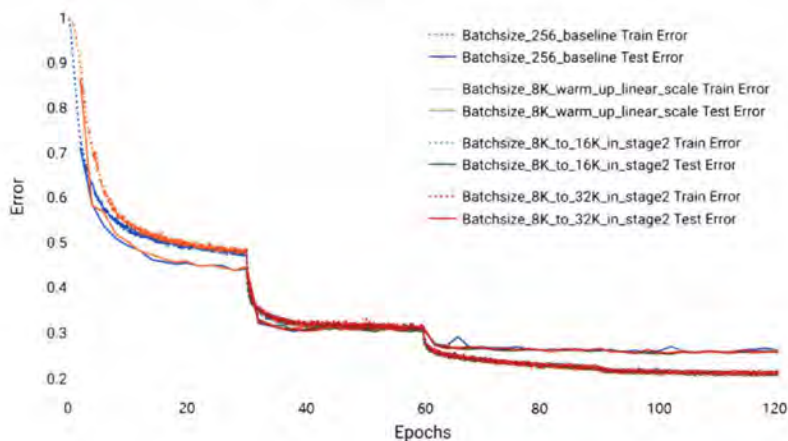


图 3.6 前期使用 8K batch size，后期使用 16K，32K。

Figure 3.6 8K batch size in the early stage, 16K, 32K in the later stage.

size 训练对最终测试的准确率影响很大，而在后期，训练过程对 batch size 不敏感。当第一阶段的 batch size 超过 8K 时，优化轨迹趋于达到局部最小值，并且模型的测试精度很差，无论在后期阶段花费多少计算工作都难以提高。

为了确保高测试精度并提高大 batch size 训练的并行性能，本文提出的设计如算法 1 所示。首先，训练过程分为几个阶段，每个阶段由几个 epoch 组成。例如，第 i 阶段有 n_i 个 epochs ($n_i \geq 1$)。其次，不同阶段的设置不同。因为第一阶段比其他阶段对 batch size 更敏感，小的 batch size 在训练初期可以充分利用随机性，不但可以帮助跳出 sharp minima 还可以使用较小的学习率快速到达合适的损失函数收敛位置，所以最好在第一阶段选择一个小的 batch size 来为第二阶段寻找一个好的输入。第三，从第二阶段到训练结束，可以使用大 batch size。基于方程 (3.2)，当 batch size 增加时，应该保持 $\frac{\eta}{|B_i|}$ 的值固定。如图 3.6 所示，从第 30 个 epoch 到第 60 个 epoch，本文增加了 batch size 但保持学习率不变。根据方程 (3.2)，batch size 的增加会导致每次迭代的步长减小，而步长太小会导致训练过程效率降低。因此，当第一阶段后 batch size 变大时，其他参数（例如权重衰减）需要适当调整以避免每次迭代中的步长过小。在下一节中，本文提出了一个自动调整引擎来在 batch size 发生变化时调整这些超参数。需要注意的是，可变 batch size 策略成功地结合了小 batch size 训练和大 batch size 训练，同时充分利用了它们各自的优势。

算法 1 Variable Batch Size Strategy**Require:** 样本数 N , 学习率 η , Batchsize B_{small} and B_{large} , Momentum m , Weight decay λ , Weight w_0 ;**Ensure:** Weight decay $w_{t_{end}}$ ($t_{end} = N \sum_{i=1}^E n_i$);1: $v_{-1} = 0$;2: $t = 0$;

3: // 更新第一个阶段;

4: $|B_t| = B_{small}$ ($t = 0, 1, \dots, Nn_1 - 1$);5: // 第一个阶段包括 n_1 个 epochs;6: **for** $\Delta e = 0$ to $n_1 - 1$ **do**7: // 一个 epoch 需要 $\lceil N/|B_{small}| \rceil$ 次迭代;8: **for** $\Delta t = 0$ to $\lceil N/|B_{small}| \rceil - 1$ **do**9: $t = t + \Delta t + M \cdot \Delta e$;10: $g_t = \frac{1}{|B_t|} \sum_{(x,y) \in B_t} \nabla l(\hat{y}(x, w_t), y) + \lambda w_t$;11: $v_t = mv_{t-1} + \eta g_t$;12: $w_{t+1} = w_t - v_t$;13: **end for**14: **end for**

15: // 更新第二个阶段

16: $|B_t| = B_{large}$ ($t = Nn_1, \dots, N \sum_{i=1}^E n_i - 1$)17: **for** $i = 2$ to E **do**18: // 第 i -th 个阶段包括 n_i 个 epochs;19: **for** $\Delta e = 0$ to $n_i - 1$ **do**20: // 一个 epoch 需要 $\lceil N/|B_{large}| \rceil$ 个迭代;21: **for** $\Delta t = 0$ to $\lceil N/|B_{large}| \rceil - 1$ **do**22: $t = t + \Delta t + M \cdot \Delta e$;23: $g_t = \frac{1}{|B_t|} \sum_{(x,y) \in B_t} \nabla l(\hat{y}(x, w_t), y) + \lambda w_t$;24: $v_t = mv_{t-1} + \eta g_t$;25: $w_{t+1} = w_t - v_t$;26: **end for**27: **end for**28: **end for**29: **Return**;

3.3 基于有效学习率的超参调优引擎

在本小章节，我们考虑在算法 1 中确定合适参数的方法。一种简单的方法是通过直接遵循 multistep 学习率策略来降低学习率。但是，通过深入分析，我们发现降低学习率相当于增加 batch size，因此，当 batch size 在第二阶段开始显著增加时，学习率不需要改变。 λ 具有衰减权重的功能。因此，调整 λ 和 tuning $\|w\|_2$ 也是等价的，而 $\|w\|_2$ 同样有调整真实学习率的作用，因此这些参数在共同的影响学习率。为了提出一种确定超参数的有效方法，本文引入了有效学习率的概念，它描述了当前梯度对权重更新的影响过程。根据最近的非线性迭代理论，重要的是保持超参数对梯度影响既不能太大也不能太小。这意味着有效学习率应该是有界的。这个思想可以用来设计用于调整超参数的自动调整引擎。

3.3.1 有效学习率

神经网络的目标函数通常是一个未正则化的目标函数和一个 L^2 正则化项的组合：

$$L_\lambda(w) = \frac{1}{|Q|} \sum_{(x,y) \in Q} l(\hat{y}(x, w), y) + \frac{1}{2} \lambda \|w\|_2^2 \quad (3.1)$$

其中 w 是网络权重， λ 是权重衰减，代表 L^2 正则化项对目标函数的相对贡献。符号 $\|w\|_2$ 表示权重 w 的 L^2 范数。其中 Q 是一个有标签的训练集， $|Q|$ 代表集合 Q 中的元素个数。 x 和 y 分别表示一个样本及其对应的标签。函数 $\hat{y}(x, w)$ 代表网络输出， l 是 $\hat{y}$ 和 y 的距离函数。小 batch size 随机梯度下降 [45]，在最近的文献中通常简称为 SGD，即它在小 batch size 上运行，并执行以下更新：

$$w_{t+1} = w_t - \eta g_t \quad (3.2)$$

and

$$g_t = \frac{1}{|B_t|} \sum_{(x,y) \in B_t} \nabla l(\hat{y}(x, w_t), y) + \lambda w_t$$

其中 t 是更新（迭代）步骤的索引， B_t 是从 Q 中采样的 mini-batch，batch size 为 $|B_t|$ 。参数 η 是学习率，对于 SGD 的收敛很重要。在实践中，动量 SGD 通常如下 [46]，

$$w_{t+1} = w_t - v_t \text{ and } v_t = m v_{t-1} + \eta g_t \quad (3.3)$$

其中系数 m 代表动量。在 mv_{t-1} 的帮助下，动量 SGD 能够抑制振荡并沿着最相关的方向加速训练。为了加快 CNN 训练并降低对网络初始化的敏感性，有必要在卷积层和非线性之间使用批归一化层（BatchNorm layer 简称 BN）[47]。batch size 归一化层对小 batch size 中的每个输入通道进行归一化。如果我们只考虑神经网络中的单个单元，在 batch size 归一化下， $\hat{y}$ 由下式给出

$$\hat{y} = y_{BN}(x; w, \gamma, \beta) = Y\left(\frac{xw - \mu(xw)}{\sigma(xw)}\gamma + \beta\right) \quad (3.4)$$

由于 batch size 归一化，BN 层具有缩放不变的性质，权重 w 的大小对目标值没有影响，但会影响 SGD 的每一步更新步骤。在 batch size 归一化下，梯度通常不会通过 batch size 均值和标准差函数传播。因此， y_{BN} 相对于权重的梯度是

$$\nabla y_{BN}(x; w, \gamma, \beta) = \frac{x}{\sigma(xw)}\gamma Y'\left(\frac{xw - \mu(xw)}{\sigma(xw)}\gamma + \beta\right) \quad (3.5)$$

不失一般性，本文在两个简单的情况下关注 SGD 的行为。在第一种情况下，所有层都需要 batch size 标准化。第二种情况不涉及任何神经网络层中的 batch size 归一化。在本文设计的自动调整参数方法中讨论了一般情况。

一方面，对于第一种情况（批处理层归一化），方程（3.5）意味着

$$\nabla L(\alpha w) = \frac{1}{\alpha} \nabla L(w) \quad (3.6)$$

算法 1 的每次迭代（带有动量项）可以写成如下：

$$w_{t+1} = w_t - (\eta g_t + mv_{t-1}) = -mw_{t-1} + (1+m)w_t - \eta g_t$$

设 $w_t = \|w_t\|_2 \cdot w'_t$ 其中 $\|w_t\|_2$ 表示 w_t 的 L^2 范数，我们推断

$$w_{t+1} = -mw_{t-1} + (1+m-\eta \cdot \lambda)w_t - \frac{\eta}{|B_t|\|w_t\|_2} \sum_{(x,y) \in B_t} \nabla l(\hat{y}(x, w'_t), y)$$

使 $G_t(B_t, w_t) = \sum_{(x,y) \in B_t} \nabla l(\hat{y}(x, w'_t), y)$ 。根据中心极限定理，我们得到 $\|G_t\|_2 \propto \sqrt{|B_t|}$ 。因此， $G_t/\sqrt{|B_t|}$ 的 L^2 范数依赖于 $|B_t|$ 。让 $u_t = -mw_{t-1} + (1+m-\eta \cdot \lambda)w_t$ 以及 $u_t = \alpha \cdot u'_t$ ，而 $\alpha = \|u_t\|_2$ 。更新公式被重写如下 (3.7)：

$$w_{t+1} = \alpha \left(u'_t - \frac{\eta}{\alpha \sqrt{|B_t|}\|w_t\|_2} \left(\frac{G_t(B_t, w'_t)}{\sqrt{|B_t|}} \right) \right) \quad (3.7)$$

其中 w'_t 和 u'_t 分别是 w_t 和 u_t 在单位球体 S 上的投影。基于球面优化理论, 如果将迭代 (3.7) 视为单位球面上向量的更新, 那么大 batch size SGD (带动量项) 的每一步本质上都包含两个过程。第一个过程涉及沿 $-G_t(B_t, w'_t)/\sqrt{|B_t|}$ 的方向更新单位向量 u' , 步长为 $\frac{\eta}{\alpha\sqrt{|B_t|}\|w_t\|_2}$ 。第二个过程涉及以 α 的比例缩放第一个过程的结果。由于 $-G_t(B_t, w'_t)/\sqrt{|B_t|}$ 的范数不依赖于 $|B_t|$ 的值, 因此步长是大 batch size SGD 效率的关键。因此, 本文定义一个有效的学习率如下:

$$\eta_{effect} = \frac{\eta}{\alpha \cdot \sqrt{|B_t|} \cdot \|w_t\|_2} \quad (3.8)$$

有效学习率 η_{effect} 还显示了几乎所有超参数之间的关系, 例如学习率 η 、batch size $|B_t|$ 、动量 m 、权重范数 $\|w_t\|_2$ 和权重衰减 λ , 它提供了一些对大 batch size 训练算法设计的新见解。例如, 从方程 (3.8), 很容易发现降低学习率 η 相当于增加 batch size $|B_t|$, 这意味着如果在训练过程中学习率 η 是固定的, 本文可以增加 batch size $|B_t|$ 以在第一阶段之后保持有效学习率较小。另一方面, 对于第二种无 BN 的情况, 很容易检验 $\nabla L(\alpha w) = \nabla L(w)$ 。与第一种情况类似, 本文可以推导出 no-BN 情况下的有效学习率如下:

$$\eta_{effect} = \frac{\eta}{\alpha \cdot \sqrt{|B_t|}} \quad (3.9)$$

当 $\alpha = \|-mw_{t-1} + (1+m-\lambda\eta)w_t\|_2$

从方程 (3.8) (或 (3.9)) 可以清楚地看出, 有效学习率随着 batch size 的增加而降低。图 3.7 中的橙色曲线表明, 在不改变剩余超参数 (例如学习率 η) 的情况下继续增加 batch size 会导致训练效率低下过程。同时, 图 3.7 中的蓝色曲线表示当学习增量速率快于 batch size 增长时, 测试准确率较低。在这两种情况下, 有效学习率要么太大要么太小。为了保证大 batch size 训练的效率, 重要的是保持有效学习率在 $\eta_{lower} \leq \eta_{effect} \leq \eta_{upper}$ 的范围内, 其中 η_{lower} 和 η_{upper} 分别是有效学习率的下限和上限。

假设一个带有 BN 层的 CNN 神经网络中有 M 层。在第 t -th 迭代步时, CNN 的权重 w_t 可以写成 $w_t = [w_t^1, w_t^2, \dots, w_t^M]$, 其中 w_t^l 代表权重元素对于 l -th 层。不失一般性, 假设前 p 层涉及批标准化, 而其他 $M-p$ 层不需要批标准化。根

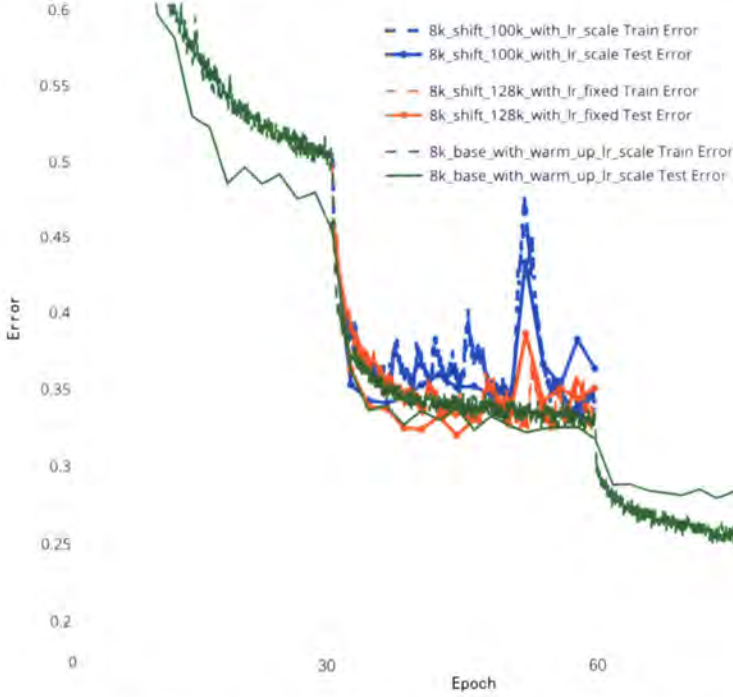


图 3.7 学习率有一个上界，过大的学习率使得 loss 发散。

Figure 3.7 Learning rate has upper bound, too large learning rate makes the loss diverge.

据上面有效学习率部分的讨论，可以将每一层的有效学习率抽象如下：

$$\eta_{effect}^l = \begin{cases} \frac{\eta}{\alpha \cdot \sqrt{|B_l|} \|w_t^l\|_2} & 1 \leq l \leq p \\ \frac{\eta}{\alpha \cdot \sqrt{|B_l|}} & p+1 \leq l \leq M \end{cases} \quad (3.10)$$

where $\alpha = \|-mw_{t-1}^l + (1+m-\lambda\eta)w_t^l\|_2$

根据最近的非线性迭代理论，本文算法设计的原则是防止 η_{effect}^l 变得太大或太小。在这个基本思想下，本文设计的方法是一种分层自动调整方法，将 η_{effect}^l 限制在给定范围内

$$\eta_{lower} \leq \eta_{effect}^l \leq \eta_{upper} \quad (3.11)$$

另一方面，在训练其他类型的神经网络时，最近的工作已经为不同情况下的 SGD 中的 η 找到了合适的学习率。为了方便直接使用一些现有的超参数，本文的设计重点是如何只在大 batch size 训练期间改变权重衰减 λ 。

算法 2 Layer-wise Auto-tuning for Hyper-parameters**Require:** 学习率 η , Batchsize $|B_i|$, Weight w_i ;**Ensure:** Momentum m , Weight decay λ ;

```

1: // 一个神经网络模型由  $M$  层;
2: for  $l = 1$  to  $M$  do
3:   初始化  $m^l$  and  $\lambda^l$ ;
4:    $\alpha = \|-m^l w_{i-1}^l + (1 - m^l - \lambda^l \eta) w_i^l\|_2$ ;
5:   if 如果第  $l$ -th 层之后有 BN 层 then
6:      $\eta' = \frac{\eta}{\sqrt{|B_i|} \cdot \|w_i^l\|_2}$ ;
7:   else if 如果第  $l$ -th 层之后没有 BN 层 then
8:      $\eta' = \frac{\eta}{\sqrt{|B_i|}}$ ;
9:   end if
10:   $\eta_{effect} = \eta' / \alpha$ ;
11:  if  $\eta_{effect} < \eta_{lower}$  then
12:     $\lambda^l = \frac{1}{\eta} (1 - m^l - \frac{\eta'}{\eta_{lower} \cdot \|w_i^l\|_2})$ ;
13:  else if  $\eta_{effect} > \eta_{upper}$  then
14:     $\lambda^l = \frac{1}{\eta} (1 - m^l - \frac{\eta'}{\eta_{upper} \cdot \|w_i^l\|_2})$ ;
15:  end if
16: end for
17: Return

```

3.3.2 基于有效学习率自动调参

为了确定可变 batch size 策略 (算法 1) 的参数, 本文提出了一个分层有效学习率自动调整引擎 (算法 2)。在可变 batch size 策略中, 第一阶段只涉及小 batch size 的训练。因此, 可以使用常用方法选择所有超参数。接下来, 随着从第二阶段开始 batch size 变大, 可以使用算法 2 来调整有效学习率。如果每一层的有效学习率大于给定的上界或小于下界, 则改变权重衰减 λ , 以确保有效学习率始终满足范围约束 (3.11)。此外, 算法 1 中的学习率 η 可以在训练过程中按照常用方式进行调整, 而算法 2 保证有效学习率始终有效。

3.4 实验

3.4.1 数据和模型

在本文的测试中,数据集是 ImageNet-1K,它有 1000 的类别,作为 ImageNet 大规模视觉识别挑战 (ILSVRC) 的一部分发布。ImageNet-1K 包含 128 万的训练图像和 5 万的测试图像 [48]。测试神经网络模型涉及标准 ResNet-50、专为移动设备设计的 ShuffleNet [43] 和标准 ResNet-101[49]。尽管一些增强的 ResNet-50 模型可能会实现更高的准确度,但本文的 ResNet-50 基线的 top-1 测试准确度是 75.4 %,用于 120 个 epoch 的训练。为了评估本文的可变 batch size 策略,本文使用 120 个 epoch 进行训练。此外,为了将运行时与最近在分布式内存系统上加速优化工作的测试结果进行比较,使用了 90 个 epoch。此外,对于本文实现中的数据增强,本文应用了 Facebook 提出的方法。

3.4.2 实验环境配置

在本文的数值实验中,本文使用了两个评估平台。第一个评估平台是配备 8 卡 NVIDIA V100 GPU 的异构系统。每个 GPU 有 32 GB 显存。由于在该系统中没有足够的内存来存储 batch size 超过 8K 的数据来训练 ResNet-50,因此使用等效的方法 Accumulate Gradients 来模拟具有多节点的分布式内存系统的梯度聚合过程。在仿真方法中,将图像的 batchsize 分成几组,系统一次使用一组样本,这些样本的梯度可以存储在 GPU 的显存中不执行更新操作。直到多次迭代后计算了一批样本来才去更新获得新的梯度,然后执行权重的更新。例如,我们在 8 块 GPU 上以 16K 的 batch size 作为模拟训练过程。首先,我们将每个 GPU 上的 batch size 固定为 32。因此,8 块 GPU 可以在前向传播和反向传播中使用 256 个图像样本。接下来,在 64 次前向和反向传播之后,则使用了 16K 的图像样本,然后我们可以更新权重。同样,我们可以模拟任何大 batch size 训练。通过使用模拟方法,可以很容易地检查训练策略的正确性。第二个平台是位于先进计算华东中心的人工智能超级计算机系统。本文最多使用 512 个节点。每个节点配备 32 核处理器和 4 块 GPU。接口节点通过 Mellanox Infini-Band EDR 互连连接。系统软件包括 Open-MPI v4.0 库和 TensorFlow with Horovod。

3.4.3 复现 facebook 的实验

为了获得测试基线，本文复现了 Facebook 结果的 8K batch size 实验结果如图 3.8 中显示。在 Facebook 的实验 [12] 中，使用 multi-steps 来调整学习率。在前 5 个 epoch，学习率从 0.1 增加到 3.2，这被称为 warmup 热身策略。在第 30、第 60 和第 80 个 epoch 结束时，学习率降低了 10%。在本文的实验中，我们同样选择前 5 个 epoch 来预热学习率，然后逐步降低学习率。与 Facebook 实验的唯一区别是我们使用了 120 个 epoch，并将在第 30、第 60 和第 90 个 epoch 结束时的学习率降低了 10%。在 multi-steps 下，表 3.1 中的测试结果验证了网络模型在 90 epoch 和 120 epoch 之后的测试准确度相似 (75.37% vs 75.4%)。如表 3.1 所示，batch size 为 8K 的 top-1 测试准确度与 batch size 为 256 的 top-1 测试准确度相同，这与 Facebook [12] 报告的结果一致。此外，图 3.8 显示，在最后的 30 时期，batch size 为 8K 的测试误差曲线与 batch size 为 256 的测试误差曲线相同。这一事实意味 multi-steps 学习率策略有效地支持大 batch size 训练。

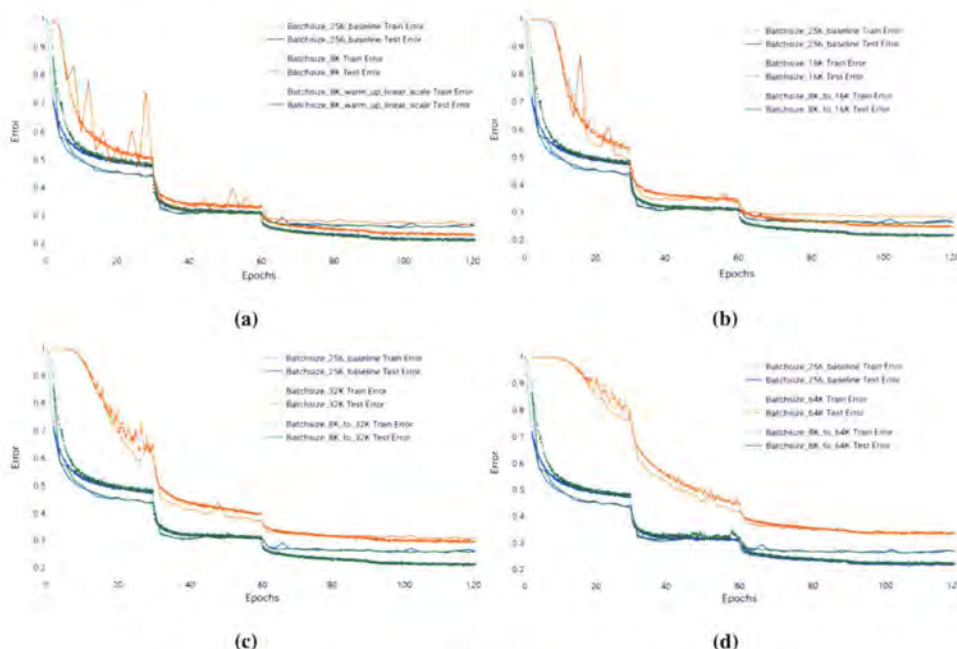


图 3.8 Batch size 分别在第二阶段增大到 16K, 32k, 64K。

Figure 3.8 The batch size is increased to 16K, 32K and 64K in the second stage respectively.

表 3.1 Top-1 ResNet-50 测试精度，使用 multi-steps 策略。

Table 3.1 Top-1 test accuracy of ResNet-50 with multi-steps strategy (baseline).

Batch Size	Learning Rate	Momentum	Weight Decay	Epochs	Test Acc
256	0.1	0.9	0.0001	120	75.4%
8192	3.2	0.9	0.0001	120	75.4%
8192	3.2	0.9	0.0001	90	75.37%

3.4.4 VBS 策略评估

根据方程 (3.10)，有效学习率 η'_{effect} 由学习率 η 、batch size $|B_t|$ 、动量 m 、权重衰减 λ 和权重范数 $\|w_t\|_2$ 决定。根据 Facebook 的实验结果，对于可变 batch size 策略，我们在第一阶段使用 8K 的 batch size，在后期使用大于 8K 的 batch size。在本文的第一个测试中，我们在第二阶段开始时将 batch size 从 8K 更改为 16K，每个 GPU 上的 batch size 为 32。此设置可确保平均梯度与 batch size 无关。如公式 (3.10) 所示，为了证明降低学习率和增加 batch size 的等价性，我们从第二阶段开始将 batch size 更改为 16K。在基线测试的第二阶段，所有阶段都使用 8K 的 batch size，有必要衰减学习率 η 以保持 η'_{effect} 不变。但是，从可变 batch size 策略的第二阶段开始，batch size 固定为 16K，之后每个阶段结束时学习率只需要降低百分之十。与第一个实验类似，我们继续增加 batch size，并从第二阶段开始采用 32K 和 64K 的 batch size。当我们在第一阶段使用 8K 的 batch size 并从第二阶段开始将 batch size 更改为 16K、32K 和 64K 时，可变 batch size 策略分别实现了 75.79%，75.79% 和 75.75% 的测试精度。与在所有阶段使用 256 或 8K 的 batch size（表 3.1）的基线相比，可变 batch size 策略在大 batch size 训练中没有表现出测试精度损失。准确地说，它的最终测试准确率提高了近 0.3%。这一事实验证了在后期增加 batch size 可以提高基线模型的准确度。

3.4.5 自动调优引擎评估

Large-batch 训练的动机是使用更大的 batch size，在不造成测试精度损失的情况下，大大减少参数更新的次数。从方程 (3.10) 可以看出，有效学习率随着 batch size 的增加而明显降低。如果我们继续增加 batch size，有效学习率会变得非常小。如表 3.2 所示，当我们使用可变 batch size 策略（算法 1）将 batch size

表 3.2 可变 batch size 策略下，Top-1 ResNet-50 测试精度。

Table 3.2 Top-1 test accuracy of ResNet-50 with VBS.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
80K	0.9	0.0001	120	75.57%
90K	0.9	0.0001	120	75.51%
100K	0.9	0.0001	120	75.36%
110K	0.9	0.0001	120	75.21%
120K	0.9	0.0001	120	75.30%
128K	0.9	0.0001	120	64.3%

扩大到 128K 时，最终测试准确度下降到 64.3%，这是很差的结果。原因是 128K 的 batch size 导致有效学习率 η_{effect}^l 不再满足约束 (3.11)。当有效学习率小于下限时，需要调整超参数以将 η_{effect}^l 移回有效范围 (3.11)。基于这个原理，提出的自动调整引擎（算法 2）在 batch size 为 128K 时自动调整权重衰减 λ 。

如表 3.3 所示，当 batch size 不大于 120K 时，使用自动调整引擎的可变 batch size 策略不会损失测试精度。当 batch size 为 128K 时，逐层自动调整引擎甚至提高了测试准确率，上升到 75.2%，这比表 3.2 中 batch size 为 128K 的结果有了很大的提高。此外，由于从第二阶段开始 batch size 设置为 128K，可变 batch size 策略显著减少了参数更新的次数，从 600,000（基线）次减少到大约 5,000 次。

表 3.3 ResNet-50 训练 Top-1 的测试精度，使用 VBS 配合超参自动调优。

Table 3.3 Top-1 test accuracy of ResNet-50 with VBS coupled hyperparameter auto-tuning.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
80K	0.9	0.0001	120	75.59%
90K	0.9	0.0001	120	75.56%
100K	0.9	0.0001	120	75.45%
110K	0.9	0.000135	120	75.47%
120K	0.9	0.00015	120	75.46%
128K	0.9	0.00016	120	75.2%

3.4.6 在不同 CNN 网络上的泛化能力评估

本文还评估了我们的算法在其他网络模型上的普遍性。除了测试 ResNet-50, 我们进一步测试了 ShuffleNet-v2-1x 和 ResNet-101。对于 ShuffleNet 训练, 通常使用 240 个 epoch。我们选择 1K 的 batch size 作为基线, 然后将初始学习率和权重衰减 (weight decay) 分别设置为 0.5 和 4×10^{-5} 。对于第一个训练阶段, 我们使用预热策略和学习率线性比例策略进行训练, batch size 为 8K。在前 24 个 epoch 中, 我们逐渐将初始学习率从 0.5 增加到 4。对于第二阶段, 我们将学习率分别衰减到 0.8、1.6 和 3.2, 以进行 batch size 为 16K、32K 和 64K 的训练。对于第三和第四阶段, 我们仍然正常衰减学习率 (衰减率同样为 10 倍), 权重衰减的值没有变化。如图 3.9 和表 3.4 所示, 使用可变 batch size 策略, batch size 为 16K、32K 和 64K 的训练误差曲线与 batch size 为 1K baseline 的曲线匹配, 并且测试误差曲线也显示没有精度损失。

当我们从第二阶段开始采用 90K、100K、110K 和 120K 的 batch size 时, 卷积层和全连接层的有效学习率需要通过自动调优引擎进行调整。如表 3.5 所示, 基于 auto-tuning 引擎的可变 batch size 策略, 在 batch size 扩展到 120K 时几乎没有精度损失, 对于 batch size 为 128K 只有轻微的精度损失。

同样, 对于 ResNet-101, 我们仍然选择 120 个 epoch 进行训练。ResNet-101 训练的设置与 ResNet-50 训练相同。初始学习率从 0.1 开始, **Warmup** 和 **Linear Scaling** 也用于 8K 的训练。当 batch size 从 16K 到 64K 时, 仅应用可变 batch size 策略 (算法 1)。如果 batch size 变得等于或大于 80K, 则自动调整引擎 (算法 2) 用于可变 batch size 策略。如图 3.10 和表 3.6 所示, 在训练结果中没有出现测试准确度的损失, 同时模型性能 (准确度) 反而得到了提高。在较大的 batch size 训练中, 表 3.7 表明, 算法 2 仍然保证模型性能, 在 batch size 为 128K 的训练中只有轻微的准确性损失。

3.4.7 与 LARS 的比较

为了验证本文方法的先进性, 我们还与当前最先进的 MLperf[50] 优化器 LARS[51] 进行了比较, LARS 优化器是解决大 batch size 训练优化问题的最公认和优秀的开源优化器。LARS 优化器 LAMB[51] 的改进版在 NLP 模型上表现更好, 在 CV 模型上表现不如 LARS, 所以我们这里选择 LARS 优化器进行对比。

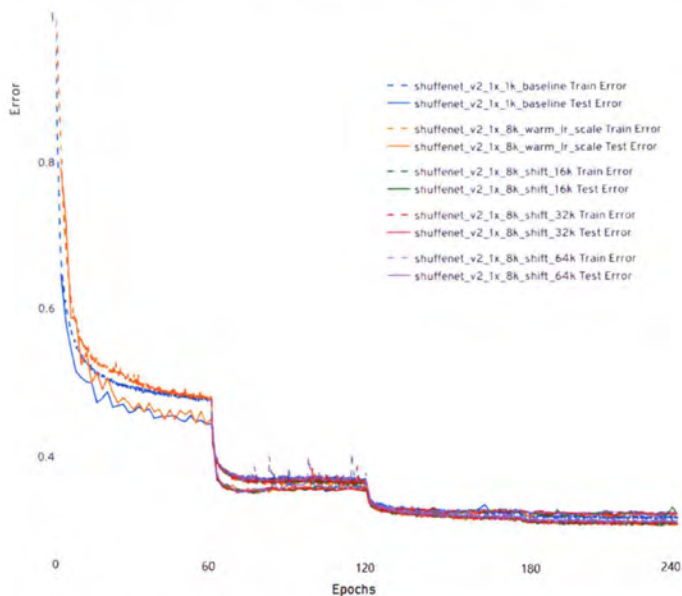


图 3.9 VBS 算法在 shufflenet 上的 top-1 测试精度。

Figure 3.9 Top-1 test accuracy of VBS on shufflenet.

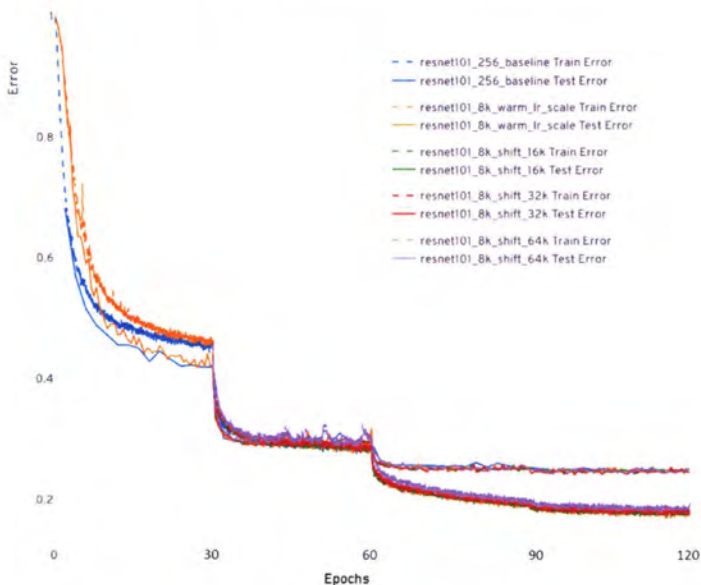


图 3.10 VBS 算法在 ResNet-101 上的 top-1 测试精度。

Figure 3.10 Top-1 test accuracy of VBS on ResNet-101.

由于 LARS 应用的学习率策略是 *poly* LR, 根据我们的测试结果, *poly* LR 策略的基线优于 multi-steps LR 策略, 在我们的测试中, 对于相同的模型, multi-steps LR

表 3.4 Batch size 64K 以内 ShuffleNet-v2-1x 的 Top-1 的测试精度，使用 VBS 自动调优。

Table 3.4 Top-1 test accuracy of ShuffleNet-v2-1x with VBS auto-tuning within batch size 64K.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
1K (baseline)	0.9	4×10^{-5}	240	68.83%
8K	0.9	4×10^{-5}	240	69.05%
16K	0.9	4×10^{-5}	240	69.07%
32K	0.9	4×10^{-5}	240	69.02%
64K	0.9	4×10^{-5}	240	69.03%

表 3.5 Batch size 64K 以上 ShuffleNet-v2-1x 的 Top-1 的测试精度，使用 VBS 自动调优。

Table 3.5 Top-1 test accuracy of ShuffleNet-v2-1x with VBS auto-tuning above batch size 64K.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
80K	0.9	4×10^{-5}	240	69.01%
90K	0.9	4×10^{-5}	240	68.97%
100K	0.9	4×10^{-5}	240	68.91%
110K	0.9	5.4×10^{-5}	240	68.83%
120K	0.9	6×10^{-5}	240	68.79%
128K	0.9	6.4×10^{-5}	240	68.11%

策略下的 batch size 256 基线准确率为 75.4 因为我们的工作基线是基于 Facebook 的实验，而 *poly* LR 相对于 multi-steps LR 不存在明显的分段训练阶段，所以学习率策略不能保持一致，我们也验证了 LARS 在 multi-steps 上表现不如 multi-steps LR。因此，我们只将最终的训练结果配合各自的 LR 策略进行比较。

我们首先使用 **LR scaling** 和 **LR warm-up** 策略在 256、4K、8K、16K、32K batch size 上测试了 *poly* LR 的基线。如表3.8所示，当 batch size 超过 8K 时，**LR scaling** 和 **LR warm-up** 会失败，模型精度会衰减迅速。然后我们应用 LARS 优化器，如图 3.12所示，我们首先测试 LARS 在按照原论文提到的参数配置的 16K batch size 上的性能，LARS 优化器将模型精度从 69.2% 提高到 74.6%。虽然与 batch size 16K 基线相比有改进，但仍比原始模型 (batch size 256) 差了近 2%。由于 LARS 优化器引入了一个新的超参数 η ，因此需要大量的手动调优尝试。如图

表 3.6 Batch size 64K 以内 ResNet-101 的 Top-1 的测试精度，使用 VBS 自动调优。

Table 3.6 Top-1 test accuracy of ResNet-101 with VBS auto-tuning within batch size 64K.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
256 (baseline)	0.9	0.0001	120	76.43%
8K	0.9	0.0001	120	76.9%
16K	0.9	0.0001	120	77%
32K	0.9	0.0001	120	77.02%
64K	0.9	0.0001	120	76.9%

表 3.7 Batch size 64K 以上 ResNet-101 的 Top-1 的测试精度，使用 VBS 自动调优。

Table 3.7 Top-1 test accuracy of ResNet-101 with VBS auto-tuning above batch size 64K.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
80K	0.9	0.0001	120	76.81%
90K	0.9	0.0001	120	76.73%
100K	0.9	0.0001	120	76.67%
110K	0.9	0.000135	120	76.6%
120K	0.9	0.00015	120	76.39%
128K	0.9	0.00016	120	75.9%

3.12所示，我们尝试调整 η ，最终得到的最佳结果是 75.1%。与原始模型的准确率相比，LARS 优化器仍然无法避免损失准确率。同时，我们观察到 LARS 更容易导致过度拟合。图 3.12 显示训练准确率过高而测试准确率相对较低。可能的原因是我们通过推导 LARS 的表达式公式发现 LARS 可能导致 L2 Norm 无效。同样，我们继续测试 LARS 在 32K batch size 上的性能。表 3.9 显示我们对超参数 η 进行了大量调整，我们在 32K batch size 上获得的最佳准确度为 74.41%。我们还测试了 batch size 64K 和 128K。正如我们所观察到的，LARS 未能取得好的结果：64K 的 batch size 仅为 69.3%，而 128K 的 batch size 仅为 67.1%。与 LARS 相比（LARS 的基线比我们的高），我们的方法在 batch size 64K 上可以达到 75.75%，在 batch size 128K 上可以达到 75.52%。

一般来说，通过我们的实验观察，LARS 配合一些可以提高模型准确率的技

巧，可以缓解大 batch size 训练在 32K 以内的准确率损失，但 LARS 相对于原始的小 batch size 模型（使用相同的技巧来提高模型准确性）仍然会丢失精度。特别是 LARS 需要大量的超参数手动调参，并且容易导致过度拟合。与 LARS 相比，我们的方法可以自动通过有效的学习率调优超参数，可以避免原模型精度损失在 120K 以内。虽然我们无法像 LARS 优化器在整个训练过程中使用 batch size 为 32K 训练模型，但 LARS 需要大量的超参数手动调整和其他辅助技巧，这是很耗时的工作。

表 3.8 ResNet-50 的 Top-1 的测试精度，poly 学习策略配合 LR scaling 以及 warm-up。

Table 3.8 Top-1 test accuracy of ResNet-50, using poly LR with LR scaling and warm-up.

Batch Size	Momentum	Weight Decay	Epochs	Test Accuracy
256 (baseline)	0.9	0.0001	120	76.31%
4K	0.9	0.0001	120	76.5%
8K	0.9	0.0001	120	76.12%
16K	0.9	0.0001	120	69.23%
32K	0.9	0.0001	120	66.15%

表 3.9 ResNet-50 的 Top-1 测试精度，LARS 优化器配合超参手动调优。

Table 3.9 Top-1 test accuracy of ResNet-50, using LARS optimizer with hand-tuning.

Batch Size	η	Momentum	Weight Decay	Epochs	Test Accuracy
32K	0.001	0.9	0.0001	120	74.33%
32K	0.002	0.9	0.0001	120	73.91%
32K	0.003	0.9	0.0001	120	74.25%
32K	0.004	0.9	0.0001	120	74.41%
32K	0.005	0.9	0.0001	120	74.10%
32K	0.007	0.9	0.0001	120	72.32%
32K	0.01	0.9	0.0001	120	71.21%
32K	0.05	0.9	0.0001	120	69.96%
32K	0.1	0.9	0.0001	120	67.81%

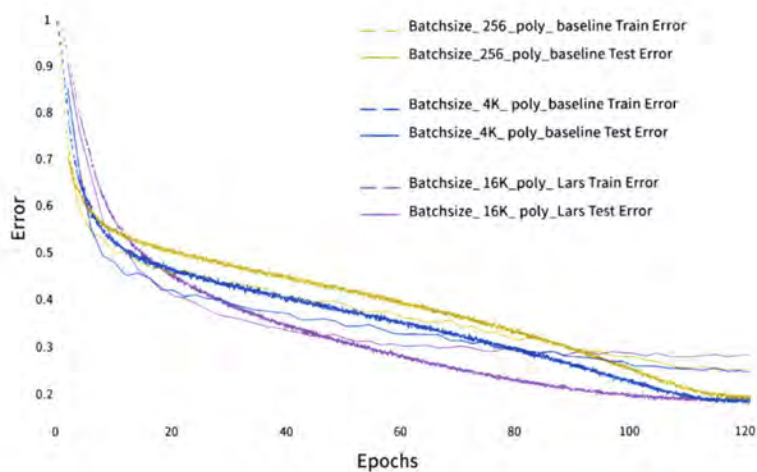


图 3.11 LARS 在 ResNet-50 训练 ImageNet-1K batchsize 16K 上的表现。

Figure 3.11 Performance of LARS on ResNet-50 training ImageNet-1K with 16K batch size.

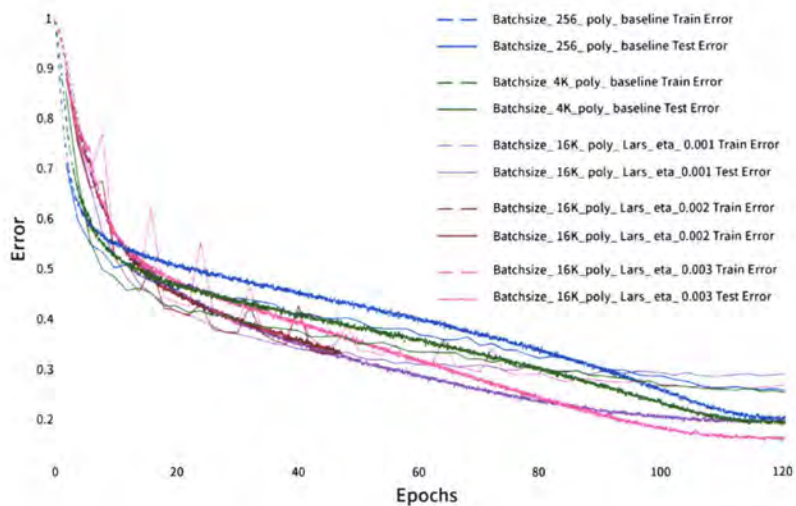


图 3.12 LARS 在 ResNet-50 batch size 16K 训练上的手动超参调优。

Figure 3.12 Hyperparameter hand-tuning of LARS on ResNet-50 with 16K batchsize.

3.4.8 与 K-FAC 和 GNC 的比较

LARS 是当前开源实现中最先进的方法。为了通过我们的方法公平地评估基本性能提升，我们复现了最近的其他 SOAT 优化方法，包括 K-FAC[52] 和 GNC[53]。我们在这些情况下使用的基线没有应用任何优化，两个实例都使用 multi-steps 学习率策略。在分布式训练场景中，由于执行 allreduce 通信操作，ImageNet 基线结果包括浮点随机性，这是一个多节点间的集合通信操作。因为我们无法获得 K-FAC 和 GNC 的原始实现配置，特别是 K-FAC 引入了一些超参数来控制 *gradient clipping*、*factor update frequency* 和 *damping*。在复现 K-FAC 和 GNC 时，我们只参考相关原论文中的核心算法来实现。如表 3.10 所示，我们分别测试了 K-FAC、GNC 和我们的方法。我们在真实的分布式计算平台上分别进行了 32k、64K、128K 的四组实验。在我们的测试中，K-FAC 通常在收敛速度方面具有优势，但在模型精度方面结果是负面的。一个可能的原因是由于梯度本身是通过随机逼近来估计的，使用 Hessian 或 Fisher 矩阵来逼近曲率信息会导致梯度噪声的积累和沉淀。高阶信息对噪声更敏感，使得 Hessian 或 Fisher 矩阵难以准确估计。GNC 的性能略好于 K-FAC，但准确度结果仍然很差。当 batch size 扩大到 64K 和 128K 时，模型测试性能严重下降。一个可能的原因是在早期训练阶段添加梯度噪声可能是有效的。梯度噪声可以帮助训练摆脱更尖锐的最小值，但在后期训练阶段添加噪声会破坏模型的收敛性。最后，与我们的方法相比，用 K-FAC 和 GNC 训练的模型比我们的差很多。

3.4.9 20 分钟训练 ImageNet-1K

为了确认在训练期间增加 batch size 可以减少模型训练时间，我们在第二个实验平台上进行了进一步的实验。首先，我们在 AI 超级计算机系统上使用 128 的 GPU 来构建 batch size 为 8 K 的基线。接下来，可变 batch size 策略使用 128 的 GPU 进行第一阶段 batch size 为 8K 的训练，然后使用 1,280、1,440、1,600、1,760、1,920 和 2,048 数量的 GPU 进行不同 batch size 的训后期练阶段。为了充分利用 GPU 的内存，我们将 batch size 更改为 64。为了确保训练过程的高性能，应用了 I/O 以及通信优化策略例如张量融合。如表 3.11 所示，使用参数自动调优引擎的可变 batch size 策略在 2,048 的 GPU 实验中仅显示测试精度略有损失，与 Facebook 的 multi-steps 策略相比，我们的 VBS 策略成功地将参数更新次数减

表 3.10 ResNet-50 的 Top-1 的测试精度，分别对应 K-FAC、GNC 和 VBS。

Table 3.10 Top-1 test accuracy of ResNet-50 with K-FAC, GNC and VBS respectively.

Dataset	Batch size	Case ID	Baseline	K-FAC	GNC	Ours
ImageNet-1K	32768	1	67.03%	68.79%	71.26%	75.74%
		2	67.31%	70.32%	69.52%	75.67%
		3	67.16%	71.05%	71.13%	75.79%
		4	67.24%	71.31%	71.32%	75.69%
	65536	1	60.12%	64.12%	67.32%	75.72%
		2	59.89%	62.21%	67.36%	75.73%
		3	60.09%	64.79%	64.13%	75.68%
		4	59.69%	64.23%	66.91%	75.66%
	131072	1	52.73%	59.92%	62.26%	75.11%
		2	53.09%	60.13%	62.97%	75.20%
		3	52.86%	58.35%	61.39%	75.19%
		4	52.91%	59.96%	63.31%	75.13%

少了约 1.7 $\times$ 。在分布式集群上，训练所有 90 个 epoch 的最短时间约为 20 分钟，除去第一个阶段的时间，单后期仅需要 3 分钟左右，而 Facebook 的 multi-steps 则需要 1 多个小时。不考虑模型精度，我们以大约 4.5 分钟的速度完成了 90 个 epoch 的 ImageNet-1K 训练。此外，需要注意的是，训练时间并不是本研究的唯一重点。相反，我们的主要动机是提出一种新的训练策略，以确保在大规模分布式系统中进行大 batch size 训练时的模型准确性。

表 3.11 ResNet-50 的 Top-1 的测试精度在超算系统上，VBS 配合自动调优。

Table 3.11 Top-1 test accuracy of ResNet-50 with VBS auto-tuning on supercomputing system.

Batch Size	Strategy	GPUs	Epochs	Test Acc	Number of Updates
8K	Facebook's	128	90	75.47%	14062
80K	ours	128+1280	30+60	75.55%	4687+937
90K	ours	128+1440	30+60	75.53%	4687+833
100K	ours	128+1600	30+60	75.41%	4687+750
110K	ours	128+1760	30+60	75.39%	4687 +681
120K	ours	128+1920	30+60	75.36%	4687+ 625
128K	ours	128+2048	30+60	75.11%	4687+ 586

3.5 相关工作

Large batch size 训练相关工作可以追溯到 [17, 54] 发现大 batch size 训练困难，这引起了人们对大 batch size 非凸优化的泛化性的广泛关注。[17] 推测，大 batch size 训练倾向于陷入 sharp 的局部最小值，导致模型泛化能力差，而小 batch size 训练倾向于陷入平坦的最小值。在这项工作中，我们从基于参数的二阶导数 Hessian 矩阵的视角 [43] 验证了 Keskar 的猜想。具体来说，我们检查较大的 batch size 会导致目标函数的 Hessian 矩阵中的前 15 个特征值较大（图 3.2），这表明随着 batch size 的增加，目标函数变得更加清晰。此外，我们对早期训练阶段效果的分析我们的测试结果都表明，早期阶段几乎完全决定了后期的主要范围搜索过程。因此，在早期阶段使用小 batch size 有助于指导优化轨迹以避免急剧的最小值并达到平坦的最小值。

为了克服大 batch size 训练中泛化性能下降的问题，[55] 和 [53] 提出了梯度

噪声注入方法，通过在权重中添加经验梯度噪声来逃离 sharp 区域的最小值，因为小 batch size SGD 引入了噪声允许从 sharp 最小值 [17] 中逃离。Haruki 声称他们的方法可以在 ResNet-50 训练 ImageNet-1K 上取得很好的效果，batch size 为 32K，但不能在 128K 上表现出好的性能。此外，由于梯度噪声的随机性，它不能每次都达到良好的性能。Lin[56] 提出了 EXTRAP-SGD 来避免 sharp 区域最小值，这是一种计算高效的局部外梯度方法，作为分布式大 batch size 训练的平滑方式。但是这项工作的性能测试只展示了关于小型网络 and 小型数据集的部分。

同时，研究者致力于利用二阶信息开发二阶方法，例如 Hessian 或 Fisher 信息矩阵。尽管在梯度协方差矩阵和 Hessian 或 Fisher 矩阵之间有多种理论相关性分析 [55, 57]，但没有一种计算简单的方法可以利用曲率信息并一般化。对于具有大量参数的网络，曲率的计算在计算上是非常昂贵的。虽然 Osawa [58] 和 Pauloski [52] 在实践中演示了使用二阶方法 K-FAC 训练 ImageNet-1K，但二阶方法仅表明收敛速度快，模型的准确率不能保证。在最近的工作中，Ma[59] 发现了相反的证据，即对于他们考虑的工作负载和 batch size 来说，K-FAC 的可扩展性并不比 SGD 好。

另一个研究方向是认为大 batch size 训练是一个优化问题，几个先前的工作 [12, 18, 60] 仔细手动调整超参数，例如学习率调度。Krizhevsky [37] 凭经验发现，简单地相对于 batch size 线性缩放学习率可以更好地应用到 1K batch size。为了避免由于学习率的线性缩放导致的优化不稳定性，Goyal[12] 提出了一种预热策略，逐渐将学习率提高到更大的值并应用常规学习率策略（例如，multi-steps 策略）。这些策略适用于训练基于 batch size 8K 的 ResNet-50 模型，而不会损失任何测试集准确性。Li[61] 提出使用控制理论来设置学习率和动量系数。Smith[62] 提出了一种控制理论来设置学习率、动量系数和 batch size。而我们的自动调整引擎包括一个分层控制理论来设置学习率、权重衰减和 batch size。batch size 调度被认为是和 LR 调度的相对的方法。Devarakonda[63] 和 Mccandlish[64] 提出了通过在前几个 epoch 逐渐增加 batch size 的预热 batch size 调度。Lin[65] 在弹性分布式训练中提出了 Dynamic Mini-batch SGD。但大多是经验调度，缺乏理论指导，最大 batch size 只能缩放到 16K。与它们相比，我们的可变 batch size 的 multi-steps 调度策略在前期不需要 batch size 预热，前期可以直接使用最大 8K 的 batch size。基于我们对不同训练 batch size 下早期阶段的观察，我们提出了一种

新的可变 batch size 策略，结合所提出的有效学习率。我们的方法成功地实现了 batch size 在 120K 内的训练，而没有任何测试集上模型准确性损失。

用于大 batch size 训练的最流行的自适应学习率已经引起了广泛的研究兴趣。You[51] 声称不同的层需要不同的梯度相对更新值，并提出了最先进的 MLperf 优化器 LARS 将 batch size 从 8K 扩展到 32K。在 LARS 中，采用了新的超参数。但是，引入的超参数仍然需要手动调整，这对开发人员来说并不容易。在我们的实验观察中，LARS 可以缓解 32K batch size 内模型精度的损失，但它无法避免精度损失并且容易导致过拟合，这在其他文献中没有提到。Huo[66] 提出了一种新颖的 Complete Layer-wise Adaptive Rate Scaling (CLARS) 算法，通过去除预热策略进行大 batch size 训练。但是 batch size 最多只能扩展到 16K。为了使大 batch size 训练更容易获得，我们的工作进一步设计了一个分层自动调整引擎，该引擎基于 BatchNorm 层对有效学习率影响的基本发现和保持有效学习率的基本思想进行自动参数调整在有效范围内。与 LARS 和 CLARS 在训练中使用的 32K 的 batch size 相比，我们的自动调优引擎可以成功地将 batch size 扩展到 120K 而没有精度损失，而 128K 只损失了一点测试精度。甚至有一个在 120K batch size 内略有性能提升。

对于并行和分布式训练的研究，Akiba[67] 展示了在 1,024 个 Nvidia P100 GPU 上进行了 90 个 epoch 的高效 ResNet-50 训练。但是，报告中缺少基线的模型准确性。在最近关于 ResNet-50 训练的工作中 [12]，与单 GPU 和单节点（每个节点 8 个 GPU）基线相比，1024 个 GPU 的扩展效率通常分别为 70% 和 80%。此外，Jia[68] 实现了在 1024 块 Nvidia P40 GPU 上的训练，扩展效率高达 87.9%。Sun [69] 在 512 个 Nvidia V100 GPU 上使用 ResNet-50 进行 ImageNet 训练的扩展效率高达 80%。Osawa [70] 在 1024 个 Nvidia V100 GPU 上使用 K-FAC 使用 ResNet-50 进行 ImageNet 训练时实现了高达约 75% 的缩放效率。在他们的实现中，GPU Direct RDMA (GDR) 用于实现不同节点上的 GPU 之间的直接数据交换。在我们的工作中，所提出的优化实现方法在具有一般硬件条件的 1024 个 GPU 上实现了 93.7% 的可扩展性。

3.6 小结

在本文中，我们分析了 SGD 不同更新阶段对训练过程的影响，发现早期阶段使用的 batch size 几乎完全决定了神经网络的最终测试精度。基于这一发现，我们提出了一种用于分布式 CNN 训练的可变 batch size 策略。接下来，基于单位球体训练动力学上的优化理论，我们推导出有效学习率，并证明有效学习率的上限和下限在大 batch size 训练中都很重要。这一观察引导设计一个自动调整引擎。这个自动调整引擎是实用的，因为它可以通过我们提出的理论指导自动调整现有的超参数。此外，为了实现分布式内存系统的高性能训练，我们设计了一种数据流 pipeline 优化策略。最后，通过使用带有自动调整引擎的可变 batch size 策略，我们能够在 ResNet-50 训练中首次成功地将 batch size 扩展到 128K。由于数据流 pipeline 优化策略和更新次数的显著减少，我们在多达 1024 个 GPU 上实现了 93.5% 的扩展效率。通过在 2,048 的 GPU 上使用可变 batch size 策略和自动调整引擎，在大约 20 的分钟内完成了 ResNet-50 训练而没有精度损失。

第4章 基于动态计算图的显存优化模块 MegTaiChi

4.1 问题描述

近年来,得益于相关编程框架的发展,深度学习在各个领域取得了前所未有的成功。当前流行的深度学习框架同时支持声明式和命令式编程,它们分别基于静态和动态计算图。计算图是一种高级计算抽象,指定计算节点(即运算符或函数)之间的数据流,这些节点由代表张量流向的边连接。静态计算图(SCG)是在执行之前构建的固定数据流图。在构建 SCG 之前不会进行实际计算。相反,动态计算图(DCG)是在执行期间动态生成的。命令式程序,例如 Python,执行计算操作并立即返回具体值。与 SCG 相比,基于 DCG 的编程模式更便于部署和调试新模型,对于深度神经网络(DNN)架构的探索具有灵活性。

随着最近两年最先进的深度学习模型不断发展,在设备内存限制内的模型训练变得越来越具有挑战性。新兴模型的内存需求阻碍了在内存有限的系统中进行训练,例如小规模集群。此外,已经证明,随着样本量的适度增加,训练往往会获得更高的性能和准确性。设备上稀缺的内存资源也限制了大 batch size 的训练。相关工作表明,除了保存样本外,DNN 训练中的主要内存消耗来自存储模型参数和中间层输出,这些输出在前向传播过程中生成并在反向传播中再次使用。在当前的深度学习框架中,中间输出通常保存在设备内存中,直到不再需要它们,导致高内存消耗,从而阻止模型大小和 batch size 的持续增加。

为了应对上述挑战,前人工作提出了两种主要技术: *tensor partition* (张量划分)和 *tensor rematerialization* (张量重新生成,在相关工作中我们称为 Checkpointing)。*tensor partition* 不是用传统的数据并行方案在所有机器上复制模型参数,而是将不同层的模型参数做切分,并将每个部分的参数放在单独的机器上进行训练 [11, 33, 71–74]。*tensor rematerialization* 专注于减少单台机器上的设备内存占用。该技术基于在前向传播中释放中间张量的内存并在必要时在反向传播中重新生成释放的张量的设计原则 [2]。一般来说,这些技术既降低了处理大型模型的壁垒,又可以在不损失模型精度的情况下实现大 batch size。此外,还有其他减少内存消耗的方法,例如使用较低精度的计算 [68] 和通过量化和稀疏化 [75, 76] 压缩模型参数。然而,这些方法会影响模型的准确性,并且需要大量的

超参数调整，这在我们的工作中没有考虑。

在实际应用中，通常在规模适中的集群上训练具有大量参数的模型或大 batch size 的 DNN，例如在具有少量 GPU [77] 的私有云平台上训练 BERT 模型。在这种情况下，我们的工作将 *tensor partition* 和 *tensor rematerialization* 结合在一起，用于 DCG 上的内存管理。虽然这两种优化技术都可以减少内存占用，但它们是根据不同的应用需求单独开发的。实际上，*tensor partition* 是为在多台机器上训练大型模型而设计的，旨在实现负载平衡以最大化并行性和局部性以最小化网络通信。然而，*tensor rematerialization* 是为了在单台机器上实现设备内存限制的训练，旨在通过依赖辅助存储或外部计算来节省设备内存空间。将它们结合起来以提高适度集群上模型训练的性能似乎很自然。然而，三个复杂性使组合变得不平凡。(1) 由于最近关于规划张量切分 plan 的工作 [28, 35, 41, 77–79] 通常基于静态图在执行前定义并在训练期间固定，因此这些 plan 没有本质上考虑 *tensor rematerialization*。因此，第一个问题是如何在运行时实现张量切分的动态调整。(2) 第二个问题是在当前张量切分 plan 生成期间，如何确定哪些历史张量应该被驱逐并稍后重新生成 [1, 2, 26, 29, 31]。(3) 由于 *tensor partition* 和 *tensor rematerialization* plan 都控制着张量在设备内存中的寿命，因此有必要考虑内存空间分配的优化。第三个问题是如何为每个张量分配内存空间以提高内存使用率。

4.1.1 动态计算图下显存优化存在的挑战

如何在多台机器上划分张量。在主要的深度学习框架中，张量切分 plan 通常在执行之前定义并在训练期间固定。因此，张量切分设计很难考虑运行时信息，而运行时信息本质上决定了张量重新生成 plan[1]。为了改变这种情况，本章工作倾向于开发一种动态张量切分策略，该策略可以在执行期间根据当前的内存使用情况进行调整。在动态张量切分设计的基础上，我们的基本假设是数据可以从远程设备以与本地设备相同的速率传输。这个假设适用于规模适中的集群，这是 MegTaiChi 的目标环境。根据对数据中心网络的研究表明，这个假设在更大范围内也成立 [80, 81]。在这个假设下，网络将不是瓶颈。在这种情况下，内存使用变得很重要，这会最终影响性能。如图 4.1(a)-(b) 所示，MP-R 比 MP-C 稍慢，但 MP-R 消耗的内存更小，这可能导致更高的吞吐量和最终性能。因此，DTP（动态张量切分）的设计需要平衡执行时间和内存使用。

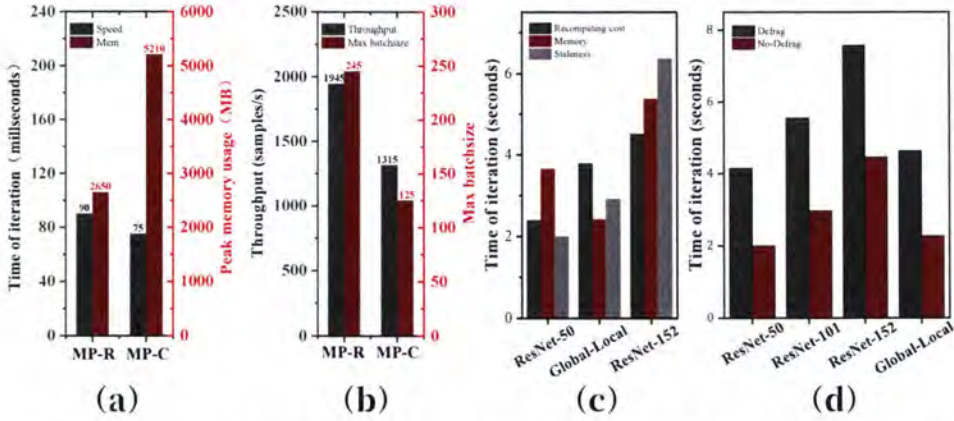


图 4.1 (a) 和 (b) 显示了 ResNet-18 训练时的性能；MP-R 和 MP-C 分别代表基于行和列的模型并行；(c) 显示了仅基于一个参数因素对不同张量驱逐方案的评估；(d) 显示了碎片整理程序对性能的影响。

Figure 4.1 (a) and (b) show the performance of ResNet-18 during training where MP-R and MP-C represent row- and column-based model parallelism, respectively; (c) shows the evaluation of different tensor eviction schemes based on only one parameter factor; (d) shows the impact of the fragmentation.

如何从设备上的内存中驱逐张量。对于张量重新生成，交换和重新计算方法已被广泛使用。在这项工作中，DTE（动态张量驱逐）尝试将交换和重新计算结合在一起。对于每个张量，许多因素会影响预测常驻张量是否价值最低并且应该被驱逐，例如 staleness（自上次访问张量以来的时间）、memory（张量的大小）和重新计算成本（从其父节点张量计算张量所需的时间）。图 4.1(c) 显示了对不同张量驱逐方案的评估，其中每个方案都是仅基于一个因素的启发式打分。很明显，这些因素在不同情况下出现交替地成为主要因素。因此，DTE 的设计应考察主要因素的整体影响。此外，为了减少内存碎片，DTE 应考虑存储在内存不同空间位置的相同大小张量之间的变化，而相关工作 [1, 26] 忽略了这一点。

如何为所有张量分配内存空间。张量切分和重新生成通常涉及重复分配和释放内存，这会导致大量内存碎片并影响内存使用。例如，尽管 ResNet-50 模型系列可以存储在单个 GPU 的设备内存中，但大 batch size 的训练仍然需要碎片整理程序来保持训练工作的继续。然而，碎片整理过程必定影响执行开销，如图 4.1(d) 所示。为了充分利用内存空间，这项工作将研究每个张量的内存分配。TMA(张量内存分配)的设计源于一个有启发的观察，即同一阶段的不同训练 epoch 共

享大量甚至全部张量，这意味着训练具有极好的任务间数据局部性。受此启发，MegTaiChi 将尝试通过跟踪内存访问序列来细粒度地指导内存分配，从而避免内存碎片的产生并降低峰值内存使用量。

4.2 内存优化模块架构

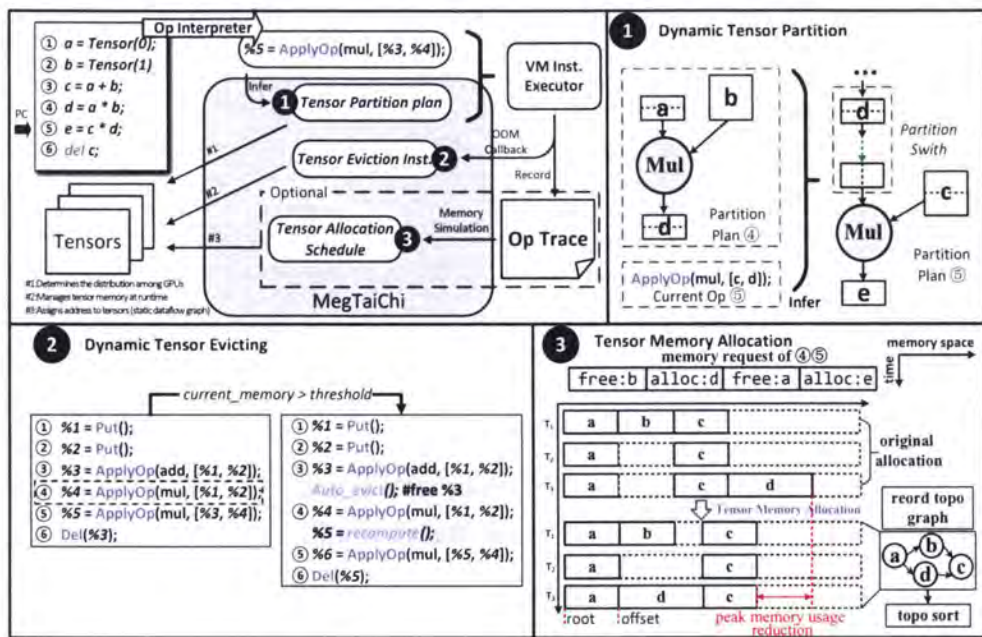


图 4.2 MegTaiChi 架构。

Figure 4.2 Architecture of MegTaiChi.

MegTaiChi 的设计基于两个关键观察。首先，DCG 上的内存管理似乎很难优化，因为计算图在执行之前是未知的，但处理过程是基于张量操作的。与传统程序的内存访问的重用和局部性分析类似，深度学习训练中的张量访问也表现出数据重用和固定访问模式。因此，我们相信动态跟踪细粒度张量访问为 DCG 上有有效的内存管理优化提供了基础。

其次，当前大部分 DNN 网络都是静态神经网络，即运行时，算子的执行顺序是不变得，DNN 结构的静态特性保证了我们方法的有效性。训练过程由数百万个迭代步骤组成。在迭代过程中，张量访问具有一些重复且固定的访问模式，因此 DCG 具有一些不变的特性，即使它们是动态构建的。这意味着分析张量访问模式和 DCG 的不变特性可以通过具体指导轻松揭示内存优化机会，例如如何

划分张量。

如图 4.2 所示, MegTaiChi 是一个虚拟机 (VM) 模块, 用于通过基本原语操作 (例如 GetValue、DeLete、SwapIn、SwapOut、Drop 等) 控制张量访问行为来在 DCG 上进行内存管理优化。MegTaiChi 可以从空间和时间位置管理张量粒度的动态内存访问。首先, 对于空间局部性, 提出了动态张量划分, 通过平衡计算成本和内存成本来确定是否或如何划分每个 Tensor。其次, 对于时间局部性, 动态张量驱逐旨在通过避免释放高频使用的数据并平衡张量重新生成所需要的时间和内存占用来确定是否或何时阻止张量分配、访问和释放。第三, 将空间局部性和时间局部性结合起来, 通过捕捉和利用 DCG 在静态网络的不变性来避免内存碎片的产生。

Dynamic Tensor Partition (DTP) 是一种启发式设计用于自动划分张量的模块。当生成 VM 指令以执行操作符 (OP) 时, MegTaiChi 需要确定如何执行 OP。根据输入和输出的元信息, 例如输入张量的切分 pattern, DTP 会为所有参与 OP 计算的张量生成一个切分 plan, 从而为 OP 生成一个执行 plan。如图 4.2 的 Panel ❶ 所示, 在生成执行 OP_{i+1} 的指令期间, 基于 DTP 的 MegTaiChi 确定需要使用两个设备执行 OP_{i+1} , 并将 OP_i 的输出和初始化的参数张量作为 OP_{i+1} 的输入, 此时会启发式的决策出是否要改变 OP_{i+1} 的并行模式, 如果需要改变并行 pattern, 那么相关的通信指令将被生成并插入到 OP_{i+1} 的计算指令之前。

Dynamic Tensor Evicting (DTE) 是一种在运行时释放张量的启发式自适应策略。当 VM 指令执行时, MegTaiChi 跟踪 OP 的执行顺序, 并记录执行信息, 例如内存分配和释放的历史、内存访问时间、每个张量的父节点和其他元数据。一旦发生内存不足 (OOM), DTE 将会插入到执行指令序列中将确定应该释放哪些张量以及是否为选定的张量选择交换或重新计算。如图 4.2 的 Panel ❷ 所示, 在执行原 OP4: $a * b = c$ 之前, 出现了 $current_memory > thre_shold$, 这表明会发生 OOM。立即生成 *Autoe_vict()* 指令, 插入 VM 指令队列, 基于 DTE 从内存中释放一些张量。此外, 如果发生内存访问失败, 内存中丢失的所需张量将通过相应的重计算或者 swapping 方法重新生成, 这些方法在其驱逐时刻被标记。值得一提的是, DTE 充分利用了 swapping 和重新计算方法在 DCG 上进行内存管理的优势。

Tensor Memory Allocation (TMA) 是一种为所有张量分配内存地址的细粒

度方法。如果 DNN 模型架构在训练过程中没有发生变化，那么即使动态生成的 DCG 的算子执行序列也是不变的。在前五个迭代期间，MegTaiChi 捕获了 DCG 的不变的特征，例如通过跟踪 VM 指令序列，计算训练时期中间张量的生命期。基于 DCG 的不变特性，TMA 可以建立一个细粒度的内存分配 plan，该 plan 将在接下来的迭代中采用，直到训练过程结束。如图 4.2 的 Panel ④所示，使用内存需求信息，TMA 模拟了内存分配过程，并在模拟中调整了一些张量的起始地址，这会生成一个拓扑图指定所有张量对时间和空间维度的依赖关系。基于拓扑图，TMA 可以利用排序算法生成接近最优的内存分配计划。值得一提的是，TMA 成功降低了内存峰值，避免了内存碎片的产生。

这三个部分由一个虚拟机解释器来协调。在迭代过程中，解释器调用 DTP 推理并执行相应的指令进行张量划分。如果在执行过程中触发了 OOM，解释器将产生一个 DTE 指令来动态地获取足够的内存。在内存访问序列被缓存后，解释器可以应用 TMA 来规划内存分配。

4.2.1 动态张量切分

形式化表达: 在神经网络训练中，如果每个算子（或层）的输入和输出变量都写成矩阵形式，则前向阶段是仿射变换 $Y = X \times W$ ，然后是非线性变换 $X' = g(Y)$ 其中 g 是一个非线性校正函数（例如 ReLU 或 Sigmoid 函数）， X 的每一行保存一个样本的输入激活值张量，类似地， Y 的每一行保存一个样本的输出激活张量。 X 和 Y 具有相同的形状。矩阵 W 保存当前层和下一层之间神经网络的权重。将 c 表示为当前层的神经元数量（或张量维度）， bs 表示 batch size。类似地，反向传播也可以写成矩阵形式 $dX = W^T \times dY$ 。这里， dX 和 dY 分别是损失函数相对于 X 和 Y 的梯度。同时，损失函数相对于模型权重 W 的梯度计算为 $dW = X^T \times dY$ 。因此，每一层的张量划分都涉及到上面的三个矩阵乘法。

Pattern 抽象: 在这项工作中，我们深入研究了所有常用的张量切分，并抽象出四种有效的模式，如图 4.3 所示。**Pattern 1** 分别沿行维度对 X 和 Y 进行切分，通常称为数据并行切分。在模式 1 下，前向传播不需要不同设备之间的通信，而后向传播必须使用 Allreduce 通信过程来获得整个 dW 。**Pattern 2** 分别沿行维度划分 W 、 X 和 Y 。由于 W 的不同部分分布在多个设备上，前向传递需要使用 Allgather 过程收集它们。在反向传播过程中，每个设备独立更新 W 的部分，这涉及到 ReduceScatter 过程。当 W 的大小很大时，可以为每个设备节省大量内

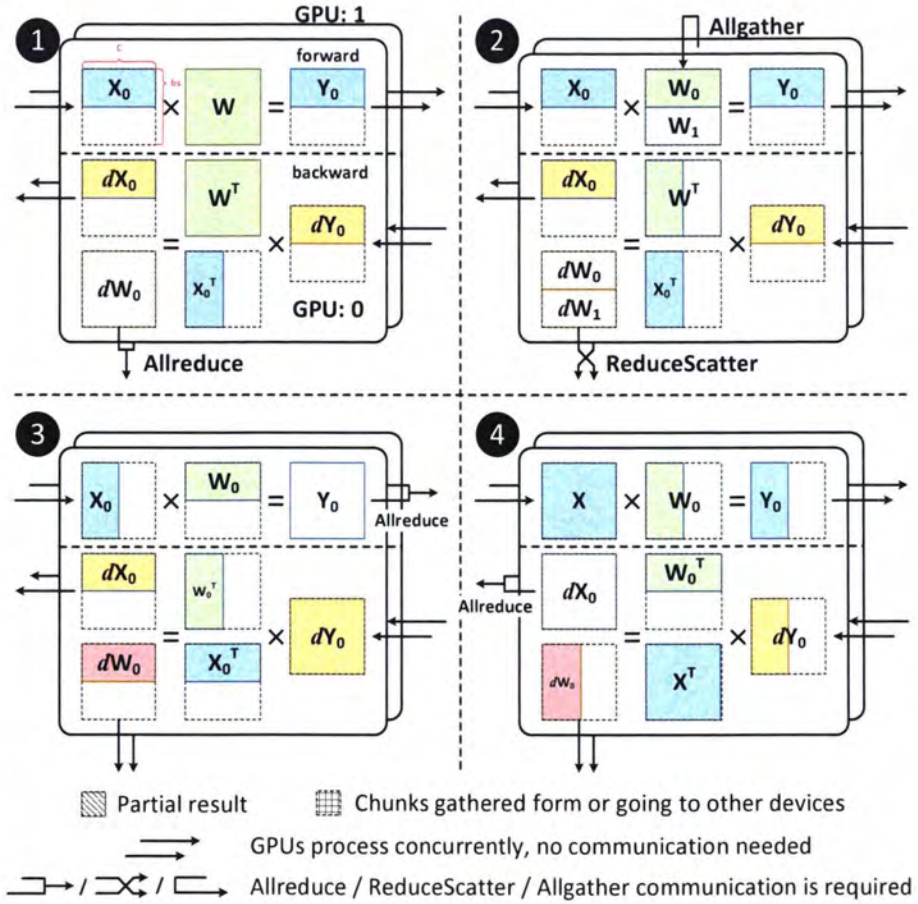


图 4.3 四种高效的张量切分模式。

Figure 4.3 Four efficient tensor partition patterns.

表 4.1 四种并行切分模式下层内通信的模式。

Table 4.1 Four intra-layer collective communication modes in parallel partition mode.

	P1	P2	P3	P4
前向计算	None	Allgather	Reduce*	None
反向计算	None	ReduceScatter	None	Reduce*
梯度更新	Allreduce	None	None	None

Reduce* 表示 Allreduce 或 ReduceScatter 通信原语

表 4.2 张量切分带来的算子间的集合通信。

Table 4.2 Inter-operator collective communication due to tensor partition.

	P1	P2	P3	P4
P1	None	None	Alltoall	Allgather
P2	None	None	Alltoall	Allgather
P3	ReduceScatter	RedcueScatter	ReduceScatter	Allredcue
P4	Alltoall	Alltoall	None	Allgather

存。**Pattern 3** 沿行划分 $\mathbf{W}$ ，沿列维度划分 $\mathbf{X}$ 。在前向传播中，每个设备都需要 Allreduce 过程来获得整个 $\mathbf{Y}$ 。但是，反向传递不需要集合通信。**Pattern 4** 分别沿列维度划分 $\mathbf{W}$ 和 $\mathbf{Y}$ 。在模式 4 下，前向传播不涉及通信，而后向传播必须将 $d\mathbf{W}$ 的分配在不同设备的部分全局值使用 Allredcue 操作合并。

此外，表 4.2 总结了前向、后向和梯度更新过程中四种模式所需的算子内通信操作（平行箭头表示不需要通信）。如果两个相邻的算子使用不同的张量划分模式，则需要额外的通信将划分模式从当前算子的模式 k_1 切换到前向传递期间下一个算子的模式 k_2 ($1 \leq k_1, k_2 \leq 4$)。此外，模式切换所需的通信流程见表 4.3。

动态张量切分：为了在训练过程中实现高吞吐量，平衡计算通信和会影响并发性内存消耗（即 batch size）是很重要的。对于第 i 运算符 o_i ，我们将 p_i^k 表示为切分配置，表示使用 Pattern k 对 o_i 的数据进行切分。使用模式 k ，总执行时

表 4.3 前向计算通信算子和对应的反向通信微分算子。

Table 4.3 Forward and the corresponding backward communication operators.

类型	前向计算通信	反向通信微分
集合通信 (一对多)	broadcast	reduce_sum
	reduce_sum	broadcast
	scatter	gather
	gather	scatter
集合通信 (多对多)	all_reduce_sum	all_reduce_sum
	all_gather	reduce_scatter_sum
	reduce_scatter_sum	all_gather
	all_to_all	all_to_all

间 $t(o_i, p_i^k, bs)$ 和内存成本 $m(o_i, p_i^k, bs)$ 可以表示如下

$$t(o_i, p_i^k, bs) = t_c(o_i, p_i^k, bs) + t_{intra}(o_i, p_i^k, bs) + t_{inter}(p_i^k, p_{i-1}^{k'}, bs),$$

and

$$m(o_i, p_i^k, bs) = m_p(o_i, p_i^k, bs) + m_t(o_i, p_i^k, bs) + m_{inter}(p_i^k, p_{i-1}^{k'}, bs),$$

其中 t_c 表示执行 OP 定义的计算所花费的时间。 t_{intra} 和 t_{inter} 分别是算子内部和算子之间的通信成本。 m_p 是存储（切分）模型参数的内存， m_t 是中间张量的内存， m_{inter} 是模式切换产生的中间张量的内存（从 o_{i-1} 的模式 k' 到 o_i 的模式 k ）。请注意，上述两个等式右侧的所有时间和内存成本都可以在训练期间测量和记录，它们是批大小 bs 的函数，因为批大小决定了中间张量的大小。

DTP 是一种启发式算法，它通过解决优化问题为 o_i 的张量切分选择模式 k_i^*

$$k_i^* = \arg \min_{k \in \{1,2,3,4\}} \lambda \cdot m(o_i, p_i^k, bs) + (1 - \lambda) \cdot t(o_i, p_i^k, bs), \quad (4.1)$$

其中 λ 是一个满足 $0 \leq \lambda \leq 1$ 的常数。请注意， λ 可以由用户调整。

由于每次张量划分推理只涉及当前算子的 4 个候选 pattern，我们设计了一个缓存机制，用 MAP 数据结构 ($key, value$) 减少推理开销，其中 key 记录历史张量划分元信息， $value$ 保存历史推理结果。如果网络结构是固定的，经过几次

迭代，缓存机制可以通过将给定的元信息映射到 *key* 上，直接从 *value* 中选择推理结果。否则，对于不固定的网络结构，MAP 数据结构只在新的元信息产生时更新缓存。

4.2.2 动态张量驱逐

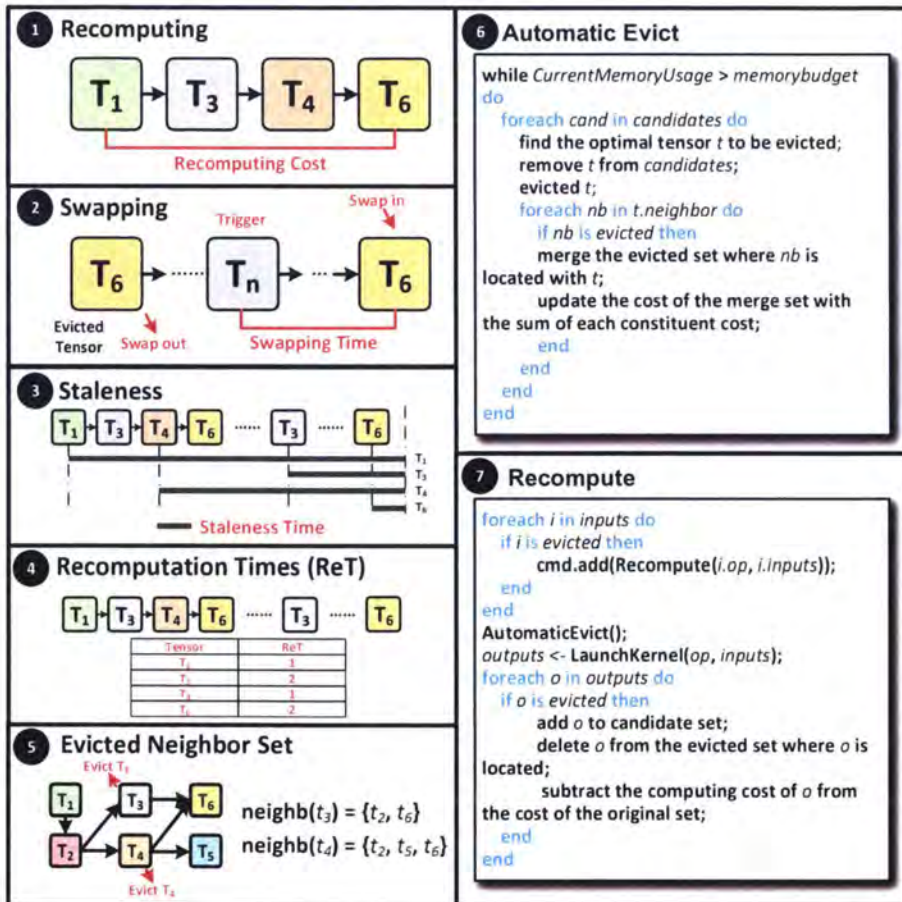


图 4.4 动态张量驱逐机制。

Figure 4.4 Dynamic Tensor Eviction mechanism.

如前文所述，当没有足够的空闲内存供当前算子执行时，需要选择和驱逐一些张量。对于张量驱逐，主要技术是张量重新生成（将为被 checkpoint 的点重新生成），它包括两种不同的方法：重新计算和交换（如图 4.4 的 Panel ① 和 ② 所示）。针对动态计算图做显存优化的最新的工作 DTR 只关注单个设备上的内存管理，而忽略了从设备内存中反复驱逐张量和重新生成所导致的严重内存碎

片。DTE 的设计是为了解决两个问题：(1) 如何管理多个设备上的 on-device 内存；(2) 如何确定哪些张量应该通过重新计算方法从设备内存中逐出或交换以避免内存碎片的产生。

内存管理机制：在多个设备上的训练涉及计算和通信。不同层需要通信的张量的梯度通常会乱序到达当前 GPU，因此很难预测其生命周期和内存的消耗。我们开发用于在多个设备上进行训练的 DTE 模块，并提出了一种内存区域管理机制。(1) 首先，我们将每个设备的 on-device 内存划分为两个区域，即 $m = m_1 + m_2$ 。第一个区域保留用于设备之间的通信以存储梯度，而另一个区域用于当前设备上的计算过程。(2) 接下来，在训练期间，将接收到的梯度放置在第一个存储区域上，该存储区域设置得足够大，以存储所有参数的梯度以及最大层的输入和输出张量在 p 设备上的 $1/p$ 。(3) 最后，我们为第二个区域 m_2 的使用定义了一个内存阈值 m^* 。如果第二个区域的内存消耗大于 m^* ，我们会选择并驱逐一些张量，直到消耗的内存小于 m^* ，这意味着内存空间大小为 $m_2 - m^*$ 保留用于当前运算符的执行。

我们使用 M 来表示由存储在设备内存中的所有当前张量组成的集合。对于每个张量 $t \in M$ ，我们将其 *evicted* 表示为 $\mathcal{N}(t)$ (参见图 4.4 的 Panel ⑤)，这是一组被驱逐的张量，要么需要重新计算才能再次获得 t ，要么需要 t 驻留才能被用于其他张量的重新计算。对于要驱逐的候选张量 t ，其重新计算成本 $C_r(t)$ 可以通过 $C_r(t) = c(t) + \sum_{\tau \in \mathcal{N}(t)} c(\tau)$ 来估计，而其交换成本 $C_s(t)$ 可以量化为从设备外存储器加载 t 的时间到设备内存。设 $m(t)$ 是 t 的大小， γ 是设备外内存和设备内存之间的带宽，我们有 $C_s(t) = m(t)/\gamma$ 。

为了减少重复驱逐张量产生的设备内存碎片，我们优先选择和释放那些驱逐不会产生新内存碎片的张量。例如，有两个张量作为驱逐候选者，如果其左右邻居已经从内存中释放，则候选者应该获得相对较高的释放优先级。对于张量 t ，我们将 $M_{left}(t)$ 和 $M_{right}(t)$ 分别表示为 t 的左右空闲内存的大小，并且倾向于驱逐张量 t 其中 $m(t) + M_{left}(t) + M_{right}(t)$ 较大 (图 4.4 的面板 ⑥)。如图 4.4 的 Panel ⑦ 所示，DTE 将通过最小化以下成本来反复驱逐张量，直到有足够的空闲内存用于执行当前操作。

$$\min_{t \in M} \frac{\min(C_r(t), C_s(t)) \cdot \beta^{ret(t)}}{(m(t) + M_{left}(t) + M_{right}(t)) \cdot s(t)}. \quad (4.2)$$

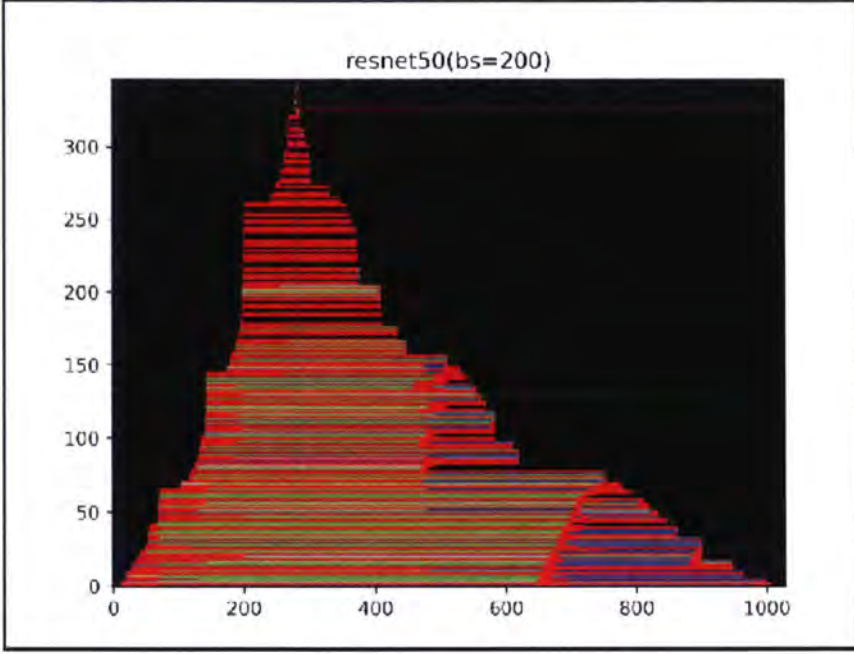


图 4.5 一次迭代中张量在 GPU 内存中的生命周期。

Figure 4.5 Life cycle of a tensor in GPU memory in one iteration.

具体来说如公式 4.2, β 被选为经验值 0.5。对于要驱逐的张量 t , 我们比较重新计算成本 $C_r(t)$ 和交换成本 $C_s(t)$ 。如果 $C_r(t)$ 较小, 则将选择张量重新计算来驱逐和重新生成 t 。否则, 将应用交换策略。(图 4.5 展示了应用打分策略后一次迭代中张量在内存中的生命周期, 红色代表张量生成, 绿色代表张量释放, 蓝色代表张量再次生成后再次释放)

4.2.3 张量内存分配

假设任务队列中有 N 个内存分配请求。第 i 次请求可以描述为四元组 $r_i = (s_i, e_i, m_i, a_i)$ ($i = 1, 2, \dots, N$), 其中 s_i 和 e_i 代表第 i 请求的分配和释放时刻, m_i 是它的内存大小, a_i 是 r_i 的未确定起始内存地址。每个内存访问请求涉及的内存分配记为 (Allocation, s_i, m_i) 和内存释放记为 (Deallocation, e_i, m_i), 它们按时间顺序排列。如果两个请求 r_i 和 r_j 满足 $s_i \leq e_j$ 或 $s_j \leq e_i$, 它们会在时间上相互冲突。如果 r_i 和 r_j 满足条件 $a_i \leq a_j + m_j$ 或 $a_j \leq a_i + m_i$, 则它们存在空间冲突。TMA 的设计是在时间冲突的任意两个请求不存在空间冲突的情况下, 最小化最大内存占用。我们提出的 TMA 包括三个步骤: 步骤 1: 在训练开始之前建立六个高效的内存管理规则。如图 4.6 所示, 它们是:

(1) *Alloc*(Panel ①); (2) *Alloc & Division*(Panel ②); (3) *Swell*(Panel ③); (4) *Free*(Panel ④); (5) *Free & Merge*(Panel ⑤); (6) *Overwrite & Free*(Panel ⑥).

步骤 2: 在前几次迭代中, 收集张量访问信息, 例如内存请求、数据依赖等。这些信息体现了 DCG 上不变的内存访问特性 (只针对网络结构不变得静态网络), 即使 DCG 是动态构建的。

步骤 3: 根据收集到的信息, 模拟张量访问过程, 并在张量访问模拟中应用六条规则进行重排张量位置。模拟结果将规划出每个张量的最佳内存块地址。

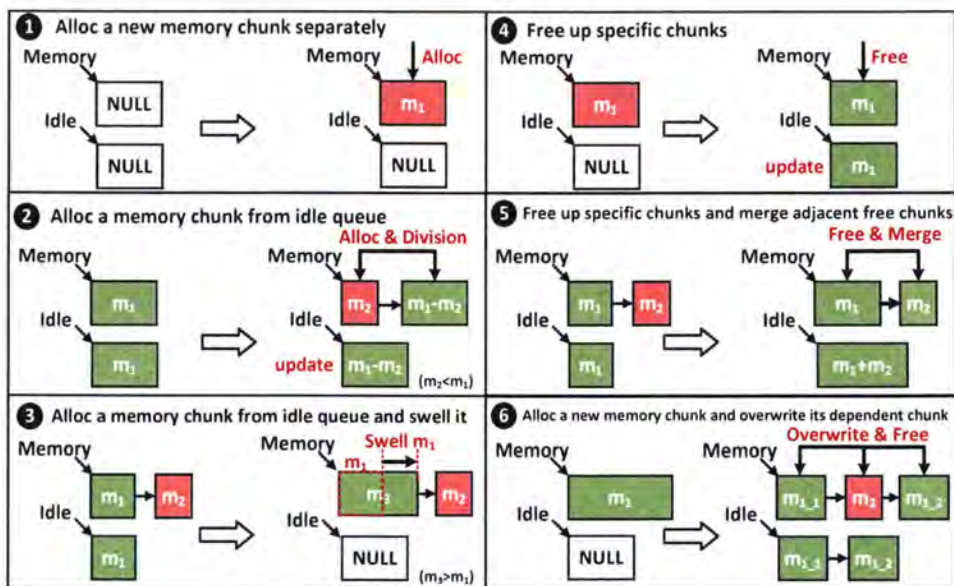


图 4.6 六个高效的内存管理规则。

Figure 4.6 Six rules for efficient memory management.

内存仿真算法分为两部分: (1) 求解出 N 个集合 $below_i (1 \leq i \leq N)$, 其中 $below_i$ 表示第 i 个请求的起始内存地址需要不低于 $below_i$ 中所有请求的最大的结束地址; (2) 根据 $below_i (1 \leq i \leq N)$, 确定每个请求的起始地址。

将每个请求拆成两个事件, (s_i, alloc, z_i) 和 (t_i, free, z_i) , 按照时间顺序依次进行分配。用链表 B 来表示当前时刻的显存分配情况, 链表中的元素 b 是一个内存块, $b.size$ 表示内存块的大小, $b.pos$ 表示该内存块目前被第 $b.pos$ 个请求占用, 若 $b.pos = 0$, 则表示该块为空闲状态。

对于 Alloc 事件, 在 B 中找到 $b.pos = 0$ 且 $b.size \geq z_i$ 中 $b.size$ 最小的 b^* 。若存在 b^* , 则在 b^* 右边插入一个新的内存块 n , $n.pos = 0, n.size = b^*.size - z_i$, 并

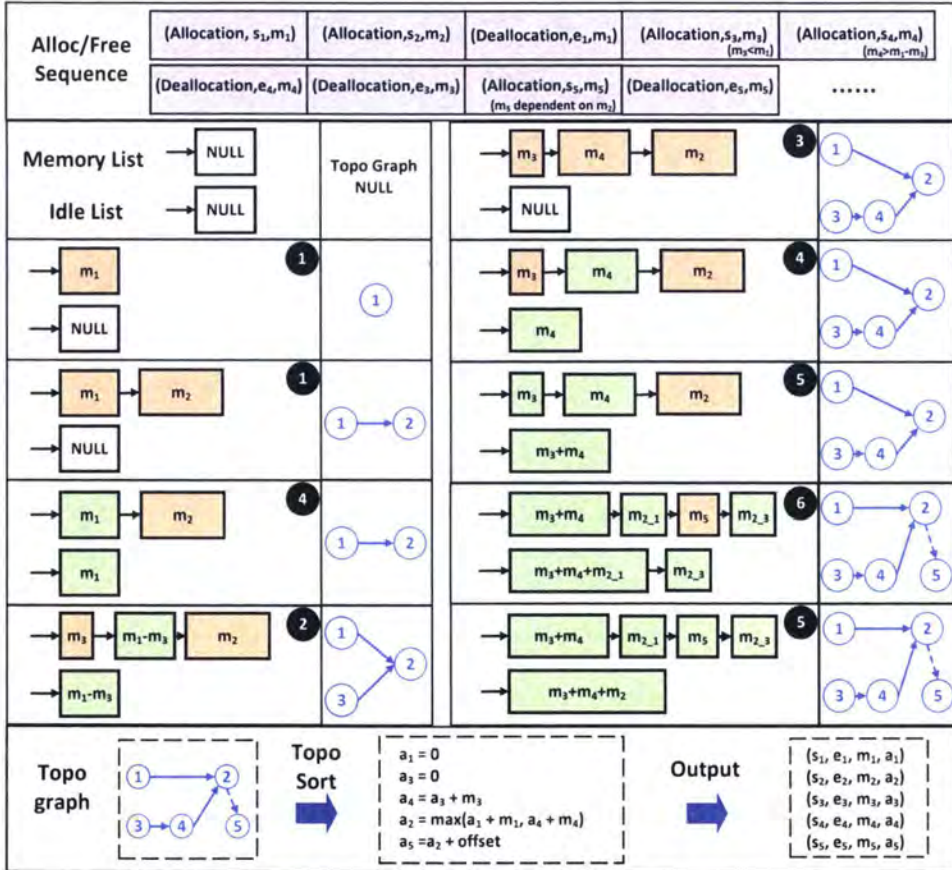


图 4.7 模拟和优化张量在 GPU 内存中的调度过程。

Figure 4.7 Simulation and optimization of tensor scheduling process in GPU memory.

将 $b^*.size$ 设置为 z_i ; 若不存在 b^* , 就找到 B 中 $b.size$ 最大的 b^* , 将 $b^*.size$ 修改为 z_i ; 若仍不存在 b^* , 就在 B 的末端插入一个新的内存块 b^* , 设置 $b.size = z_i$ 。确定 b^* 后, 找到 b^* 在左边第一个 $pos \neq 0$ 的内存块 p , 定义 $prev = p.pos$, 若 p 不存在, 则 $prev = 0$; 找到 b^* 右边第一个的 $pos \neq 0$ 内存块 s , 定义 $succ = s.pos$, 若 s 不存在, 则 $succ = 0$ 。若 $prev \neq 0$, 更新 $below_i = below_i \cup \{prev\}$, 若 $succ \neq 0$, 更新 $below_{succ} = below_{succ} \cup \{i\}$ 。

对于 Free 事件, 在 B 中找到 $pos = i$ 的内存块 b , 设置 $b.pos = 0$, 检查 b 左右两边如果也有 $pos = 0$ 的内存块, 则合并它们 ($size$ 相加), 直到 B 中不存在两个相邻的 $pos = i$ 的内存块。求解出 $below_i (1 \leq i \leq N)$ 后, 对于 $below_i = \emptyset$ 的 i , $a_i = 0$; 否则 $a_i = \max \{a_j + z_j \mid j \in below_i\}$ 。用平衡二叉搜索树和线性链表来维护 B 中的所有内存块, 可以在 $O(N \log N)$ 的时间内求解出所有 $below_i (1 \leq i \leq N)$; 用拓扑排序可以在 $O(N)$ 的时间内求解出所有 $a_i (1 \leq i \leq N)$ 。

图 4.7 显示了一个示例来说明 TMA 如何将六种规则应用于内存访问模拟。利用收集到的内存请求和数据依赖关系, 我们可以得到一个内存访问序列, 即 Alloc/Free Sequence, 如图 4.7 所示。对于序列中的每个请求, 选择六个规则之一来执行它。链表 *Memory List* 记录内存使用情况, 其中每个块代表某个张量的内存块, m_i 表示内存块大小。橙色和绿色分别代表 *alloc* 和 *free*。链表 *IdleList* 记录空闲内存块。在模拟过程中, 内存块的相对位置由拓扑图记录。基于拓扑图, 我们定义了 $Front_i (1 \leq i \leq N)$, 它是一个包含第 i 请求在某个时刻的所有前端非空块索引的数组。第 i 次请求 r_i 的起始地址是根据 $front_i$ 确定的。

如图 4.7 所示, Alloc/Free 序列中有 9 个内存请求, 每个请求都在一个模拟阶段实现。在内存访问模拟开始时, 内存列表是空的。在前两个阶段, 请求 (Allocation, m_1) 和 (Allocation, m_2) 基于规则 ① 实现。由于 m_1 在 m_2 前面, 我们得到 $front_2 = \{1\}$ 。接下来, 请求 (Deallocation, m_1) 在第三阶段由规则 ④ 实现, (Allocation, m_3) 使用规则 ② 模拟在第四阶段。同时, 集合 $front_2$ 更新为 $front_2 = \{1, 3\}$ 。在第五阶段, 由于空闲块小于 m_4 , 选择规则 ③ 来实现请求 (Allocation, m_4)。 $front_2$ 变为 $front_2 = \{1, 4\}$, $front_4$ 被构建为 $front_4 = \{3\}$ 。同样的, 剩下的请求也是分阶段实现的。模拟过程将生成最终的拓扑图和集合 $front_i$ 。根据 $front_i$, 可以确定第 i 请求的起始内存地址。具体来说, 如果 $front_i = \emptyset$, 则设置 $a_i = 0$ 。否则, 令 $a_i = \max_{j \in front_i} \{a_j + m_j\}$ 。

表 4.4 数据重排代价, p 表示设备数量, $|T|$ 代表全局张量 T 的大小。Table 4.4 Data redistribution cost. p is devices number, $|T|$ represents the global tensor T size.

Original Pattern	Current Pattern inferred by DTP			
	P1	P2	P3	P4
P1	0	0	0	0
P2	$O((p-1) \cdot T)$	0	0	$O\left(\frac{p-1}{p} T \right)$
P3	$O((p-1) \cdot T)$	0	0	$O\left(\frac{p-1}{p} T \right)$
P4	$O((p-1) \cdot T)$	$O\left(\frac{p-1}{p} T \right)$	$O\left(\frac{p-1}{p} T \right)$	0

此外, 我们可以证明上述基于规则的模拟导致了可以最小化最大内存的最佳内存分配在时间上发生冲突的任意两个请求不存在空间冲突的情况。

4.2.4 运行时开销

在下文中, 我们分析了 MegTaiChi 的开销, 其中包括 DCG 上的 DTP、DTE 和 TMA 的运行时开销。

首先, DTP 的成本主要涉及单个张量划分推理的运行时间开销和数据重新分配的开销。一方面, 由于优化问题只有 4 个候选解, 张量划分推理的开销与每次迭代的运行时间相比可以忽略不计。另一方面, 数据重新分配的开销主要包括将当前算子的权重的切分模式从一种变为另一种。例如, 假设 OP_i 的张量划分模式在上一次迭代时是 P3, 而 DTP 在当前迭代时推断 OP_i 的张量划分模式为 P1。在执行 OP_i 之前, MegTaiChi 需要执行一次 Allgather 通信, 将 OP_i 的相关权重的划分模式从 P3 改为 P1。假如 OP_i 的全局张量 T (权重) 的大小表示为 $|T|$ 。数据重新排布所需的开销约为 $O((p-1) \cdot |T|)$ 。表4.4显示了数据重新排布成本的所有可能情况。顺便说一下, 一个算子的张量切分模式从 P1 切换到任何其他, 这不涉及任何通信。

接下来, DTE 的开销包括两部分: 搜索要驱逐的张量的代价, 以及跟踪和维护候选集合中每个张量的元数据的代价。对于搜索要驱逐的张量, DTE 必须遍历所有的候选张量, 为每个张量打分, 并找到分数最小的最优张量, 这很耗时。为了跟踪每个张量的元数据, DTE 需要更新和维护被驱逐的候选集中的剩余张量的元数据, 其开销可以在训练过程中被计算隐藏。

最后, TMA 的开销可以忽略不计, 因为 TMA 在热身阶段后只执行一次, 其主要成本是对拓扑图进行排序, 其时间复杂度为 $O(N \log(N))$, 用于 N 个内存块分配请求。

4.2.5 API

我们同时在 Python 的前端提供了简易的 API 如图4.8, 用户只需要添加几行代码就可以自动实现训练超大的模型和 batch size。

```

1 # single device
2 import megengine as mge
3 import DNN model # your training model
4 mge.dte.eviction_threshold = "x GB" # set the eviction threshold
   to x GB
5 mge.dte.enable() # enable the DTE optimization
6 mge.tma.enable() # enable Tensor Memory Allocation
7 # ..... your training code
8
9 # multi devices
10 import megengine as mge
11 import DNN model #your training model
12 import megengine.distributed as dist # model parallel training
13 @dist.launcher
14 def main():
15     mge.dte.eviction_threshold = "x GB" # set the eviction
       threshold to x GB
16     mge.dte.enable() # enable the DTE optimization
17     mge.dtp.enable() # enable Dynamic Tensor Partition
18     mge.tma.enable() # enable Tensor Memory Allocation
19 # .....your training code

```

图 4.8 Python API 的使用。

Figure 4.8 Python API usage.

4.3 实验

4.3.1 实验配置

平台: 我们的实验是在一个适度的云集群上进行的, 配置见表4.5。

该实验在 Ubuntu 18.04 上运行。CUDA Toolkit 版本是 10.0, cuDNN 是 7.3.1。MegTaiChi 是基于 MegEngine 1.7 开发的, MegEngine 1.7 [82] 是一个开源命令式 DL 框架。评估的重点是 MegTaiChi 与最先进的作品的比较, 即 Capuchin [26]、DTR [1] 和 Sublinear [2]。Capuchin 是提出交换策略以重新生成张量的最先进的工作, 而 DTR 和 Sublinear 是开发重新计算的最先进的工作。由于 Capuchin 不

表 4.5 实验平台配置。

Table 4.5 Experimental platform configuration.

Number of Nodes (Machines)	32
Number of GPUs	256
GPU	Nvidia RTX 2080Ti (PCIe)x4 (16 GiB)
CPU	Intel(R) Xeon(R) E5-2650 v3 (32 GiB×8)
CPU-GPU Interconnect	PCI-Express Gen3x16 (16 GB/s)
GPU-GPU Interconnect	PCI-Express Gen3x16 (16 GB/s)
System Interconnect	25Gb Ethernet x 2 (6GB/s)

是开源的，我们使用 MegEngine 将其实现为 Meg-Capuchin。Meg-Capuchin 不能在多个 GPU 上运行，因为它的设计仅适用于单个 GPU。DTR 和 Sublinear 是开源的。原始的 DTR 表示为 Pytorch-DTR，它也不能在多个 GPU 上运行。为了在多个 GPU 上比较 DTE 和 DTR，我们基于 MegEngine 开发了 Meg-DTR。表 4.6 展示了评估中的所有方法。

应用负载：我们在表 4.7 中显示的 DNN 上评估 MegTaiChi，其中包括 8 个具有固定架构的代表性 DNN 和 2 个在训练期间具有不固定架构变化的 DNN。首先，分别给出了 DTP、DTE 和 TMA 的评价。下一个，我们展示了训练过程的内存使用情况，以检查 TMA 是否减少了内存碎片并进一步降低了峰值内存。最后，我们证明了 MegTaiChi 显着增加了内存受限下的样本的 batch size，并通过名为 Global Local 的大规模人脸识别模型的训练来评估 MegTaiChi，该模型可以在实际应用中识别数百万人。

4.3.2 DTP 评估

对于 DTP 的评估，我们选择了四个 DNN 模型，如图 4.9 所示。与 DP 相比，DTP 对 ResNet-50 模型的性能提升贡献很小，而 DTP 对 VGG-16、GL-base 和 GL-large 的性能提升分别为 1.3 x、1.4 x 和 1.5 x。主要原因是 ResNet-50 是一个以卷积为主的网络，其中每个卷积层通常都很小，并且更喜欢数据并行性（本质上是模式 1）。然而，VGG-16、GL-base 和 GL-large 模型都包含许多全连接层。我们发现 DTP 自适应地将模式 1 应用于卷积层，将模式 3 和模式 4 应用于全连

表 4.6 评估中用到的方法。

Table 4.6 Methods used in the evaluation.

Symbol	Definition
Meg-Capuchin	我们实现的 Capuchin [26] 基于 MegEngine.
Pytorch-DTR	DTR 开源版本 [1] 基于 Pytorch.
Meg-DTR	我们实现的 DTR [1] 基于 MegEngine.
Sublinear	Sublinear 的开源版本 [2] 基于 Pytorch.
MegEngine-dir	直接使用 megengine 训练 DNN 网络.
DTP	Dynamic Tensor Partition.
DTE	Dynamic Tensor Evicting.
TMA	Tensor Memory Allocation.
DP	Date Parallelism Strategy.
MP	Model Parallelism Strategy.
DP+MP	Empirical strategy using DP and MP for different layers.

表 4.7 评估中用到的神经网络模型。

Table 4.7 Neural network models used in the evaluation.

Model	Type	Architecture	Num. of Param.
ShuffleNet[83]	CNN	fixed	2.3M
VGG-16[49]	CNN	fixed	138M
ResNet-50[49]	CNN	fixed	25.6M
ResNet-101[49]	CNN	fixed	44.5M
ResNet-152[49]	CNN	fixed	60.2M
Bert-base[84]	Transformer	fixed	110M
SPOS[85]	CNN	unfixed	3.4M
Global Local (GL)-base	CNN	fixed	100M
Global Local (GL)-large	CNN	fixed	1.5B
GPT[86]	Transformer	fixed	1.3, 2.6, 6.7, 12(B)
MoE[35]	MoE	unfixed	0.35, 1.3, 2.4, 8(B)

接层，从而提高了性能。

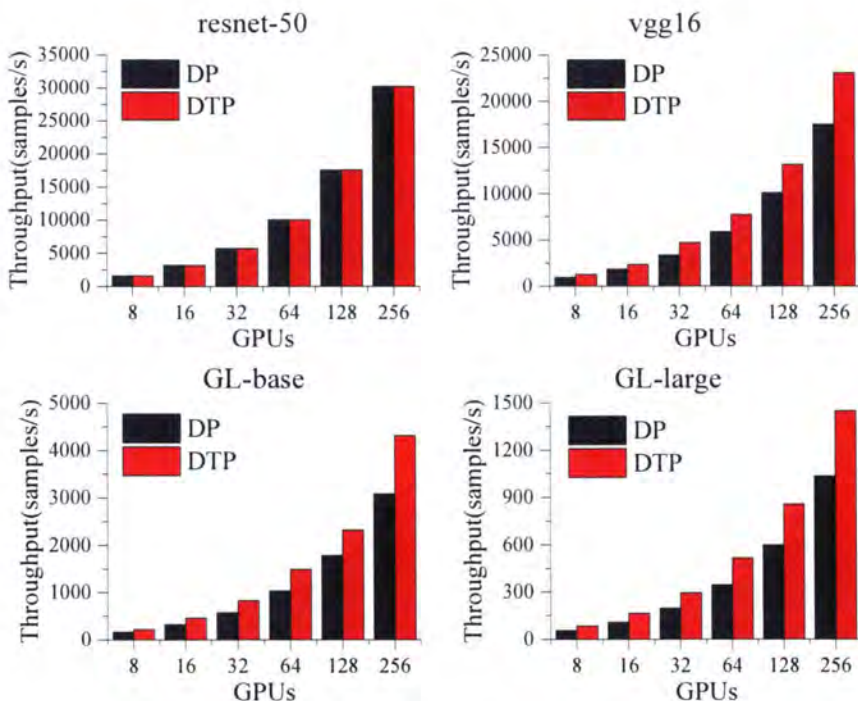


图 4.9 DTP 评估。

Figure 4.9 DTP evaluation.

4.3.3 DTE 和 TMA 评估

对于 DTE 和 TMA 的评估，分别给出了单 GPU 和多 GPU 的测试结果。在本节中，多 GPU 上的测试不涉及 DTP，而 DP 用于所有层的张量切分。

4.3.3.1 单 GPU 上的评测

我们比较了 Meg-Capuchin、Pytorch-DTR、Meg-DTR、DTE 和 TMA 在 5 个 DNN 模型的训练上的吞吐量。在比较之前，我们评估了两个深度学习框架 Pytorch 和 Megengine 的性能。通过运行 10 次 ResNet-50 训练迭代，两个框架以小于 10 毫秒的小差距实现了几乎相同的性能，这有助于我们专注于不同方法的比较。

如图 4.10 所示，与 Capuchin 和 Pytorch-DTR 相比，DTE 的平均性能提升分别约为 1.5 x 和 1.23 x。此外，通过将 DTE 与 TMA 合作，平均吞吐量提高分别比 Capuchin 和 Pytorch-DTR 快 1.6 x 和 1.5 x。我们从实验结果中得到三个重要的观察结果。

首先，对于 Bert-base 模型的训练，随着批大小从 20 增加到 100，不同方法

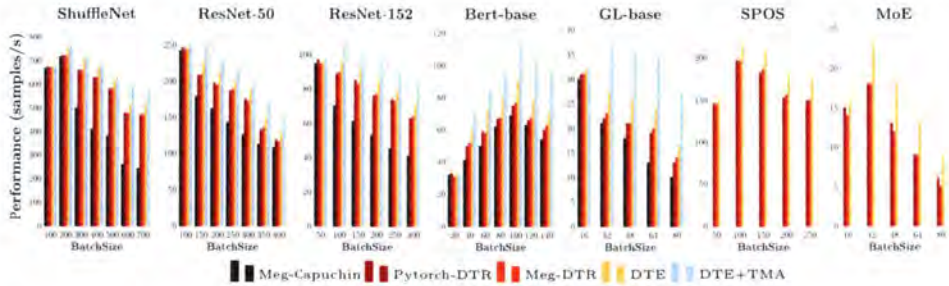


图 4.10 在单个 GPU 上评估 DTE, Meg-Capuchine, Pytorch-DTR 和 Meg-DTR 是三个基线。
Figure 4.10 Evaluating DTE on a single GPU where Meg-Capuchine, Pytorch-DTR and Meg-DTR are the three baselines.

的吞吐量都在增加, 因为在此过程中空闲内存相对丰富。随着 batch size 从 100 到 140 的不断增加, 吞吐量单调下降。这是因为当空闲内存不足时, 会频繁发生张量交换/重新计算操作, 从而影响执行开销。ShuffleNet 模型的训练也遵循了类似的趋势。然而, 对于 ResNet-50、ResNet-152 和 GL-base, 最小的 batch size 会导致更少的可用内存。因此, 不同方法的吞吐量随着 batch size 的增加而降低。

其次, 如表 4.8 所示, 交换的执行时间是我们平台重新计算成本的几十倍, 并且 Capuchin 的代价模型差于 DTR 以及 DTE。因此, 基于交换的 Meg-Capuchin 比 Pytorch-DTR 和 Meg-DTR 慢, 它们都应用重新计算以进行张量重新生成。我们还发现 DTE 会自适应地选择重新计算而不选择 swapping 策略。

第三, 与 Pytorch-DTR 和 Meg-DTR 相比, DTE 平均分别实现了大约 1.24 x 和 1.22 x 的性能加速。这主要归功于 DTE 的设计考虑了内存碎片的影响。避免内存碎片的产生可以减少张量驱逐操作的频率, 从而提高性能。此外, 通过协调 DTE 和 TMA, 即 DTE+TMA, 内存使用遵循细粒度分配, 可以实现更高的性能加速。

4.3.3.2 多个 GPU 上的测试结果

图 4.11 显示了 8 个 GPU 上的测试结果。与 Meg-DTR 相比, DTE 平均实现了 1.6 倍的加速。此外, 与仅使用 DTE 相比, 协调 DTE 和 TMA 可以获得 1.4 倍的性能提升。这是因为 DTE+TMA 实现了细粒度的内存分配, 使得内存碎片进一步减少, 驱逐操作变得不那么频繁。需要注意的是, 无论是使用单个 GPU 还

表 4.8 Conv-BN-Relu 块在 1 个 GPU 和 8 个 GPU 上的重新计算和交换开销。

Table 4.8 Recomputing and swapping overhead for Conv-BN-Relu block on 1 GPU and 8 GPUs.

Batch Sizes	Recomputing Cost (ms)			Recomputing	Swapping
	Convolution layer	BN layer	Relu layer	Frequency	Cost (ms)
32 (24.5 M)	(0.12, 0.12)	(0.19, 0.20)	(0.10, 0.11)	(0, 0)	(2.18, 7.98)
64 (49 M)	(0.22, 0.22)	(0.34, 0.36)	(0.20, 0.21)	(0, 0)	(4.36, 43.11)
128 (98 M)	(0.41, 0.41)	(0.64, 0.67)	(0.39, 0.39)	(2, 2)	(8.71, 26.22)
256 (196 M)	(0.81, 0.81)	(1.23, 1.24)	(0.77, 0.78)	(2, 3)	(17.41, 52.26)

Here, (a, b) shows test results a and b on 1 GPU and 8 GPUs respectively.

是使用 8 个 GPU，不同方法的吞吐量随着 batch size 的增加都遵循类似的变化趋势，而 8 个 GPU 的吞吐量明显高于单个 GPU。

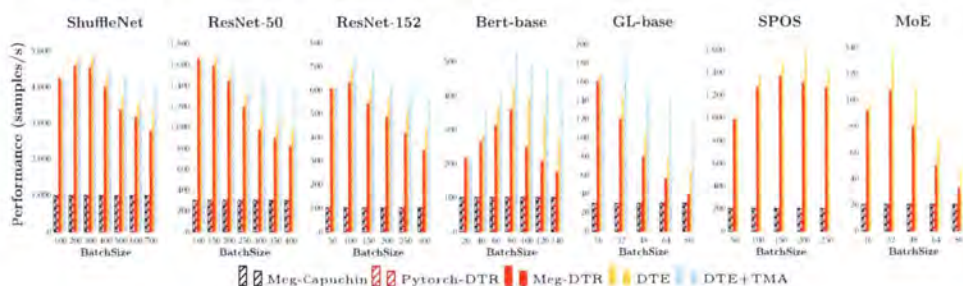


图 4.11 DTE 在 8 个 GPU 上的评估，带斜线的条形图表示基线。

Figure 4.11 DTE evaluation on 8 GPUs where bars with slashes indicate baseline.

4.3.3.3 动态神经网络上的评估

对于动态模型，运算符的执行顺序是不固定的。在相关工作中，目前只有 DTR[1] 可以支持动态神经网络的训练。因此，对动态模型的评估主要集中在 DTE 与 DTR 和 Meg-DTR 的比较。

Figure 4.10 和 Figure 4.11 显示了不同方法在训练两个动态模型（即 SPOS 和 MoE）时的表现。与 Pytorch-DTR 和 Meg-DTR 相比，DTE 分别实现了 1.3× 和 1.4× 的平均性能加速。此外，DTE 在 8 个 GPU 上的改进比在单个 GPU 上的改进要高。值得一提的是，DTR[1] 只是考虑了单 GPU 上的张量重新生成。然而，MegTaiChi 将动态张量再生成扩展到多个 GPU 并减少了内存碎片。

表 4.9 碎片整理代价。

Table 4.9 Defragmentation costs.

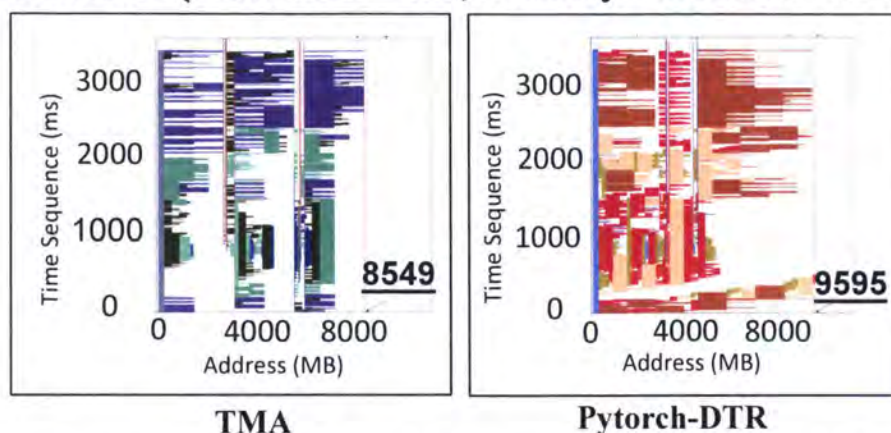
Model	Batch Sizes	Defragmentation		
		Average execution cost of one defragmentation	Frequency per 10 iterations	
			Pytorch-DTR	TMA
ResNet-50	400	1367 ms	19	0
ResNet-101	300	1605 ms	6	0
ResNet-152	300	1471 ms	9	0
GL-base	90	1048 ms	10	0

4.3.4 关于内存使用

图 4.12 显示了使用 TMA 和 Pytorch-DTR 在每次训练迭代的任何时刻的内存使用情况。在图 4.12 的每个面板中，X 轴和 Y 轴分别代表内存地址和时序。沿 X 轴，最左边的灰色区域用于存储模型参数和梯度（第 4.2.2 节中提到的第一个区域 m_1 ），颜色区域用于参与当前的张量计算（在 4.2.2 节中提到的第二个区域 m_2 ）。很明显，与 Pytorch-DTR 相比，TMA 产生的内存碎片更少。由于这一事实，ResNet-50 的峰值内存使用量分别减少了 11% 和 GL-base 的 10.5%，这是相比于 DTR[1]。

随着 batch size 的不断增加，我们发现即使应用了 DTR，也会发生 OOM。如表 4.9 所示，当 batch size 在单个 GPU 上增加到 400 时，ResNet-50 模型的训练必须利用碎片整理过程才能继续执行。为了生成一个新的完整的空闲内存空间，应该足以满足当前的执行，碎片整理过程涉及重新分配设备内存中的所有数据并将所有内存碎片放在一起，这非常耗时。表 4.9 表明，一个碎片整理过程的成本平均超过一千毫秒，这严重影响了执行开销。从表 4.9 可以清楚地看出，Pytorch-DTR 仍然必须平均每 10 次迭代去多次调用碎片整理程序以训练具有大 batch size 的不同模型。然而，在同样的情况下，TMA 成功地避免了碎片整理。这一事实再次证明，由于细粒度的内存分配设计，TMA 可以有效减少内存碎片的产生。

ResNet-50 (Batch Size = 400, Memory Threshold = 5GB)



GL-base (Batch Size = 80, Memory Threshold = 5GB)

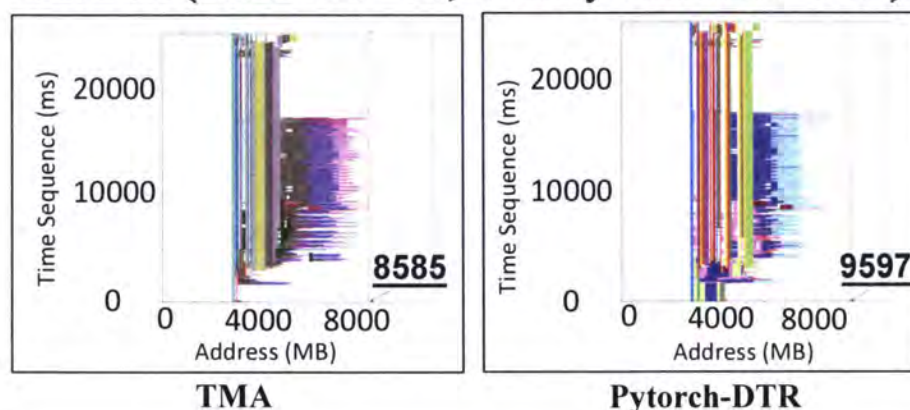


图 4.12 每次迭代中的内存使用情况对比。

Figure 4.12 Comparison of memory usage in each iteration.

4.3.5 MegTaiChi 的评估

在本节中，我们评估了 MegTaiChi 的运行时开销，我们将 MegTaiChi 与 Sub-linear [2] 进行比较，后者以内存优化而闻名，以支持单 GPU 上的大批量训练，然后我们将 MegTaiChi 应用于大规模人脸识别模式的训练。

4.3.5.1 MegTaiChi 运行时开销评估

如图4.13所示，MegTaiChi 的运行时间开销小于每个次迭代的运行时间的5%。从图4.13中，我们得到三个观察结果。

首先，MegTaiChi 在单个 GPU 上的开销比在多个 GPU 上的开销要小，因为

DTP 在单个 GPU 上的训练是不必要的。

第二，与 ResNet-50、ResNet-152 和 SPOS 等网络相比，GL-base 网络包含更多的全连接层和更少的卷积层，这会导致更多的通信和内存使用成本。因此，MegTaiChi 用于训练 GL-base 模型的开销增加到其他模型开销的 1.5 倍左右。

第三，对于 SPOS 网络的训练，尽管 SPOS 是动态的，MegTaiChi 在所有工作负载中引入的开销都小于 3%，平均为 2.68%。

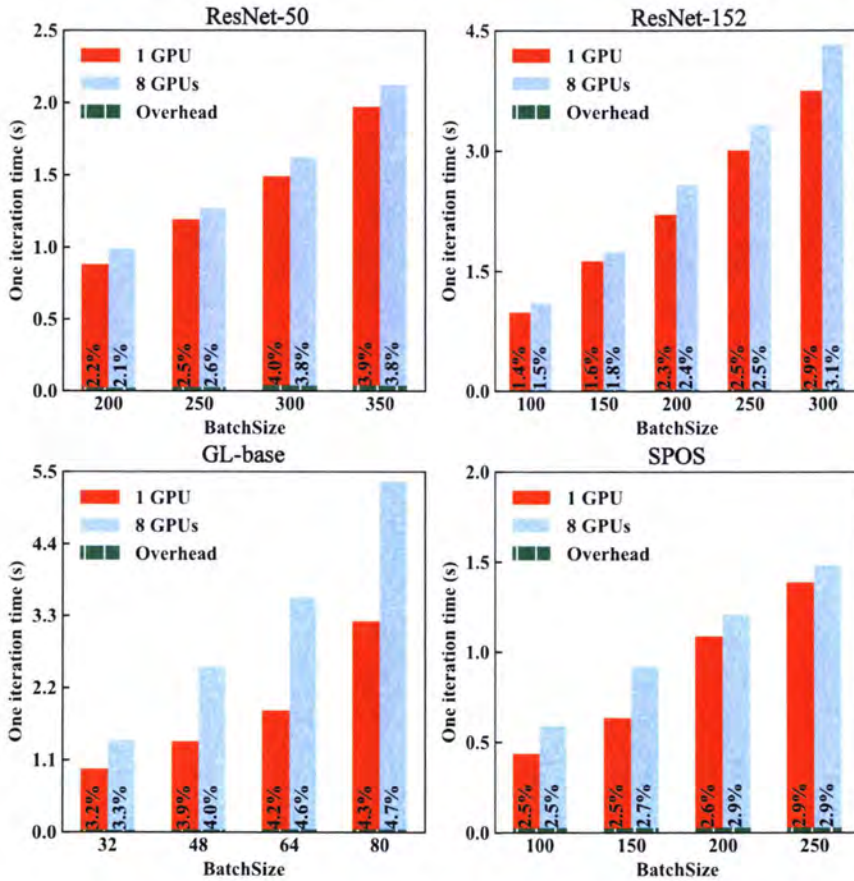


图 4.13 MegTaiChi 的运行时开销。

Figure 4.13 Runtime overhead of MegTaiChi.

4.3.5.2 MegTaiChi 和 Sublinear 的对比

图 4.14 展示了 MegTaiChi 与 Sublinear 在 ResNet-50 的训练中的比较，其中内存阈值 m^* 从 4G 增加到 6G。我们从实验结果中总结出三个重要的观察结果。首先，对于相同的 batch size，增加 m^* 使得 MegTaiChi 的性能逐渐接近 Sublinear。

当 m^* 设置为 6G 时, MegTaiChi 比 Sublinear 稍慢, 而每次迭代的差距小于 0.04s。其次, 当 batch size 分别为 100、150 和 200 时, 很明显 MegTaiChi 消耗的内存比 Sublinear 少。第三, 对于 ResNet-50 在单 GPU 上的训练, Sublinear 的最大批大小为 200, 而 MegTaiChi 支持更大的批大小。如图 4.14 所示, $bs = 300$ 的 MegTaiChi 的内存消耗仍然小于 $bs = 200$ 的 Sublinear。

此外, 表 5.5 显示了用于训练 6 个模型的不同方法支持的最大 batch size。很明显, 与其他相比, MegTaiChi 支持更大的最大 batch size。与 MegEngine-dir 和 Sublinear 相比, MegTaiChi 的最大 batch size 分别达到了 $5\times$ 和 $2.4\times$ 的提升。

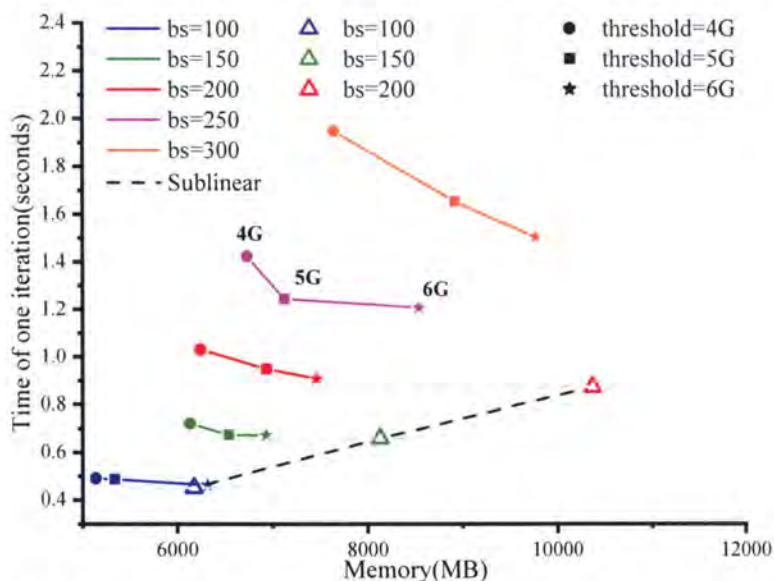


图 4.14 MegTaiChi 与 Sublinear 在单个 GPU 上训练 ResNet-50 模型的对比。

Figure 4.14 MegTaiChi versus Sublinear for training ResNet-50 models on a single GPU.

4.3.5.3 MegTaiChi 和 FlexFlow, Gshard, ZeRo-3 的对比

对于 MegTaiChi 与 FlexFlow、Gshard 和 ZeRo-3 的比较, 不同模块的弱可扩展性是通过增加批次大小和模型大小以及 GPU 数量来评估的。由于不同数量的 GPU 的模型大小是不同的, 使用吞吐量作为衡量标准似乎是不合理的, 例如, 每秒训练的 tokens 数。因此, 我们根据最近工作, [28] 的建议, 选择了 GPU 的每秒浮点运算量 (PFLOPS) 作为评估指标。如图 4.16 所示, 与 FlexFlow、Gshard 和 ZeRo-3 相比, MegTaiChi 分别实现了 $1.2\times$ 、 $1.8\times$ 和 $1.5\times$ 的平均性能提升。

随着批量大小和模型大小的增加, 训练过程需要更多的额外内存空间, 用于

表 4.10 不同的方法支持的最大 batchsize。

Table 4.10 The maximum batch size supported by the different methods

Model	MegEngine-dir	Sublinear [2]	MegTaiChi
ShuffleNet	200	550	950
ResNet-50	110	200	430
ResNet-152	50	130	300
GL-base	16	45	90
Bert-base	22	45	140
SPOS	50	N/A	350

GPU 之间的数据移动，这将导致在弱扩展测试中内存成本的超线性增长。因此，如果内存管理不有效，那么就可能触发内存 OOM。从图中可以看出，FlexFlow 和 Gshard 在 8 个 GPU 的弱扩展测试中未能成功训练 GPT 模型。事实上，FlexFlow 和 Gshard 试图利用 SCG 上多维张量划分的并行性。然而，MegTaiChi 抽象了张量切分模式，在 DCG 上立即生成了内存优化策略，导致了更好的内存使用行为。

ZeRo-3 通过手动调整张量重新生成和张量切分来管理内存使用。因此，它能够训练更大的 batch size 和网络模型。但是，ZeRo-3 并不关注张量重新生成和张量切分行为的性能优化。具体来说，ZeRo-3 必须在 GPU 之间通信所有的梯度。当梯度大于激活时，其性能就会退化。然而，MegTaiChi 自动将张量切分与张量重新生成相结合，以生成 DCG 上近乎最佳的内存管理计划，这可以实现更高的性能。

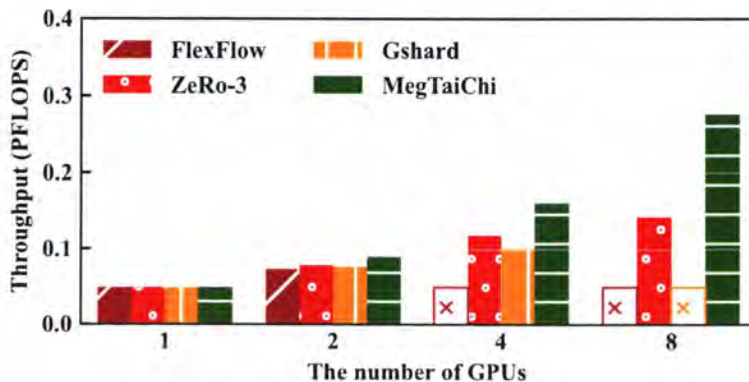


图 4.15 GPT 模型上的性能评估对比。

Figure 4.15 Performance evaluation comparison on GPT model.

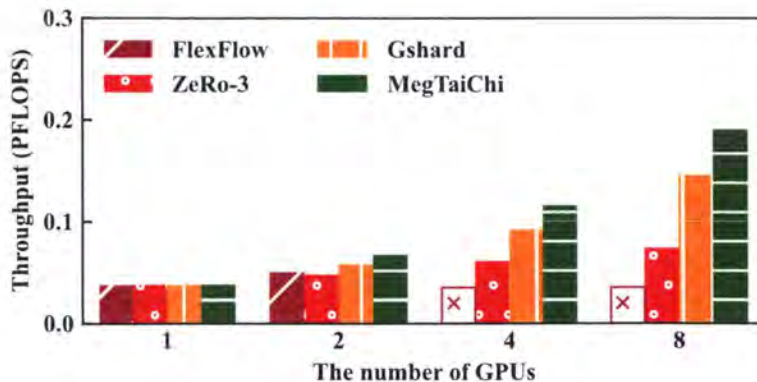


图 4.16 MoE 模型上的性能评估对比。

Figure 4.16 Performance evaluation comparison on MoE model.

4.3.5.4 大规模人脸识别模型的训练

最后的实验用大规模人脸识别模型的训练来评估 MegTaiChi。这些模型选自包括 GL-base 和 GL-large 的 Global Local 系列模型。在实际应用中，GL-large 支持超过一百万的人脸分类。GL-base 和 GL-large 的网络结构是交错卷积层和全连接层的架构。

从图 4.17，我们有四个重要的观察结果。首先，与经验策略 DP+MP 相比，DTP 平均实现了 $1.3\times$ 的性能加速。这是因为 DTP 通过在运行时平衡计算通信代价和内存使用来动态引导更合理的张量划分。其次，与 DTE 相比，DTP 和 DTE 的协同设计平均实现了 $1.2\times$ 的性能加速。这主要是由于 DTE 在内存限制约束下进一步实现了更大的 batch size。第三，平均而言，TMA 使 DTP 和 DTE 的协同设计进一步实现了 $1.2\times$ 的加速，这是由于细粒度的内存分配减少了内存碎片并避免了碎片整理过程。第四，通过将 DTP、DTE 和 TMA 结合在一起，与经验策略 DP+MP 相比，MegTaiChi 平均实现了 $1.87\times$ 的加速。

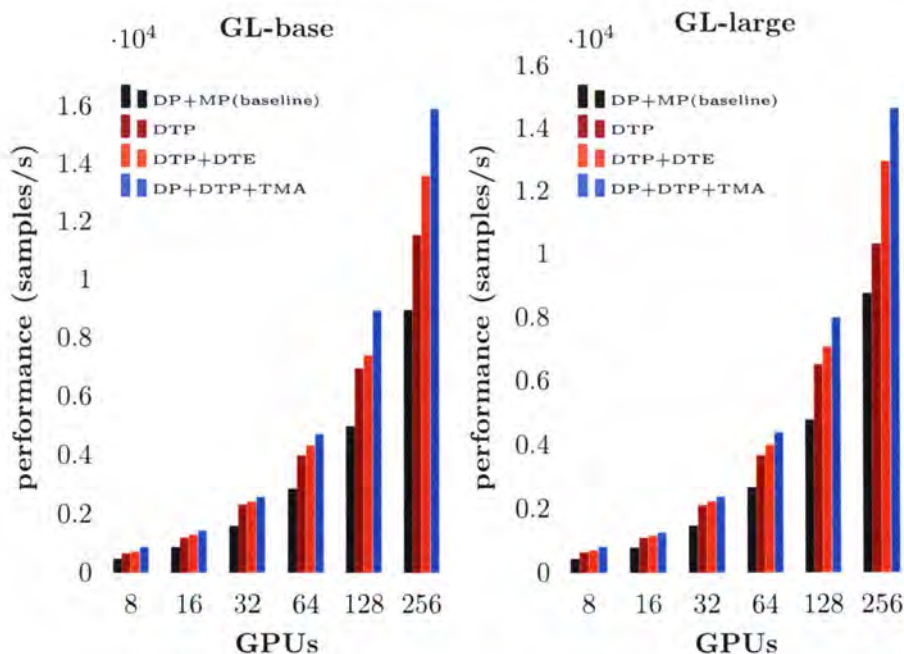


图 4.17 训练大规模人脸识别模型。

Figure 4.17 Training large-scale face recognition models.

4.4 相关工作

张量划分: 为了促进深度学习模型训练,当前的深度学习框架,例如 TensorFlow [87] 和 PyTorch [88] 提供了良好支持的数据并行和普通的模型并行通过将算子和模型显式分配给特定设备。Mesh-TensorFlow [74] 设计了一种特殊的专用语言来重写 DNN 模型以实现分布式训练。Tofu [89] 要求开发人员使用称为 TDL 的描述语言为算子指定张量切分。Megatron [28] 和 DeepSpeed [77] 成功地将张量模型并行性的实现与模型编程相结合。GShard [35] 使用并行注释来推断张量的划分。然而,这些主要的相关工作都是基于 SCG 的,并且它们的张量切分是在执行之前根据经验构建的。MegTaiChi 提出了一个动态张量切分策略,在训练过程中动态调整 DCG 上每个张量的切分。DTP 的设计考虑了运行时信息,并为有效协同张量切分和张量重新生成技术提供了机会。

张量重新生成: 基于计算图进行内存优化的工作主要包括与 host 交换显存策略和重新计算张量两大类。对于交换策略, vDNN [24] 和 SuperNeurons [25] 建议在前向阶段将数据交换到 CPU 并在后向阶段预取它,这样可以训练在有限的内存预算下的大型模型。SwapAdvisor [29] 设计了一种基于 SCG 自动生成交换

计划的遗传算法。此外, Capuchin [26] 代表了在 SCG 和 DCG 上研究交换策略的最先进的工作。它在运行时收集张量访问的信息, 用于内存管理。另一方面, 对于重新计算, Sublinear [2] 提出在前向传播中释放一些张量并在反向传播中重新计算它们, 这样可以在 N 层线性前馈网络用 $\mathcal{O}(\sqrt{N})$ 内存预算和 $\mathcal{O}(N \log(N))$ 额外的重新计算成本。Sublinear 很好地支持在静态计算图上有限的内存约束下进行大批量的训练。Checkmate [31] 分析 SCG 的模型训练并构造整数线性规划模型以找到最佳被驱逐的张量。据我们所知, DTR [1] 目前是在 DCG 上探索重新计算的最先进的工作。DTR 在执行期间应用启发式方法来指导哪些张量被驱逐。然而, DTR 只关注具有代价最小的张量驱逐策略, 而忽略了相同大小张量在内存中的位置变化。因此, 他们无法充分利用节省显存的机会。在这项工作中, MegTaiChi 提出了一种动态张量驱逐策略, 它利用不同内存位置的张量邻居内存块信息来减少内存碎片。此外, 张量内存分配旨在实现每个张量的细粒度内存管理。

4.5 小结

这项工作提出了 MegTaiChi, 这是一个基于动态张量的自动内存管理优化模块, 用于 DNN 训练, 实现了张量划分和张量重新生成的有效协同。具体来说, DTP、DTE 和 TMA 是为启发式、自适应和细粒度的内存管理而设计的。实验结果证实了 MegTaiChi 的高性能。

第5章 自动并行模块 MAP

5.1 并行策略自动优化

5.1.1 自动并行策略架构

如图5.1所示自动并行策略搜索模块的核心是首先根据上一章节提取出的单个算子的并行模式构造一个整图的搜索空间，然后给整图的搜索空间构造性能模型，进而选择合适搜索算法能够最快的搜索出最优的或者次优的并行策略。其中计算图每个节点有不同的切分策略，每条边代表不同的通信模式。

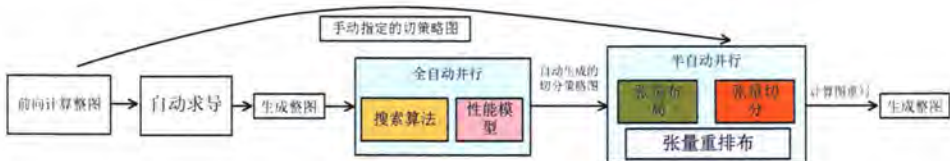


图 5.1 自动并行策略搜索框架。

Figure 5.1 Automatic parallel plan search framework.

5.1.2 分布式训练符号抽象

我们以 Oneflow[90] 深度学习训练框架中已有的数据和算子抽象为基础，由于 Oneflow 里已经存在一套 1-D 算子和数据的抽象描述，但是 N-D 的表达还不完善，因此我们将 1-D 的表达推广到 N-D。

Oneflow 中提供了数据的一致性视角，称为 **分布式张量描述**：分布式张量描述如图5.2所示，SBP 描述了“超级计算设备”一致性视角下的数据与集群中真实的物理设备上的数据的映射关系，它由 Split, Broadcast, Partial 的首字母组合而成。

- Split 表示物理设备上的物理 Tensor 是将一致性视角的逻辑 Tensor 切分得到的。物理切分时，需要指定切分的维度，图5.2展示了分别按行切分和按列切分。物理设备上的 Tensor 经过拼接可以还原得到一致性视角的 Tensor。

- Broadcast 表示一致性视角下的 Tensor 会复制并广播到所有的物理设备上。

- Partial 表示一致性视角下的 Tensor 与物理设备上的 Tensor 的形状相同，但是物理设备上的值，只是一致性视角下 Tensor 的一部分。以 partial sum 为例，

如果我们将集群中所有设备的张量按位置相加，那么就可以还原得到一致性视角的 Tensor。除了 SUM 外，MIN、MAX 等操作也适用于 Partial。

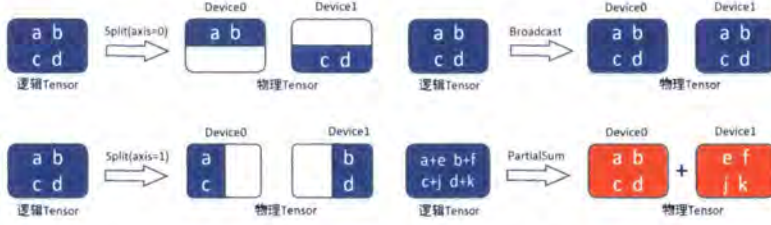


图 5.2 一个逻辑张量的 1-D 分布式描述。

Figure 5.2 1-D distributed description of a logic tensor.

Oneflow 中同时提供了算子的分布式表达，称为分布式算子标签：SBP 描述了一致性视角下的数据与物理设备上的数据的映射关系，当进行分布式训练时，根据数据的 SBP 属性，将数据分发到各个物理设备，进行计算，并输出结果。对于一个独立的 Tensor，我们可以任意设置它的 SBP 属性。但是，对于一个有输入、输出数据的算子，我们却不可以任意设置它的输入、输出的 SBP 属性。这是因为随意设置一个算子输入输出的 SBP 属性，可能不符合一致性视角下算子的运算法则。让我们以矩阵乘法为例观察一下在有 2 个设备的分布式系统中，矩阵乘法的输入、输出的 SBP 要如何组合才合法，如何组合不合法。

假设一致性视角下有一个形状为 (m, k) 的矩阵 A 与形状为 (k, n) 的矩阵 B 相乘得到 Y ，依据矩阵乘法的规律 Y 的形状必然为 (m, n) ，我们可以将矩阵 A 按第 0 维进行切分，切分为形状分别为 (m_0, k) ， (m_1, k) 的两个矩阵： A_0 和 A_1 ，然后在 2 个设备上分别计算：

设备一：

$$A_0 \times B = Y_0$$

$$(m_0, k) (k, n) (m_0, n)$$

设备二：

$$A_1 \times B = Y_1$$

$$(m_1, k) (k, n) (m_1, n)$$

我们容易得到物理设备上的 A_0, A_1 与一致性视角 A 的关系，以及 Y_0, Y_1 与一致性视角数据 Y 的关系：

$$\begin{aligned}
 A &= \text{concat} \quad (A_0, \quad A_1) \\
 (m, k) & \quad (m_0, k) \quad (m_1, k) \\
 Y &= \text{concat} \quad (Y_0, \quad Y_1) \\
 (m, n) & \quad (m_0, n) \quad (m_1, n)
 \end{aligned}$$

按照以上的方式，将一致性视角的数据分发到各个物理设备上，能够完成运算并且最终得到一致性视角上的正确结果。以上较为复杂的表达，若使用 SBP 来描述，会变得简单： A 为 Split(0), B 为 Broadcast, 运算结果 Y 为 Split(0)。对于矩阵乘法而言，其输入输出的 SBP 按以上方式组合是合法的。对于矩阵乘法而言，合法的 SBP 组合不止一种，比如还可以是如表 5.1 所示列举了 6 种合法的矩阵算子签名。

表 5.1 1D-SBP MUL 算子分布式签名集合。

Table 5.1 Distributed signature set for 1D-SBP MUL operator.

X	W	$Y = XW$
S(0)	B	S(0)
B	$S(1)$	$S(1)$
S(0)	$S(1)$	$P(\text{sum})$
$P(\text{sum})$	B	$P(\text{sum})$
B	$P(\text{sum})$	$P(\text{sum})$
B	B	B

5.1.3 分布式通信自动路由

Oneflow 提供了子图生成模块**自动路由模块**，负责构建两个逻辑上的 Op 对应的两组物理上的 Op 在任意情形下的物理子图，完成了分布式训练中各个机器各个设备之间的数据拷贝、切分、传输、通信的子图搭建。如图 5.3 所示描述了 Split(1)->Broadcast 的过程。在这种情况下，我们需要合并先前通过切分原始大张量的第一维获得的张量。为了保持一致性，自动路由会自动插入 AllGather 通信原语进而满足下游 Op 需要的输入数据。

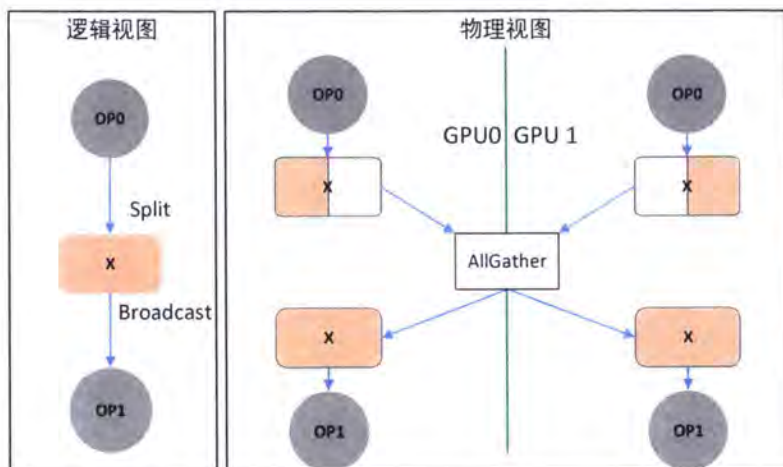


图 5.3 自动路由机制完成张量从 S(1) 到 B 的物理数据重排。

Figure 5.3 The data redistribution from S(1) to B by automatic routing mechanism.

5.1.4 推广到高维

多维设备拓扑: 原本的一维 SBP 是假设所有的设备是一个一维的向量, 由其排序的设备编号 ID 顺序表示。如单个节点 4 GPUs 为 machine_id: 0, device_id: 0-3, 则表示为 0:0-3, 4 个设备 id 分别是 0, 1, 2, 3。如 2 个节点 8 GPUs 表示为 0:0-3; 1:0-3, 8 个设备 id 分别是 0, 1, 2, 3, 4, 5, 6, 7。如果是 S(0), 则会将逻辑 Tensor 均匀切分到这 8 个设备上。如果是 B, 则广播到这 8 个设备上。为了支持 2D SBP, 首先我们对设备进行扩维: 假设共有 8 个设备, 编号分别是 0, 1, 2, 3, 4, 5, 6, 7。原本一维的设备拓扑 DeviceShape = (8), 这 8 个设备按照 [0, 1, 2, 3, 4, 5, 6, 7] 的方式排列。当设备拓扑扩展为二维时: 一种合法的 2 维设备拓扑 DeviceShape = (2, 4)。则这 8 个设备按照: [[0, 1, 2, 3], [4, 5, 6, 7]] 的方式排列。在设备拓扑的第 0 维上, 共分为 2 组, 设备 [0, 1, 2, 3] 是一组, [4, 5, 6, 7] 是另一组。在设备拓扑的第 1 维上, 共分为 4 组, 分别是: [0, 4], [1, 5], [2, 6], [3, 7]。

张量的多维 SBP 描述: 多维的 SBP 描述是对设备拓扑上每一维度分别设置一个 SBP。先举一个多维跟一维等价的情况: DeviceShape=(8), 假设某个 Tensor 的 SBP 描述为 S(0), 该张量的逻辑形状为 LogicalShape = (8, 20, 28), 表示 Tensor 的第 0 维会均匀的切成 8 份, 每个物理上的 Tensor 的物理形状为 PhysicalShape=(1, 20, 28)。这种情况等价于: DeviceShape=(2, 4), 该 Tensor 的 SBP 描述为 (S(0),

$S(0))$ 。

假设该 Tensor 是模型参数, 其 $\text{LogicalShape}=(8, 20)$, 我们举一种 Megtron[28] 中的列并行的例子: 假设数据并行度是 4, 模型并行度是 2, 那么在 Oneflow 的 2D SBP 中的描述为: $\text{Device Shape}=(2, 4)$, SBP 签名为 $(B, S(1))$, 逻辑图转化为物理图如 5.4 所示。

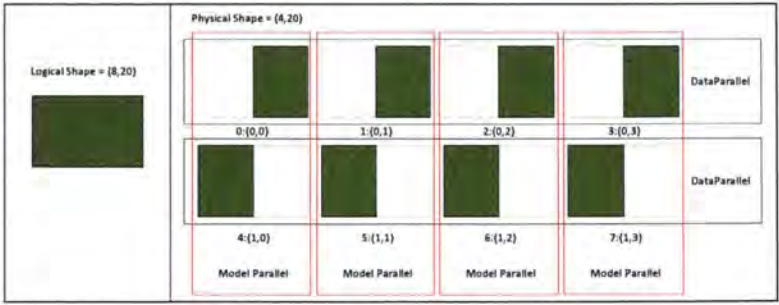


图 5.4 2D-SBP 中逻辑张量的物理排布。

Figure 5.4 Physical layout of the logical tensor in 2D-SBP.

其中设备 ID 编号 $[0, 1, 2, 3]$ 和 $[4, 5, 6, 7]$ 分别做数据并行; 设备 ID 编号 $[0, 4], [1, 5], [2, 6], [3, 7]$ 共 4 组分别做模型并行。则上述的数据并行度 4, 模型并行度 2 的情况可以被 DeviceShape 和 SBP 签名共同描述。

表 5.2 矩阵乘算子的 2D-SBP 分布式签名集合。

Table 5.2 2D-SBP distributed signature sets for MUL operator.

X	W	$Y = XW$
$(S(0), B)$	$(B, S(1))$	$(S(0), S(1))$
$(S(0), S(1))$	$(B, S(0))$	$(S(0), P)$
$(S(0), S(1))$	$(B, S(1))$	$(S(0), S(1))$
...	...	...

2D-SBP 自动路由: 自动路由子图构建需要考虑 2D SBP 配合 2 维设备拓扑的情形。例如: 设备的拓扑 $\text{DeviceShape}=(2, 4)$, 某个 Tensor 在生产者 Op 看来其 SBP 签名为 $(B, S(1))$, 在其消费者 Op 看来其 SBP 签名为 (B, B) (纯数据并行), 那么其实只是第一维的 $S(1) \rightarrow B$ 的问题。即调用 AllGather 就可以实现, 路由子图构建为图 5.6 所示。

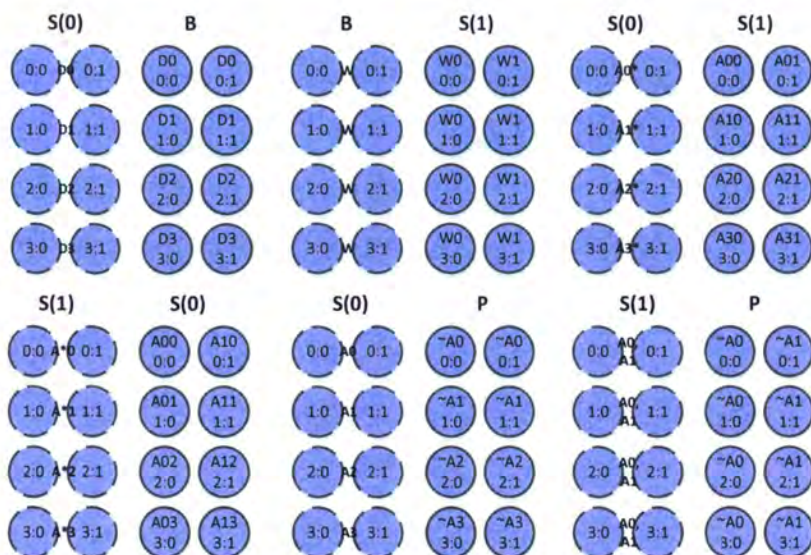


图 5.5 分布式张量的 2D-SBP 可能的描述种类。

Figure 5.5 Possible description of 2D-SBP for distributed tensor.

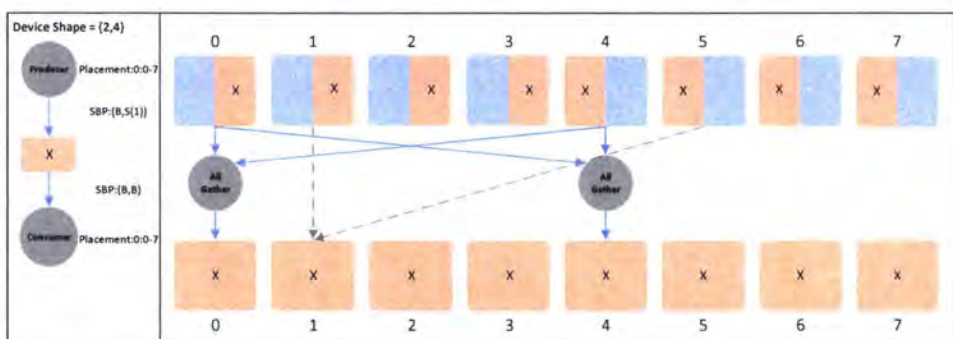


图 5.6 2D-SBP 自动路由将逻辑子图生成物理子图。

Figure 5.6 2D-SBP automatically routes physical subgraphs from logical subgraphs.

从图5.6中我们可以看出：2D SBP 就是在对应的设备拓扑维度上分别（按照分组）插入相应的 SBP 通信路由操作。上述图5.6所示是简单的 $(B, S(1)) \rightarrow (B, B)$ 。如果是更复杂的情形呢？例如图5.7所示，假如是 $(B, S(1)) \rightarrow (S(0), B)$ 的情形，那么需要分两步插入 SBP 通信路由操作，可以按照 $(B, S(1)) \rightarrow (B, B) \rightarrow (S(0), B)$ 或者 $(B, S(1)) \rightarrow (S(0), S(1)) \rightarrow (S(0), B)$ 两种策略。但是不同的策略通信量可能不同。下图可以解释这两种通信路由子图构图策略：

方式一： $(B, S(1)) \rightarrow (B, B) \rightarrow (S(0), B)$ ，如图5.8所示，先插入 AllGather 通

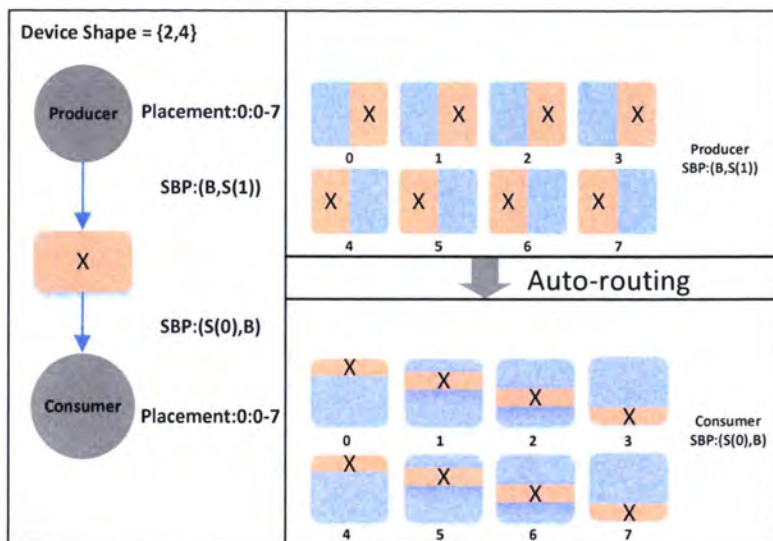


图 5.7 2D-SBP 自动路由将逻辑子图生成物理子图。

Figure 5.7 2D-SBP automatically routes physical subgraphs from logical subgraphs.

信，再插入 Scatter 通信。

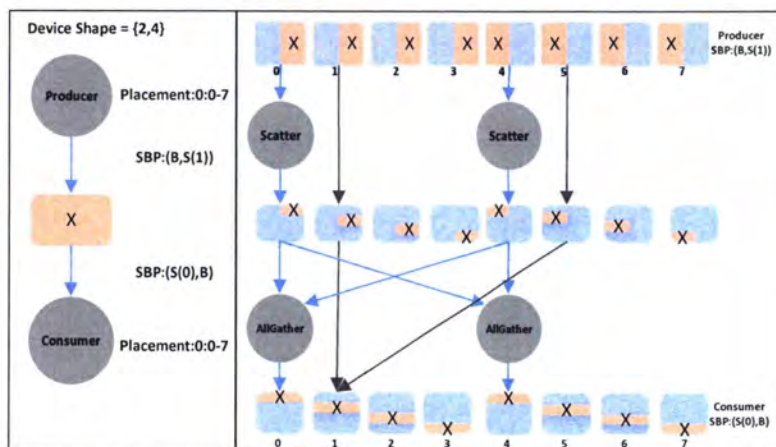


图 5.8 2D-SBP 自动路由将逻辑子图生成物理子图 (方式一)。

Figure 5.8 2D-SBP automatically routes logical subgraphs to physical subgraphs(Method 1).

方式二：(B, S(1)) -> (S(0), S(1)) -> (S(0), B)，如图5.9所示，先插入 Scatter 通信，再插入 AllGather 通信。

因此我们可以把一个矩阵乘映射到一个 2D-SBP 描述上，图 5.5列举了两个张量矩阵乘的两种可能的 2D SBP 描述，表5.2给出了矩阵乘算子的部分合法算子签名。而对于一个矩阵计算来讲选用不同的 SBP 签名无论是 1D 还是 2D 通信代

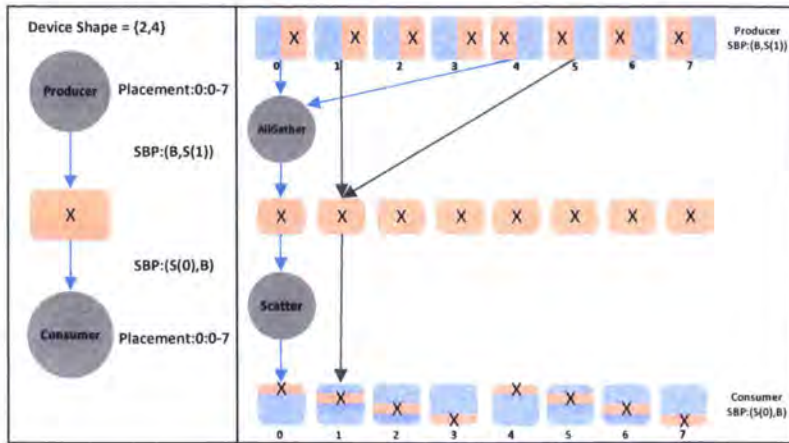


图 5.9 2D-SBP 自动路由将逻辑子图生成物理子图 (方式二)。

Figure 5.9 2D-SBP automatically routes logical subgraphs to physical subgraphs(Method 2).

表 5.3 矩阵乘中不同的 SBP 签名通信量不同。

Table 5.3 Different SBP signatures have different communication cost in MUL.

类型	A	B	C	通信量
SUMMA	$S(0), S(1)$	$S(0), S(1)$	$S(0), S(1)$	$m * k/q + k * n/q$
2D	$S(0), S(1)$	$B, S(1)$	$S(0), S(1)$	$m * k/q$
1D 模型列切分	B	$S(1)$	$S(1)$	$m * k$
1D 模型行切分	$S(1)$	$S(0)$	P	$m * k$
无通信	$S(0), B$	$B, S(1)$	$S(0), S(1)$	0

价和计算代价都是存在差异的如表5.3 (对于矩阵乘 $C=AB$, $(m, n) = (m, k) * (k, n)$, 假设在 p 个 GPU 设备上运行, 2D 时并行维度为 $p = q * q$), 因此我们也需要一个性能模型和搜索算法来决策出每个算子应该选用哪种 SBP 签名。

5.1.5 性能模型

计算代价: 对于每个逻辑结点, 我们衡量该逻辑结点在不同的备选数据并行决策下的计算时间, 也就是计算量除以计算速度。我们可以通过大量实验来测量估算出单位计算量所需要的运行时间。我们称这个计算时间为逻辑结点的计算代价, 并储存在一个数组里以保证在 $O(1)$ 时间复杂度内能找到该逻辑结点在特定备选并行决策下的计算代价。

传输代价: 因为计算图是有向图, 所以计算图的每一条边代表着数据从这条

边的起始结点传输到这条边的终端结点。我们衡量这条边在两端结点取不同数据分布决策时所需要的传输时间并称之为传输代价。具体可以通过大量实验来测量带宽并估算出这两个结点间传输单位计算量所需要的时间。最后把这条边在两端的不同决策下的传输代价储存在这条边的一个二维数组里以保证常数的访问时间。

建模：给定一个计算子图，这个计算子图可能只是整个计算图的一部分，也可能就是整个计算图。给定了每个逻辑结点的备选并行决策，并且计算出了每个逻辑结点在不同并行决策下的计算代价以及每条边在端点不同并行决策下的传输代价，给定了设备数或者节点的数量。假定该计算子图内每一段程序都会在各个设备上同时进行，那么运行完整个计算子图所需要的时间约等于各设备运行完所有程序的时间。根据这个我们可以建立均分代价模型。在均分代价模型下，当所有逻辑结点都决定了一个并行决策时，各设备的计算代价以及卡间的传输代价都是一样的。我们需要选定一个决策，该决策决定了每一个逻辑结点该取什么样的并行决策，使得每一个设备上总的代价最小。在这个模型下，我们把原有向无环图转换成无向图。

本文引入了一个代价模型来定量评估不同并行化策略的运行时间，并使用一个基于贪心算法的图搜索算法来搜索出一个较优的并行策略。在我们的代价模型和搜索算法下找到一个最佳的并行化策略。该代价模型取决于以下假设。

- (1) 对于一个层 $l_i \in \mathcal{G}$ ，处理 l_i 的时间是可以预测的且变化不大，而且基本上与输入数据的内容无关。
- (2) 运行时系统的开销可以忽略不计，一旦输入张量可用并且设备完成先前层的任务，设备就会开始处理本层。

我们在计算图上定义了两个代价函数：

- (1) 对于每一层 l_i 及其并行化配置 c_i ， $t_c(l_i, c_i)$ 是处理层 l_i 下配置 c_i 的时间。这表示计算单个算子的时间，并通过在设备上多次执行该配置下的算子并测量平均执行时间来估计。
- (2) 对于每个张量 $e = (l_i, l_j)$ ， $t_x(e, c_i, c_j)$ 利用要移动的数据大小和已知的通信带宽来估计将输入张量传输到目标设备的时间。

使用上面的两个代价函数，我们定义：

$$t_O(G, D, S) = \sum_{l_i \in G} \{t_c(l_i, c_i)\} + \sum_{e=(l_i, l_j) \in G} t_x(e, c_i, c_j) \quad (5.1)$$

$t_O(G, D, S)$ 估计并行化策略 S 的每步执行时间，包括前向处理、反向传播，和参数同步。

上面的形式化表达式是将寻找最优并行化策略的问题表达为图搜索问题：我们的目标是找到一个策略 S ，使得总运行时代价 $t_O(G, D, S)$ 被最小化。

均分代价模型的核心算法：在均分代价模型下，定义当前计算图所有可能的情况为备选空间，它的大小为当前所有逻辑节点的备选决策的数量的乘积，所以这是一个 NP-hard 的问题。所以可以通过一些算法进行剪枝，然后再在剪枝后的计算图上运用贪心算法得到最优解/次优解。剪枝算法包含了点消除，边消除，树消除，点融合以及局部贪心算法。其中点消除跟边消除是根据工作 [39] 得到的。点消除以及树消除都是能减小备选空间的算法。边消除以及点融合都是为了更好进行消除的算法。局部贪心算法可以作用在任何一个计算图上得到次优解的算法。当计算图不能再被剪枝时，本文将运用局部贪心算法来得到最终的解。如果最终的计算图的每一个连通分量的逻辑结点数不超过 3 个，那么局部贪心算法会得到最优解。

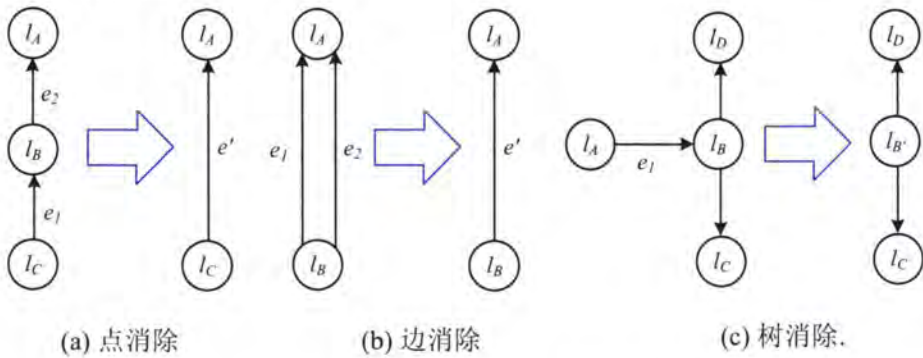


图 5.10 三种基本的图消去方式。

Figure 5.10 Three basic graph elimination methods.

点消除：如图5.10所示，如果一个逻辑结点 B 的度数为 2（原有向图里入度 + 出度 = 2），则该结点的决策的取定可以依赖于两个相邻结点 A 跟 C 。把计算图

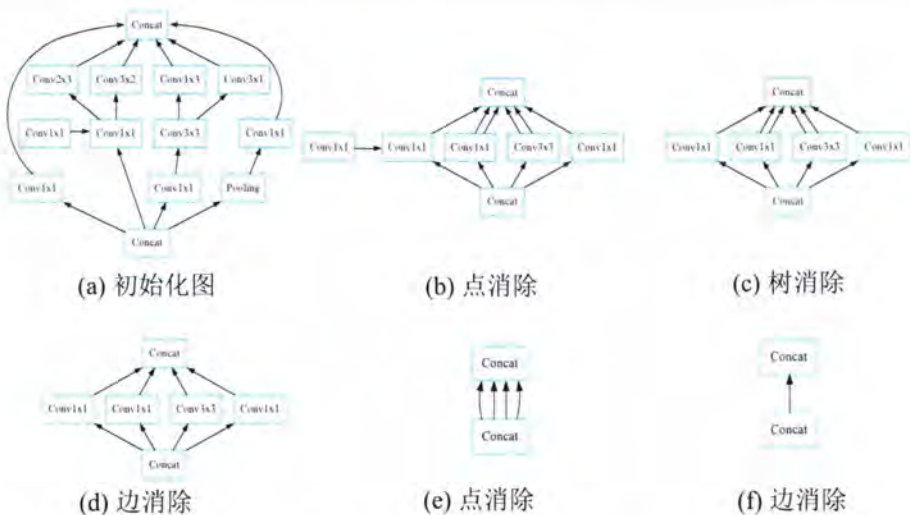


图 5.11 计算图如何通过三种消去方式做图消去。

Figure 5.11 Graph elimination for computational graphs by three elimination methods.

该结点 B 以及连接该节点的两条边 BA 与 BC 合成一条新的边 AC，这条新的边 AC 连接着该逻辑结点连接的两个结点 A 跟 C。任意给定两个相邻结点 A 与 C 的决策，遍历该节点 B 所有决策使得该结点 B 与连接该节点的两条边 BA，BC 的代价之和最小，并把这个和定义为新的边 AC 在 A 跟 C 给定决策组下的代价。遍历 A 跟 C 的所有决策组，找出每个决策组下新的边 AC 的代价并储存在边 AC 的二维数组中。如果一个计算图 G 包含一个节点 l_j 和一个单入边 $e_1 = (l_i, l_j)$ 和一个单出边 $e_2 = (l_j, l_k)$ ，节点消除删除来自 G 的节点 l_j 和两条边 e_1 和 e_2 ，将新边 $e' = (l_i, l_k)$ 插入回 G ，并返回修改后的图。以保留最优并行化策略的方式定义 $t_x(e', \cdot, \cdot)$ 。

$$t_x(e', c_i, c_k) = \min_{c_j} \{t_c(l_j, c_j) + t_x(e_1, c_i, c_j) + t_x(e_2, c_j, c_k)\} \quad (5.2)$$

直观地说，我们使用动态规划为节点 l_j 计算 c_i 和 c_k 的每个可能组合的最优配置 c_j ，并使用与 l_j 来定义 $t_x(e', c_i, c_k)$ 。

定理 1. 假设 $G' = \text{NodeElimination}(G)$ 并且 l_j 是被消除的节点。如果 S_o' 是 G' 的最优并行化策略，则 $S_o = S_o' + c_j$ 是 G 的最优并行化策略，其中 c_j 最小化方程 5.2。

边消除：如图5.10所示，如果两个逻辑结点 A、B 间有多条边，则可以合并为一条边。任意给定两个端点 A 与 B 的决策，新的边的代价为原所有边代价之和。变换端点的决策组，求出原边的代价之和并储存在新的边的二维数组中。

如果一个计算图 G 包含两条具有相同源节点和目标节点的边（即 $e_1 = (l_i, l_j)$ 和 $e_2 = (l_i, l_j)$ ），边消除从 G 中移除 e_1 和 e_2 ，将一条新边 $e' = (l_i, l_j)$ 插入到 G 中。我们使用 $t_x(e_1, \cdot, \cdot)$ 定义 $t_x(e', \cdot, \cdot)$ 和 $t_x(e_2, \cdot, \cdot)$ 。

$$t_x(e', c_i, c_j) = t_x(e_1, c_i, c_j) + t_x(e_2, c_i, c_j) \quad (5.3)$$

定理 2. 假设 $G' = \text{EdgeElimination}(G)$ ， $S_{O'}'$ 是 G' 的最优并行化策略，则 $S_O = S_{O'}'$ 是最优的 G 的并行化策略。

树融合（树消除）：如图5.10如果一个逻辑结点 A 的度数为 1（原有向图里入度 + 出度 = 1），则该节点的决策的取定可以依赖于它的相邻结点 B。把该结点 A 以及连接该节点的一条边 AB 融进它的相邻结点 B 里面，也就是删除结点 A 以及连接的边 AB 并更新 B 的代价数组。任意给定结点 B 的决策，遍历该结点 A 的所有决策使得该结点与连接该节点的边 AB 的代价之和最小，并把这个和加到相邻结点 B 在给定决策下的代价里面。遍历 B 的所有决策，找出 B 的每个决策下 A 的对应决策并更新 B 的代价。

如果一个计算图 G 包含一个节点 l_j 和一个单入边或者一个单出边 $e = (l_j, l_k)$ ，节点消除删除节点 l_j 和一条边 e 来自 G 并合并为一个新的节点，将新节点插入回 G ，并返回修改后的图。我们以保留最优并行化策略的方式定义 $t_c(l_{j'}, c_{j'})$ 。

$$t_c(l_{j'}, c_{j'}) = \min_{c_j} \{t_c(l_j, c_j) + t_c(l_i, c_i) + t_x(e, c_i, c_j)\} \quad (5.4)$$

点融合：如果上述三种消除算法都无法在当前计算图进行下去而且该图的某个连通分量里仍有 2 个以上的结点，也就是在该连通分量里，每一个逻辑结点的度数都大于等于 3 的情况下，则需要一个算法来改变图的拓扑结构来推进，使得消除算法能继续下去。找到两个适合的结点 A 与 B，把他们合并成一个逻辑结点 C。新结点 C 的决策为两个原结点 A、B 的决策的笛卡尔积。假设 A 有 m 种决策，B 有 n 种决策，那新结点 C 有 $m * n$ 种决策。原结点 A 取第 i 种决策，原

结点 B 取第 j 种决策对应着新结点 C 取第 $(i-1)*n+j$ 种决策。新结点 C 取第 k 种决策对应着原结点 A 取第 $[(k-1)/n+1]$ 种决策, 原结点 B 取第 $[(k-1)\%n+1]$ 种决策。换成以 0 为基的数据表示就是 A、B 的 (i, j) 决策组对应着 C 的 $i*n+j$ 决策, C 的决策 k 对应着 A、B 的 $(k/n, k\%n)$ 决策组。新结点 C 在备选决策 k 下的代价 = 原结点 A 在决策 k/n 的代价 + 原结点 B 在决策 $k\%n$ 的代价 + 连接两原结点的边 AB 在决策组 $(k/n, k\%n)$ 的代价。如果两原结点之间没有边, 则去掉最后一项。把只连接到原结点之一的边全部转移为连接新结点的边。然后对新结点进行边消除就能得到新的边。把原结点 A、B 合并为新结点 C, 如果存在两条边 AD、BD, 合并后新的边为 CD。假设在 A、B、D 的决策分别为 i, j, l 。则 CD 在决策组 $(i*n+j, l)$ 下的代价 = AD 在决策组 (i, l) 下的代价 + BD 在决策组 (j, l) 下的代价。如果只有连接到其中一个结点而没有连接到另一个结点的边, 则在代价计算时把对应项去掉, 遍历所有连接 A 或者 B 的边, 代价转移边, 遍历所有决策组, 更新边的代价数组。

寻找用于点融合的两个结点, (1) 两个原结点决策数量的积不能超过一个阈值 λ 。如果没有设置阈值, 在一个带有 n 个结点的完全图里, 点融合会一直进行下去, 最后生成一个备选决策数量为所有原结点备选决策数量的乘积的大结点, 此时整个算法无法在一定时间内解决原问题。同时设置阈值可以限制结点与边占用数据空间的大小。合并后结点占用空间大小为 $O(\lambda)$, 合并后边占用空间大小为 $O(\lambda^2)$ 。对于原结点与原边不做限制, 输入备选决策数量大于阈值的结点是可以接受的, 只是该结点不会再跟别的点进行合并。(2) 他们直接邻域重合度尽可能高。直接邻域是指一个逻辑结点以及与其连接的所有逻辑结点的集合。对于任意两个结点, 如果他们备选决策的积小于等于阈值, 利用位运算的集合比较算法来找出他们直接邻域的重合度, 记录下重合度最高的一对结点用于点融合。重合度越高, 消除的边就越多。

全局缩小备选空间的算法: (1) 循环检索, 记一个逻辑结点的检索为检查该点是否能进行点消除 (度数是否为 2), 是否能进行树消除 (度数是否为 1), 是否能进行边消除 (是否有连接到同一个结点的两条不同的边)。如果检索出能进行某一个消除, 则立马执行该消除并马上重新检索该结点。(2) 建立一个待检索的队列, 一开始全部结点入队。每次出队一个结点并进行检索一直到队列为空。建立并维护一个结点是否在队列里的标签, 标签为真当且仅当该结点在队列里。把

入队重新定义为如果该结点不在队列里，则放到队尾，否则不做任何操作。(3) 执行消除后的入队操作，执行树消除以后被消除的结点依赖的结点入队。执行边消除以后边的两个端点入队。执行点消除以后新合成的边的两个端点入队。执行点融合以后合并成的新结点的直接邻域入队。当待检索的队列为空时，才会寻找用于点融合的两个结点。如果找到了，进行点融合，重新开始检索非空的待检索队列。如果没有找到，计算图已经不能再进行简化。此时应用局部贪心算法得出最终解。

5.1.6 局部贪心算法

局部贪心算法就是每次取一个计算图的逻辑结点子集，遍历他们所有备选决策组，取出使得计算图总代价最小的一个决策组。对不同的逻辑结点子集操作多次使得总代价逐渐降低。

点局部贪心算法：点局部贪心算法就是循环遍历整个计算图直到达到一定次数 n 次或者某次遍历计算图时代价不再减少。 n 可以取当前计算图的逻辑结点数量。定义点邻域代价为一个逻辑结点在它的一个备选决策下，结点的代价与它的所有边的代价之和。点贪心操作就是对一个逻辑结点遍历其所有的备选决策，取使得点邻域代价最小的备选决策。点局部贪心算法的一次计算图遍历就是遍历该计算图的所有结点，对每个结点执行点贪心操作。

边局部贪心算法：如图5.12边局部贪心算法就是循环遍历整个计算图直到达到一定次数 n 次或者某次遍历计算图时代价不再减少。 n 可以取当前计算图的逻辑结点数量。定义边邻域代价为一条边在它的两个端点的一个备选决策组下，两端点的点邻域代价之和再减去边的代价（因为在计算两端点的点邻域代价时当前边被计算了两次，而对每个拓扑结构我们都只需要计算一次）。边贪心操作就是对一条边遍历其两个端点所有的备选决策组，取使得这条边的边邻域代价最小的备选决策组。边局部贪心算法的一次计算图遍历就是遍历该计算图的所有边，对每条边执行边贪心操作。

算法分析：对于消除与合并的算法，以上四个消除以及融合的算法首先不会增大备选空间，但不一定缩小备选空间。这四个算法一定会消除计算图的拓扑结构，所以能执行的次数一定是有限的。记 M 为所有逻辑结点备选决策数量与阈值中的最大值。 L 是合并的两个原结点的度数之和。

局部贪心算法：对一个计算图的逻辑结点子集进行一次贪心操作的复杂度

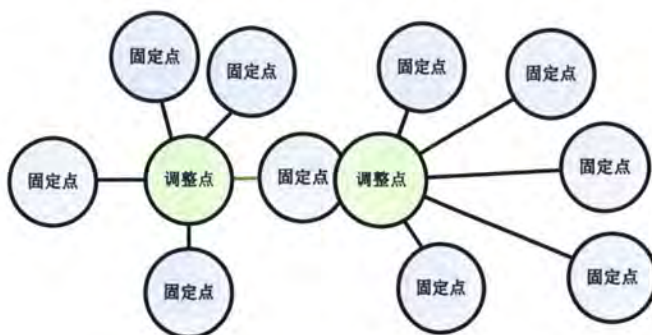


图 5.12 边局部贪心算法。

Figure 5.12 Local greedy algorithm on the edge.

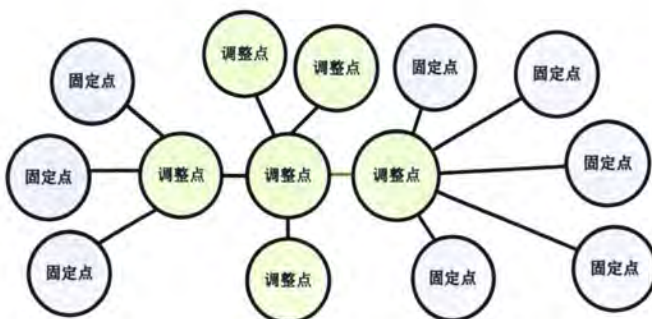


图 5.13 邻域局部贪心算法。

Figure 5.13 Neighborhood local greedy algorithm.

表 5.4 图消去算法的算法复杂度。

Table 5.4 Algorithmic complexity of graph elimination algorithm.

算法	是否减少解空间	消除的点拓扑结构	消除的边拓扑结构	算法复杂度
点消除	是	1 个	1 条	$O(M^3)$
树融合	是	1 个	1 条	$O(M^2)$
边消除	否	0 个	k 条, $k \geq 1$	$O(kM^2)$
点融合	否	1 个	k 条, $k \geq 1$	$O((k+L)M^3)$

为 $O(eMp)$, p 为该子集的逻辑结点数量, e 是该逻辑结点子集的直接邻域包含边的数量。在点贪心操作时 $e \leq n$, 在边贪心操作时 $e \leq 2n$. 定义优化率 = (原代价 - 应用算法后代价) / (原代价 - 最小能达到的代价)。

在缩小备选空间的算法部分, 因为每次都至少消除 1 条边, 所以算法复杂度为 $O(EM^3)$, E 为初始计算图边的最大值。在局部贪心算法部分, 因为边局部贪

表 5.5 贪心算法的优化率。

Table 5.5 Optimization rate of greedy algorithm.

算法	算法复杂度	优化率
点局部贪心算法	$O(n^3M)$	92%
边局部贪心算法	$O(n^3M^2)$	98%

心算法的优化率更高，所以我们尽可能使用边局部贪心算法。对于只有一个逻辑结点的连通分量，我们使用点局部贪心算法（因为该连通分量没有边），对于有多个逻辑结点的连通分量，我们使用边局部贪心算法。所以全局算法的复杂度为 $O(EM^3 + n^3M^2)$ 。其中 n 为简化后计算图的逻辑结点个数而非初始计算图的逻辑结点个数。

为每个算子找到一种策略，以使训练中的迭代时间最小化，前提是峰值内存消耗小于设备内存容量。最终总的性能模型定义为：

$$\begin{aligned}
 m(S^k) &= \sum_{o_i \in V} m(o_i, s_i^k) \\
 t(S^k) &= \sum_{o_i \in V} t_c(o_i, s_i^k) + \sum_{e_{ij} \in E} t_x(e_{ij}, s_i^k, s_j^k)
 \end{aligned} \tag{5.5}$$

5.1.6.1 搜索算法

我们首先提取出了全局图的三种基本网络结构如图5.10所示，我们基于图消去初步提出了一个支持张量重排布的贪心搜索算法具体执行流程如下：

- 为所有算子生成合法策略
- 遍历计算图
- 在计算图用图消去算法缩减计算图
- 用点融合继续削减计算图
- 继续遍历计算图进行图消去，如果无法进行图消去，则完成模型简化

算法 3 寻找较优的并行策略 S **Require:** 计算图 $\mathcal{G}$, 设备图 $\mathcal{D}$, 以及提前计算的代价方程 (例如: $t_C(\cdot)$ 和 $t_x(\cdot)$)**Ensure:** 并行策略 S 最小化 $t_O(\mathcal{G}, \mathcal{D}, S)$

```

1:  $\mathcal{G}^{(0)} = \mathcal{G}$ 
2:  $m = 0$ 
3: while true do
4:    $\mathcal{G}^{(m+1)} = \text{点消去}(\mathcal{G}^{(m)})$ 
5:    $\mathcal{G}^{(m+2)} = \text{边消去}(\mathcal{G}^{(m+1)})$ 
6:    $\mathcal{G}^{(m+3)} = \text{树融合}(\mathcal{G}^{(m+2)})$ 
7:   if  $\mathcal{G}^{(m+2)} = \mathcal{G}^{(m)}$  then
8:     break
9:   end if
10:   $m = m + 3$ 
11: end while
12: 找到一个最优的并行策略  $S^{(m)}$  对计算图  $\mathcal{G}^{(m)}$  通过贪心算法遍历所有的候
    集中的策略
13: for  $i = m - 1$  to 0 do
14:   if  $\mathcal{G}^{(i+1)} = \text{点消去}(\mathcal{G}^{(i)})$  or  $\text{点融合}(\mathcal{G}^{(i)})$  then
15:      $\triangleright$  假如  $l_j$  是从计算图  $\mathcal{G}^{(i)}$  中消去的点
16:     找到配置  $c_j$  最小化方程 5.1
17:      $S^{(i)} = S^{(i+1)} + c_j$ 
18:   else
19:      $S^{(i)} = S^{(i+1)}$ 
20:   end if
21: end for
22: return  $S^{(0)}$ 

```

5.2 点融合的选择优化

以前的点融合算法会挑取一个图里共同连接节点数最多的一对节点进行合并，这种挑取保证了能尽可能地削减拓扑结构，减少了 n 或者 $n-1$ 条边（ n 为共同连接节点数）。但是存在以下一种情况，例如：

$$S0- > S1- > S1- > S1- > S1- > S0$$

上面为一个张量在几个节点内流通时的 SBP，每个节点的度数都大于等于 3，以致于不能进行任何的消除算法。那最好的策略应该是：

$$S0- > S0- > S0- > S0- > S0- > S0$$

然而在调整策略下不会有任何变动，上面节点的任意三个以内连续的 $S1$ 变为 $S0$ ，它的总传输代价都不会减少，所以调整代价的减少没有少于原代价，则取消调整。上述情况也印证了使用消除算法来简化图拓扑的必要性。在上述情况存在的前提下，搜索出来的策略往往不够好。

改善上述的方法有两种，一个是延长调整节点的长度，该方法会增大算法复杂度。另一个是多进行点融合。如果把这几个点合成一个点，那么通过搜索就能找出最优的那个策略。但是一般的点融合不会缩减策略空间，所以就是合成出来的这个点它本身可能的策略为原合并点的策略的乘积，5 个点最高能到 100 的 5 次方。在阈值的限制下，我们不允许有策略数超过 100 的最终乘积。

但是有一些边，它要求上下游具有相同的 SBP 策略，比如 weight 算子跟对应的 optimizer 算子如表 5.6 所示。

那么合并这条边的两个 SBP 得到的策略数是 6，该次合并使得策略空间缩小到六分之一。更有甚者，上下游的 SBP 数量并不一致，比如 ResNst-50 里面 Resnet-res4_2_branch2c_bn-gamma 这个 variable_op 有 6 种可能的策略，(B, S0, S1, S2, S3, S4)，一般的 variable_op 都是这六种策略，然而其对应的 Resnet-res4_2_branch2c_bn-gamma_optimizer 只接受两种策略 (B, S0)，所以该边的代价是如表 5.7 所示。

在 5.1 章节的融合算法中选择融合这种边的时候会剔除边代价超过 e^{38} 的 SBP 组合。此时就是优先挑选尽量削减策略空间的边来进行合并。首先比较并

表 5.6 上游 weight 算子和下游 optimizer 算子之间的代价。

Table 5.6 Cost of between the upstream weight OP and the downstream optimizer OP.

Optimizer \ Weight	Weight	B	S0	S1	S2	S3	S4
B		0	∞	∞	∞	∞	∞
S0		∞	0	∞	∞	∞	∞
S1		∞	∞	0	∞	∞	∞
S2		∞	∞	∞	0	∞	∞
S3		∞	∞	∞	∞	0	∞
S4		∞	∞	∞	∞	∞	0

表 5.7 部分算子只支持少量 SBP 可以减小搜索空间。

Table 5.7 The operator that supports few SBPs can reduce the search space.

Optimizer \ Weight	Weight	B	S0	S1	S2	S3	S4
B		0	∞	∞	∞	∞	∞
S0		∞	0	∞	∞	∞	∞

找出最小的 $cut_ratio = (\text{组合代价} < e^{38} \text{ 个数}) / (\text{所有的组合})$ 。优先合并具有最小的 cut_ratio 的边。(有效 SBP 组合数除以总 SBP 组合数会作为挑选合并点的依据。 cut_ratio 越小，点融合以后策略空间就缩减得越多。)，其次为了防止多次合并后点的策略过大，也做出了相应的限制，如果组合代价小于 e^{38} 并且大于两个节点的策略的最小代价的两倍，就放弃该边的优先级（具体表现为 cut_ratio 设为 1.0）。所以 $cut_ratio < 1/3$ ，也就是每次合并策略空间至少缩减到 1/3 或者以下。而对于很多的边，比如绝大多数的 `variable_op` 到对应 `optimizer` 的边组合代价小于 e^{38} 并且等于两个节点的策略的最小代价，这意味着点融合后该点的策略数不会超过原节点策略数的最大值，这样的点融合缩减了策略空间以及拓扑结构并且维持了阈值对节点策略数的控制。

5.3 传输代理节点

当一个上游节点有复数个下游节点时，我们可以节省一些传输开销。比如节点 A 有两个下游节点 B 跟 C，A 的输出张量 y 会传给 B 跟 C。假如 A 的 SBP 是 S_0 ，B 和 C 的 SBP 都是 S_1 。此时按照代价模型的建立，两条边都会有通信代价，他们分别会进行一次 AlltoAll 的传输。然而只需要建立一个中间节点 5.14，A 的输出 y 先进行一次 AlltoAll 并传到这个中间节点，这时 y 在这个中间节点上的 SBP 就是 S_1 。从中间节点到两个下游节点 B、C，由于他们的 SBP 都是 S_1 ，所以没有通信产生，直接将 y 在节点内拷贝，这样会节省一次通信。这个中间节点被定义为 SBP proxy，也就是代理节点。



图 5.14 传输代理节点。

Figure 5.14 Transmission proxy nodes.

我们在代价模型里面抽象出了 SBP proxy 这一个概念。而这个 SBP proxy 在自动并行算法结束后不会产生任何实际的节点，它只是会帮助自动并行算法认识估算并利用这个优化。在代价模型里面，SBP proxy 建立以后，什么是 SBP proxy 的策略呢？传输的张量的 SBP 策略吗？如果是这样的话，那如果下游的所需该张量的 SBP 不一样呢？比如说上游节点 A 输出张量 y ，传给 4 个下游节点 B、C、D、E， y 在 A 是 S_0 ， y 在 B 和 C 是 S_1 ， y 在 D 和 E 是 S_2 。实际操作中 B 跟 C 会被统筹，D 跟 E 也会被统筹起来，这时候 SBP proxy 的策略就不能是 S_1 或者 S_2 。那该如何安排 SBP proxy 的策略以及从 SBP proxy 到下游节点的代价呢？

注意到上游节点到下游节点的代价在这个优化下跟下游节点某种 SBP 的个数无关，只与它存不存在有关。也就是上游 A ($y: S_0$) 到一个下游节点 B ($y: S_1$) 的代价跟上游节点 A ($y: S_0$) 到 100 个下游节点 ($y: S_1$) 的代价是一样的。所以，我们把 SBP proxy 的策略定义为下游可能策略的集合。例如，如果上游节点 A 传输张量 y 给两个下游节点 B、C，张量 y 在节点 B 的 SBP 有 B、 S_0 、 S_1 ，张量 y 在节点 C 的 SBP 有 B、 S_0 、 S_2 。那么上游节点 A 后面插入的 SBP proxy

的策略有 {B}, {S0}, {S1}, {S2}, {S0, S1}, {S0, S2}, {S1, S2}。

注意到策略里面没有 {S0, S1, S2}, 因为节点 A 只有 2 个下游节点, 我们限制集合的个数不能超过下游节点的个数, 否则只会无意义地增加策略空间。同时注意到策略里面 B 没有跟其他的 SBP 处于同一个集合, 这是由于 B 到其他 S0、S1、S2 的传输代价为 0, 当一个张量在 SBP proxy 的 SBP 策略为 B 时, 它到下游其他任何的 SBP 都不会产生通信, 最多只需要本地的处理, 所以只要有一个 B 策略, 在代价估计上就不需要 SBP。(实际中中间节点尽管有了 B 的 SBP, 也还是会产生其他的一些代理来保管其他 SBP, 如 S0 的张量来避免 B 到 S 时一些操作的重复, 但是这里理论上不会有通信代价的产生。) 根据 SBP proxy 的这种策略定义, SBP proxy 到下游节点的边的代价取值为: 如果下游节点的 SBP 被包含在 SBP proxy 的 SBP 内或者 SBP proxy 的策略为 B, 则代价为 0, 否则为无穷大 e^{38} 。

表 5.8 SBP proxy 策略到下游节点策略的代价。

Table 5.8 The cost from SBP proxy policy to downstream node policy.

SBP proxy 的策略 下游节点的策略	B	S0	S1	S2
{B}	0	0	0	0
{S0}	∞	0	∞	∞
{S1}	∞	∞	0	∞
{S2}	∞	∞	∞	0
{S0,S1}	∞	0	0	∞
{S0,S2}	∞	0	∞	0
{S1,S2}	∞	∞	0	0

因此总的来说, 通信在到 SBP proxy 的时候就已经做完了, SBP proxy 到下游节点只是取值的一个过程。

5.4 延迟时间及覆盖算法

在使用 nvprof/nsight system 对模型训练进行性能分析的时候, 发现了有一些传输量很小的上下游节点之间的时间间隙很大。

如图5.16中的 pthread_cond_wait 部分便是不同的卡在传输前的延迟等待。而

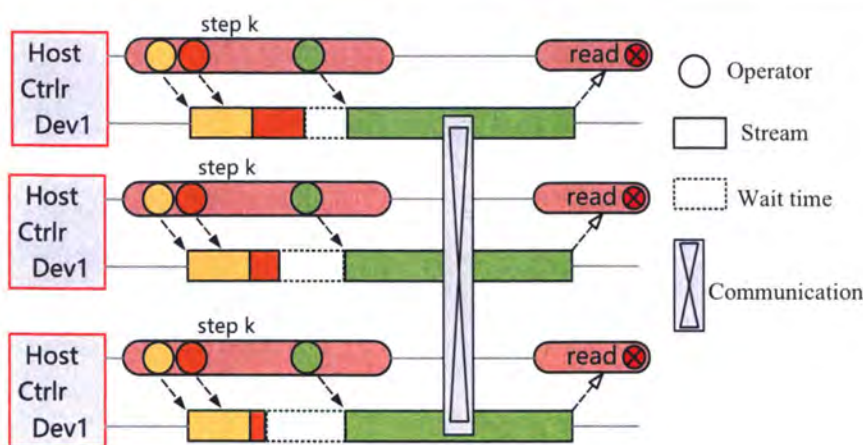


图 5.15 不同进程在通信之前出现了等待时间。

Figure 5.15 Waiting time occurs before process communication.

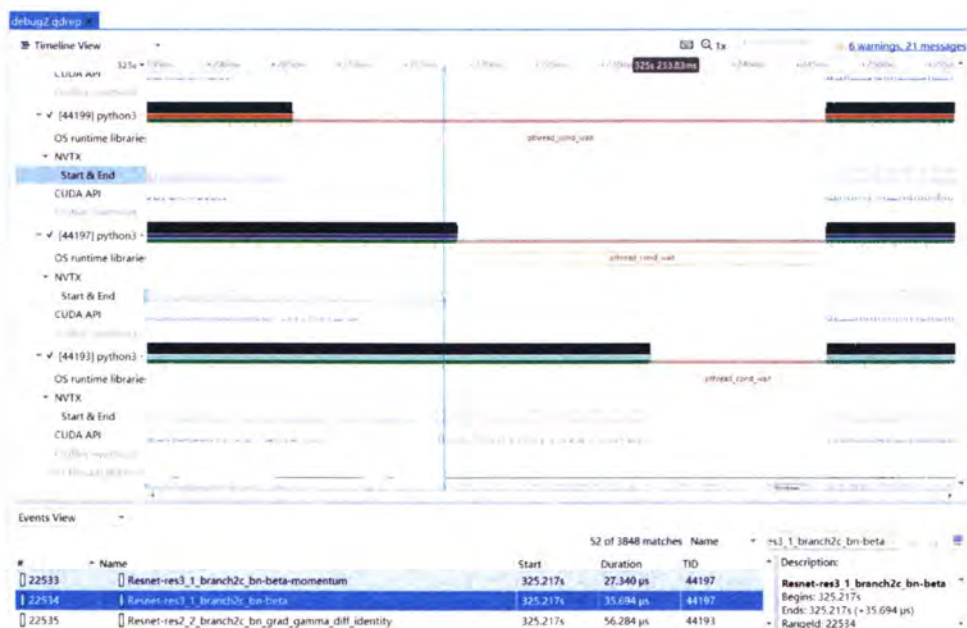


图 5.16 Nsight system 中的等待现象。

Figure 5.16 Waiting phenomenon in Nsight system.

需要传输的张量仅有 512×8 字节，与其他的传输量大小为 16M 字节的张量相比可以说是传输量非常小。为模拟不同卡在发生数据传输前的等待行为，我们为每条边的传输代价增加一个等待时间（wait time），只要上下游节点 SBP 不一致，我们便会增加等待时间。该等待时间不会随着传输量的多少来改变，所以等待时间可以设定为一个常数，它的估计可以根据多次试验来得到一个平均值。在等待

期间，可以运行其他待执行的节点（如果存在的话）。这就像是计算时间跟传输时间的互相重叠覆盖（overlap），只不过等待时间更加特殊，两个等待时间也可以相互覆盖，因为等待的时候实际没有任何消耗，也没有任何 Op 在执行，只是一直在等待。

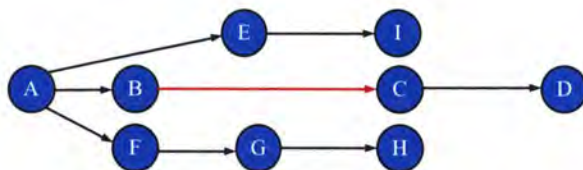


图 5.17 神经网络主干和支流之间相互 overlap。

Figure 5.17 Neural network backbone and tributaries overlap each other.

例如模型的这一段有两个分流，在 B 到 C 之间发生传输，在传输的等待期间，可以执行另外两个支流的节点 E、I、F、G、H。所以我们设计的延迟时间及覆盖算法通过定义执行层次的概念引入主干跟支流，主干跟支流之间的等待时间可以被对方的节点覆盖掉一部分。如果不存在支流，只存在主干，则覆盖不会发生，传输会引入非常大的等待时间。通过观察 ResNet-50 的性能测试，发现在无支流的主干上发生传输时，所有卡都在等待，中间既没有传输发生也没有任何的计算节点。ResNet-50 的前向传播就基本是纯主干，没有支流，所以比较适合前向无通信发生的数据并行。在有支流的情况下，不仅等待时间可以被覆盖掉，中间发生的传输的时间也可以被覆盖掉。因此给通信代价模型加上一个等待时间可以使得模型更准确。

5.5 通信代价

前人的工作都是使用通信量的传输时间作为代价函数的估计，假设加速器 i 的网络带宽为 b_i 。并且 $\mathbf{T}$ 是需要从一个加速器传输到另一个加速器的访问张量。到另一个，我们定义通信代价 $\text{cost } E_{\text{cm}}$ 为张量传输的通信代价为

$$E_{\text{cm}} = \frac{A(\mathbf{T})}{b_i} \quad (5.6)$$

张量 $\mathbf{T}$ 的大小被定义为所有维度的乘积，但是只用通信量作为通信的 cost 是不准确的，如图 5.22 和 5.19 所示，通过实测对一个张量大小为 $\text{size}(t)$ 的 t 做 allreduce 操作和对这个张量做 redcuescatter 与 allgather 操作的通信量是相等的，但是

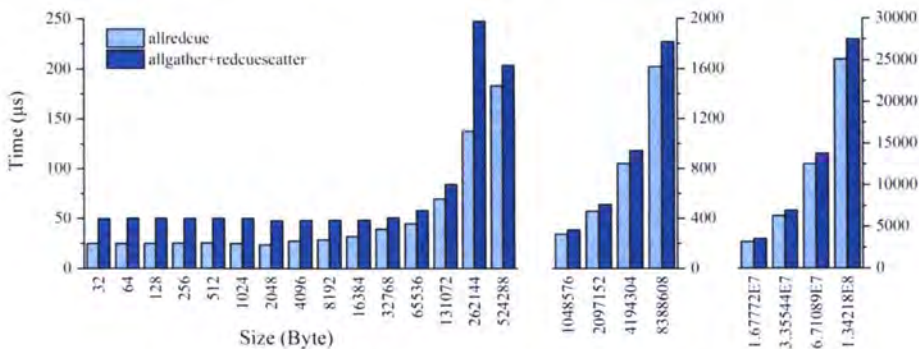


图 5.18 NCCL 通信库中的 Allreduce 和 Reducscatter 的通信特征。

Figure 5.18 Communication behavior of Allreduce and Reducscatter in the NCCL library.

二者的通信时间是有差异的，经过我们实测，在发送小包的情况下，reducescatter 和 allgather 的时间是大于 allreduce 的通信时间，这就会影响我们对张量的切分代价估计不准，例如卷积如果前向做切分本身会引入额外的数据重排，且前向通信无法被 overlap，如果只是根据通信量来描述通信代价有可能会把卷积层做切分。再比如，如表5.9所示，对于一个二维张量和二维设备拓扑，上游 Op 的 SBP 描述是 S^0S^1 ，而下游 Op 的 SBP 可能是 S^0B 或者 $S^0I B$ ，两种通信方式通信量的大小一样，但是 Allgather 和 Alltoall 的通信时间不一样。因此我们采用实测的方式。

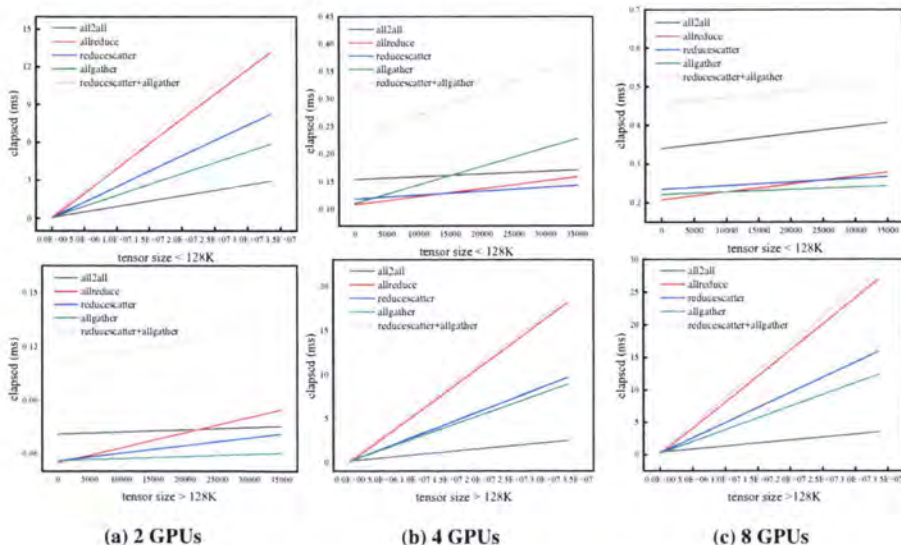


图 5.19 不同通信原语的通信代价。

Figure 5.19 Communication cost of different communication primitives.

表 5.9 上游 Op 到下游 Op 所用的通信和代价。

Table 5.9 Communication and cost from the upstream Op to the downstream Op.

#	Src Spec	Dst Spec	Communication Cost
1	BB	S^0S^1	0
2	S^0B	BB	all-gather ($M, 0$)
3	S^0S^1	S^0B	all-gather ($\frac{M}{n_0}, 1$)
4	S^0B	BS^0	all-to-all ($M, 0$)
5	S^0S^1	$S^{01}B$	all-to-all ($\frac{M}{n_0}, 1$)

表 5.10 通信代价的函数拟合。

Table 5.10 Function fitting of communication cost.

Nodes	Type	Fitting parameters($kx + b$)			
		tensor size < 128K		tensor size > 128K	
		k	b	k	b
2nodes	all2all	1.20E-07	0.071	8.49E-08	0.077
	allreduce	8.53E-07	0.0549	3.93E-07	-0.006
	scatter	4.25E-07	0.056	2.44E-07	0.051
	allgather	1.17E-07	0.056	1.72E-07	0.086
4nodes	all2all	4.95E-07	0.154	6.90E-08	0.176
	allreduce	1.43E-06	0.109	5.64E-07	0.025
	scatter	6.98E-07	0.119	2.85E-07	0.138
	allgather	3.35E-06	0.111	2.61E-07	0.163
8nodes	all2all	1.92E-06	0.34	9.25E-08	0.42
	allreduce	2.04E-06	0.208	7.99E-07	0.157
	scatter	9.29E-07	0.235	4.65E-07	0.304
	allgather	6.16E-07	0.222	3.62E-07	0.245

如表5.10所示，我们通过实测采样系统运行时的通信 Log，每种集合通信不同节点下，对一种通信原语采样多次，取平均值，然后做分段拟合，由于不同通信包的大小情况下，受延迟和带宽的影响不同，小包受延迟影响比较大，大包受通信带宽影响比较大，因此我们采用分段线性拟合的方式，将拟合后的参数制作为表格方便编译时检索查询。

5.6 API

我们在用户层即 Python 层也提供了简易的 API, 只需要一行代码就可以开启自动并行，并可以根据需求开启本章节里提到的一些优化策略参数。

```

1 import oneflow as of
2
3 class Trainer(object):
4     def __init__(self):
5         args = get_args()
6         for k, v in args.__dict__.items():
7             setattr(self, k, v)
8
9         self.rank = flow.env.get_rank()
10        self.world_size = flow.env.get_world_size()
11
12        self.cur_epoch = 0
13        self.cur_iter = 0
14        self.cur_batch = 0
15        self.is_consistent = (self.world_size > 1 and not self.ddp
16                               ) or self.graph
17        self.is_train = False
18        self.meter_lr = self.graph is False
19        ...
20        self.config.enable_auto_parallel(True)
21        self.config.set_auto_parallel_computation_cost_ratio(0.25)
22        self.config.set_auto_parallel_wait_time(1.65e7)
23        self.config.set_auto_parallel_transfer_cost(1.65e7)
24        self.config.enable_auto_parallel_mainstem_algo (True)
25        self.config.enable_auto_parallel_sbp_collector(True)

```

图 5.20 MAP API 的使用。

Figure 5.20 Usage of MAP API.

5.7 实验

5.7.1 实验配置

平台：我们的实验是在实验室两个节点上进行的，配置见表5.11。

表 5.11 平台配置。

Table 5.11 Platform configuration.

Number of Nodes (Machines)	2
Number of GPUs	16
GPU	Nvidia V100 (PCIe)x4 (32 GiB)
CPU	Intel(R) Xeon(R) E5-2650 v3 (32 GiBx8)
CPU-GPU Interconnect	PCI-Express Gen3x16 (16 GB/s)
GPU-GPU Interconnect	PCI-Express Gen3x16 (16 GB/s)

该实验在 Ubuntu 20.04 上运行。CUDA Toolkit 版本是 11.0，cuDNN 版本是 8.3.1。MAP 是基于 Oneflow 开发的，Oneflow 是一个开源 DL 框架 [90]。评估的重点是 MAP 与手动调优的策略相比较。

5.7.2 ResNet-50 性能测试

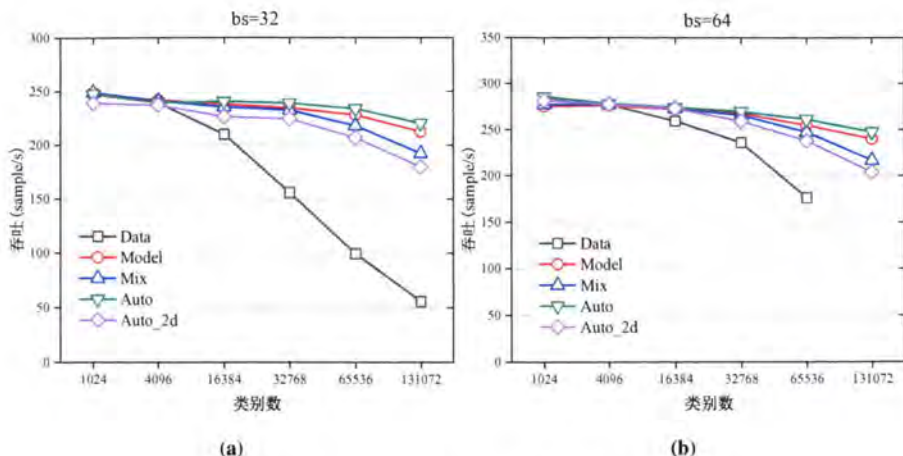


图 5.21 MAP 1D-SBP 在 ResNet-50 上的测试。

Figure 5.21 Testing of MAP 1D-SBP on ResNet-50.

如图5.21所示，我们分别采用不同的 batchsize 和类别数，训练 1000 次迭代，吞吐单位是每秒训练的样本数，去掉吞吐最慢和最慢的 10%，对于相同的类别数，我们分别测试了不同的手动切分方法包括，数据并行（按数据维度切分），模型并行（模型参数按列切分），混合并行（模型参数按行切分），以及自动切分方法 1-D 自动并行，2-D 自动并行。由于 ResNet-50 只有一层全连接，所以我们通过增加类别数来增大最后一层计算量即矩阵大小，从图中可以看出，正常情况下，从 1024 分类到 4096 分类 ResNet-50 更倾向于数据并行，而我们观察计算图编译时生成的切分 plan 也能得出在前向计算图中，卷积层和全连接都是采用的数据并行，即数据张量的 SBP 是 $S(0)$ 切分，weight 参数的 SBP 是 B 。而随着分类数的增加最后一层的矩阵也越来越大，此模型并行逐渐表现出了较优的性能，而我们通过观察编译生成的 plan 也得出此时自动并行前向计算给出的策略和手动切分的模型并行的策略是一致的都是 weight 按照 $S(1)$ 切分。而随着分类数的继续增大，此时基于手动切分模型的速度就差于自动并行，原因在于手动切分只能控制前向计算的切分，无法控制反向的计算图的切分策略，自动并行在反向计算图中同样可以做切分策略的优化。我们观察到手动切分时正向计算图通过自动求导模块生成的反向计算图和自动并行生成的反向并行策略图不一致，因此得出结论自动并行在反向计算时同样能带来性能上的提升。而 2D 自动并行前期也

给出了等价于数据并行的策略，但是随着分类数增加时，2D 自动并行给出的策略带来了较多的通信开销因此在只有单个全连接层的网络上性能上的表现不如 1-D 自动并行。

5.7.3 并行策略-VGG16

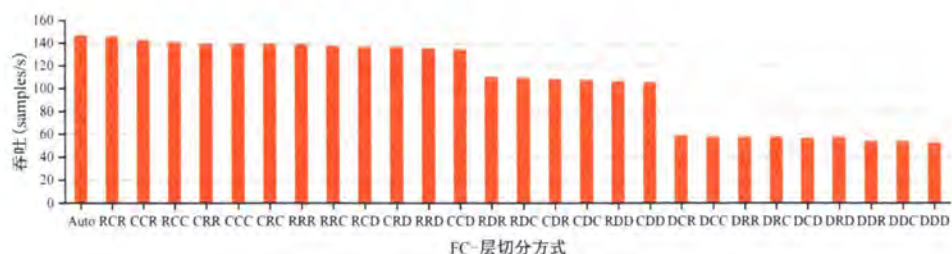


图 5.22 MAP 1D-SBP 在 VGG-16 上的测试。

Figure 5.22 MAP 1D-SBP tested on VGG-16.

如图5.22所示，我们在 VGG-16 上做了策略的测试，VGG 有三个全连接层，每一层的切分方式有三种，按行切分、按列切分、广播，分别对应的 SBP 是：S(0)，S(1)，B。则所有层的策略组合就有 27 种，如图所示我们分别手动枚举了 27 种策略，从枚举出的策略看出当三个全连接层分别按行切分 R，按列切分 C，按行切分 R 时整体的吞吐最高，而自动并行的性能要略强于最优的手动策略。

5.7.4 InsightFace 评估

如图5.23我们在超大规模人脸识别模型 InsightFace 上做了测试，在单机 4GUPs 上，在 batch size 为 16 时最大吞吐是手调模型并行的 1.2 $\times$ 。在其他 batchsize 下基本也和手动调优的模型并行性能持平。如图5.24所示，自动并行给出的方案是全连接层采用模型并行，即对权重矩阵做切分而使用全量的特征数据，因此输入特征的 SBP 属性为 Broadcast，即表示每个 GPU 设备上都会拷贝一份特征数据；而参数的 SBP 属性为 S(1)，即参数矩阵在维度 1 被切分到各个 GPU 设备上，假设有 P 个 GPU，则每个 GPU 上有 (emb_size, 10000000/P) 大小的参数。全连接层的输出形状取决于输入 features 的形状以及类别 ID 数，故每个设备上的输出形状为 (batch_size, 10000000/P)，且 SBP 属性也为 S(1)。由于本方案对参数做了切割 (S(1))，故通常来说，需要对全连接层的输出做合并，并转为按 S(0) 切分的数据并行，但是由于全连接层的输出数据块较大，如果直接由 S(1) 转为常规的

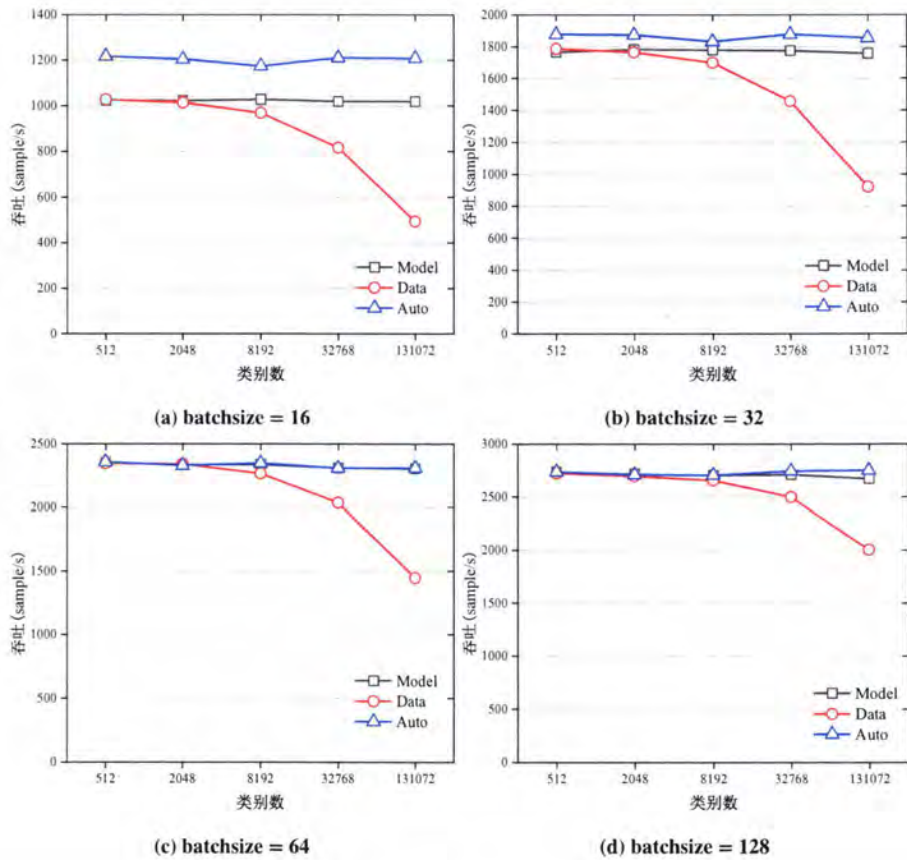


图 5.23 MAP 1D-SBP 在 InsightFace 上的性能测试。

Figure 5.23 Performance test of MAP 1D-SBP on InsightFace.

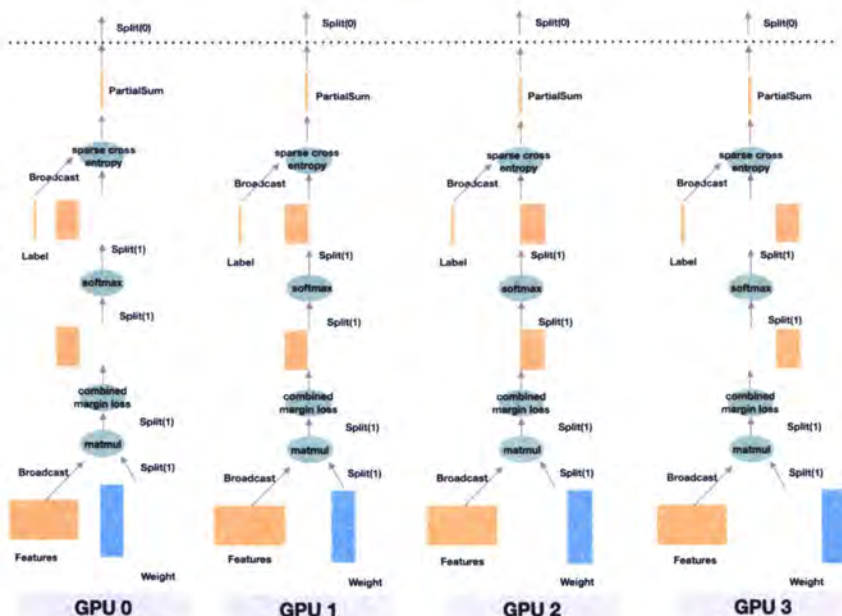


图 5.24 InsightFace 在 MAP 下生成的策略。

Figure 5.24 Parallelism plan generated on InsightFace by MAP.

S(0), 会引入大量通信。因此, 在自动并行策略中, 并不先执行行和列的转化, 而是将全连接层的输出直接作为 *softmax* 的输入进行计算, 因为 *softmax* 的运算特性, 可以使得输出的 SBP 属性依然是 S(1)。类似的, *softmax* 的输出 (按 S(1) 切分) 继续作为 *sparse_cross_entropy* 的输入进行计算, 由于算子本身的特性和框架的机制, *sparse_cross_entropy* 的输出的 SBP 属性依然可以保持 S(1)。经过 *sparse_cross_entropy* 处理后的输出, 逻辑上获得最终的 *loss* 结果。此时, 数据块的形状为 (batch_size, 1), 此时数据已经很小, 这时候再将模型并行的 S(1) 模式转为按 S(0) 切分的数据并行可以减少通信开销。

5.7.5 2D SBP 测试

我们测试了 2D SBP 的性能, 如图 5.25 所示, 我们选用了 2 种网络, 分别是 Bert-large 和 GPT2, 两种网络都由 transformer block 拼接而成, 大部分都是矩阵乘计算, 我们分别使用了手动调优的策略, 通过手动指定控制一部分网络层的 SBP 而剩余部分的网络层采用搜索的方式, 以及全自动搜索。从图中可以看出, 自动并行在 2D SBP 上表现不如 1-D 有效, 这是因为目前 2D SBP 的搜索空间较大, 无法快速得搜到一些较优的 SBP 策略, 甚至由于搜索空间爆炸, 在 Bert 的

hidden layer 的个数到 348 的时候就无法搜出策略。而通过手调控制部分 block 层的 SBP 后搜到的策略是可以优于原始的手调策略。

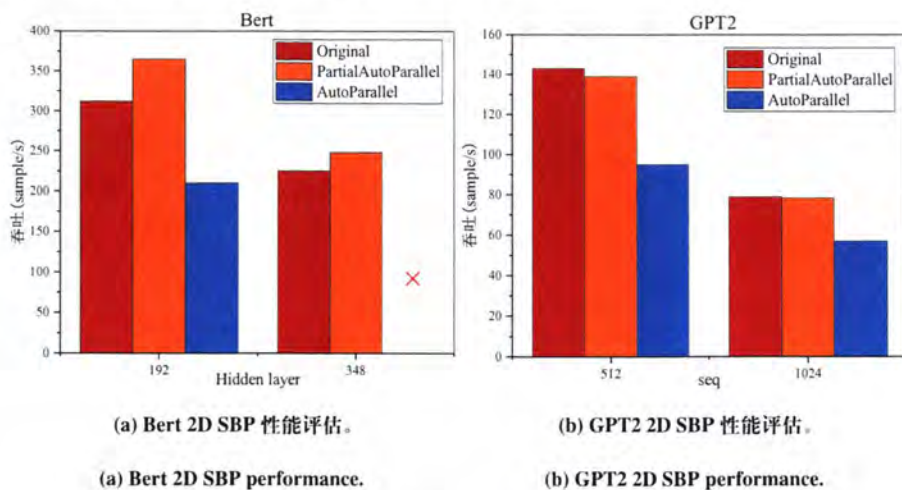


图 5.25 MAP 在 2D SBP 模式下 Bert 和 GPT2 上的性能表现。

Figure 5.25 Performance of MAP on Bert and GPT2 in 2D SBP mode.

5.7.6 对比 Megatron 和 DeepSpeed

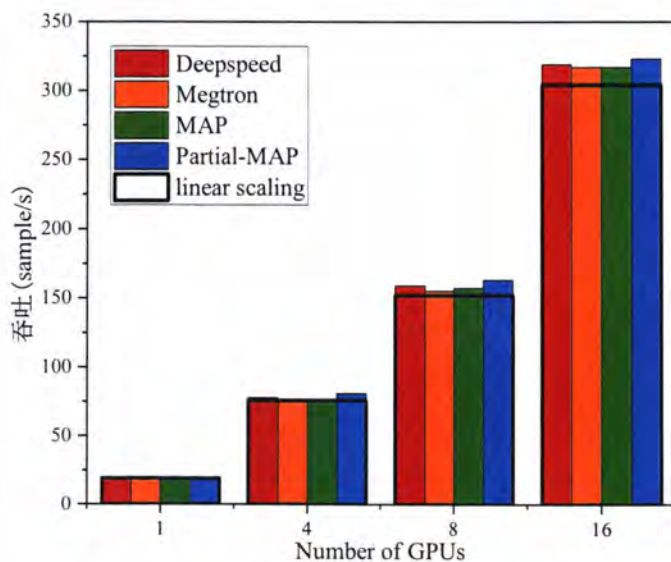


图 5.26 MAP 与 DeepSpeed 以及 Megatron 的对比。

Figure 5.26 MAP compare with DeepSpeed and Megatron.

我们使用 Megatron-LM v2 作为 GPT-2 的基线系统。Megatron-LM 是最先进

的系统用于在 GPU 上训练基于 transformer 的语言模型。它结合了数据并行, 流水线并行, 手动设置的算子内并行。这些技术的组合由三个整数参数控制, 这些参数规定了分配给每种技术的并行度。我们未开启 Pipeline 并行。DeepSpeed 也为在 GPU 上训练 GPT-2 提供了最先进的实现。它结合了 Transformer 层的手工模型并行调优和基于 ZeRo 的数据并行调优。这些技术的组合由几个整数参数控制, 这些参数指定分配给每种技术的并行度。GPT-2 的并行化策略已经广泛研究。我们在图5.26中观察到, 这个具有最佳网格搜索参数的手动策略使 Megatron-LM 和 DeepSpeed 能够在 GPT-2 上实现超线性弱扩展。尽管如此, 与 Megatron-LM 相比, MAP 自动生成执行策略甚至在几个设置上实现了稍微更好的扩展。在只与模型并行的方法相比下, 我们的结果与最近的研究基本一致。

5.8 相关工作

数据并行训练系统, Horovod[91] 和 PyTorch DDP[88] 是两种常用的数据并行使用 Allreduce 同步梯度的训练系统。BytePS[92] 统一了 All-reduce 和参数服务器, 利用数据中心集群中的异构资源。AutoDist[93] 使用基于学习的方法来组合数据并行训练策略。ZeRO[34] 通过减少复制张量改进了数据并行的内存使用。在 MAP 中, 数据并行简化为沿批处理维度切分的算子内并行。

模型并行训练系统, 两大主要的模型的并行已经被之前章节讨论。Mesh-TensorFlow[74] 和 GSPMD [40] 是两个系统为算子内并行设计的。他们依靠用户注释来获取切分策略。Tofu [89] 开发了一种 DP 算法为单节点上的线性图生成最优切分策略。PipeDream[9] 通过使用异步改进 GPipe 训练算法、减少内存使用和将它和数据并行集成, 但它是异步的, 而 MAP 是同步训练系统。TeraPipe[10] 为基于 Transformer 的 LM 模型发现了一个新的流水线并行策略。尽管有 Megatron-LM[28], 但离我们最近的系统是 FlexFlow [94]。FlexFlow 提出“SOAP”并行策略形式化并开发了基于 MCMC 的随机搜索算法。然而它的搜索算法无法扩展到大型图或集群并且没有最优性保证。

用于深度学习的编译器, 编译器技术有被引入以优化 DL 模型的推理执行 [95, 96]。他们中的大多数人专注于优化单个设备的计算。相比之下, MAP 是支持全执行空间的图编译分布式训练。

5.9 小结

我们介绍了 MAP，一种用于分布式训练自动化模型并行的计算图 PASS 模块，是建立在一个新的数据和计算视图之上的深度学习并行化生成模块（算子内的并行）。MAP 构建了一个层次化的空间和使用一个搜索过程来推导最优并行策略，为分布式深度学习模型并行制定有效的并行化策略是一项耗费人工资源的任务，我们相信 MAP 将使分布式模型并行学习简单高效化并加速新兴大型深度学习模型的使用。

第6章 总结与展望

随着深度学习技术的快速发展,深度学习不仅在计算机视觉,语音识别中得到了广泛应用,而且还为传统的科学计算应用带来了创新,例如计算化学,计算物理和医学成像。而这些应用程序也产生越来越多的训练数据,现在已经达到PB级别。另一方面,神经网络模型也变得越来越,例如深度学习早期的相对较大的模型VGG-16网络的参数为1.35亿个参数,而Switch Transformer是Google提出的最新语言模型包含1.6万亿多个参数,现在已经达到TB级别。数据和模型的增大使得分布式训练成为必用的解决方案,训练平台规模甚至达到E级别。这些应用负载使用的更大数据集,更大模型和更大的集群给训练过程带来了挑战。大数据集在并行分布式训练上带来了大batchsize训练的挑战,即SPMD并行模式下节点数越多batchsize越大,训练出的模型精度越低。大模型带来的问题则是显存受限,尤其是在动态计算图上的显存优化带来了新的挑战。训练集群的增大带来的是并行策略的增加,高效的大规模模型训练需要调整神经网络中数据并行、算子并行和流水线并行方法的粒度。正确地调整并行策略已被证明可以在训练性能上有一个数量级的改进,但取决于强大的ML和系统专业知识。

6.1 论文总结

针对这三个问题,我们提出了三个优化模块,并在深度学习框架中设计了三个优化模块,这三个模块分别是:用于数据并行下用于缓解大batch size造成模型精度丢失问题的可变batch size训练优化器-VBS优化器;用于基于动态计算图深度学习框架下的显存优化模块-MegTaiChi内存管理模块;用于生成较优并行策略的自动并行策略模块-MAP自动并行模块。论文创新工作主要体现在以下四个方面:

1. **VBS优化器**用于数据并行下大batch size训练时避免模型精度丢失,该优化器包括分阶段可变batch size优化策略和超参自动调优器两个部分。首先,我们分析了SGD不同更新阶段对训练过程的影响,发现早期阶段使用的批大小几乎完全决定了神经网络的最终模型精度。基于这一发现,我们提出了一种用于分布式CNN训练的可变batch size训练策略。接下来,为了进一步提高batch size,

基于 BatchNorm 层的特点，我们推导出有效学习率，并证明有效学习率的上界和下界在大批量训练中都很重要。这一观察导致设计一个超参自动调优引擎。这个自动调整引擎的效果显著，因为它可以通过我们提出的理论指导自动调整现有的超参数。最后，通过使用带有自动调优引擎的可变 batch size 策略，在不丢失模型精度的情况下，我们能够在 ResNet-50 训练中首次成功地将 batch size 扩展到 120K。在只有轻微模型精度损失的情况下，将 batch size 扩展到 128K。

2. MegTaiChi 内存管理模块是一个用于 DNN 训练的基于动态张量的内存管理优化模块，它首先实现了张量划分和张量重新实现的高效协调。MegTaiChi 的关键特性是它根据运行时跟踪的动态张量访问模式做出内存管理决策。这种设计的动机是观察到在训练迭代期间对张量的访问模式是有规律的。基于识别出的模式，MegTaiChi 利用总内存优化空间，实现启发式、自适应和细粒度的内存管理。实验结果表明，与 DTR 相比，MegTaiChi 可以将 ResNet-50 的内存占用减少高达 11%，GL-base 减少 10.5%。对于 6 个具有代表性的 DNN 的训练，MegTaiChi 比 MegEngine 和 Sublinear 平均分别比最大批量大小分别高出 5× 和 2.4×。对于百万规模的人脸识别应用，与 256 个 GPU 上的最优经验并行策略相比，MegTaiChi 实现了 1.8× 的加速。

3. MAP 自动并行模块是一个用于自动生成并行策略的模块，MAP 提供一个全局视角的概念，用于简化分布式训练。简单而言，在 MAP 的全局视角下，机群被抽象为一台“超级计算设备”。并描述了“超级计算设备”全局视角下的数据与机群中真实的物理设备上的数据的映射关系。我们采用和传统的数据和模型并行性不同的观点，并将算子的并行方法重新表达为一种符号抽象，指定逻辑数据张量和相应物理张量之间的映射。通过对数据，计算设备和算子的抽象，我们对整个神经网络构造一个并行策略搜索空间，通过构建整个神经网络对应的计算图的性能模型，并利用贪心搜索算法，搜索出每个算子的并行策略，使得搜索出的整个神经网络模型的并行策略能够接近甚至超越手动设置的策略。

6.2 未来工作

大规模并行深度学习训练优化是当前以及未来人工智能应用在 HPC 负载中重点挑战之一，因此将在未来始终是学术界和工业界热点研究对象。本文系统性地分析了大规模并行深度学习训练技术的三个挑战。在未来，我们将继续推进以

下研究：

- 大 batchsize 训练：目前大 batch size 训练问题仍然存在比较大的争议，优化问题还是泛化问题仍然还没有一个定论，最近两三年的工作并未取得实质性的进展，目前来看从优化的角度去尽量避免泛化差是一个可行的路线，但是仍然停留在大量的调超参的层面，而大 batch size 训练丢失精度的本质并未得到很好的解释，调参方法也缺乏一定的理论指导，有效学习率是一个可以继续探索的一个方向，最近也有相关的工作，从球面动力学的角度将训练映射到一个等效的球面运动，将所有影响训练的超参都统一到一个角度更新，只通过调整角度参数，训练出的模型效果可以等效原始训练效果。因此如何寻找较优的有效学习率是一个可以继续探索的方向。

- 动态图上的显存优化问题：纯动态图上的显存优化问题由于无法获得全局计算图信息，无法像静态图提前做优化，只能在运行时启发式的去选择 checkpoint 点，所以运行时的开销是个较大的问题，而动态图上的显存优化最大的问题是会带来显存碎片，本文涉及纯动态图计算的部分虽然改进了打分函数用于减缓显存碎片的产生但是仍然无法避免显存碎片。而我们结合静态图的特征使用了静态内存分配策略实际是将动态图下的静态神经网络转化为静态执行 plan，是一种动静结合的方法，所以无法像纯静态图下支持任意网络，是一个折中的方法，这是因为大部分网络都是静态的。所以在未来的工作中，我们会重点解决纯动态下如何尽可能避免显存碎片。以及在运行时如何尽量减少打分的开销。多机下的动态张量切分则是如何更系统性的给张量切分建模。提出更优的切分策略模型。

- 自动并行：在自动并行模块中，目前本文只考虑了层内并行，即切分算子，未考虑 pipeline 并行。对于加入 pipeline 并行，节点间采用 pipeline 并行节点内使用算子并行是一个可行的方案，因为节点间的带宽小于节点内的通信带宽，适合层间并行，因为层间并行的通信是 send/receive，而节点内的带宽高可以采用复杂的算子并行。其次是如何结合集群的特征让性能代价模型更准确。最后目前的自动并行都是基于静态图去做，而对于动态图上的自动并行如上一个研究点所说则更具有挑战，是我们未来的研究目标。

参考文献

- [1] Kirisame M, Lyubomirsky S, Haan A, et al. Dynamic Tensor Rematerialization [Z]. 2021.
- [2] Chen T, Xu B, Zhang C, et al. Training Deep Nets with Sublinear Memory Cost [J/OL]. CoRR, 2016, abs/1604.06174. <http://arxiv.org/abs/1604.06174>.
- [3] Gholami A. AI 内存和算力墙 [EB/OL]. 2021[March 9, 2021]. <https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>.
- [4] Ben-Nun T, Hoefler T. Demystifying Parallel and Distributed Deep Learning: An In-depth Concurrency Analysis [J]. ACM Computing Surveys (CSUR), 2019, 52(4): 1-43.
- [5] 迟学斌, 赵毅. 高性能计算技术及其应用 [J]. 中国科学院院刊, 2007, 22(4): 306-313.
- [6] 张军华, 臧胜涛, 单联瑜, 等. 高性能计算的发展现状及趋势 [J]. 石油地球物理勘探, 2010 (6): 918-925.
- [7] 夏培肃, 胡伟武. 高性能计算技术展望 [J]. 中国科学院院刊, 1998, 5.
- [8] HERNANDEZ D A. AI 算力需求 [EB/OL]. 2018[March 9, 2021]. <https://cloud.tencent.com/developer/article/1145733>.
- [9] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training [C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 1-15.
- [10] Li Z, Zhuang S, Guo S, et al. Terapipe: Token-level Pipeline Parallelism for Training Large-scale Language Models [C]//International Conference on Machine Learning. PMLR, 2021: 6543-6552.
- [11] Wang M, Huang C c, Li J. Supporting Very Large Models Using Automatic Dataflow Graph Partitioning [C]//Proceedings of the Fourteenth EuroSys Conference 2019. 2019: 1-17.
- [12] Goyal P, Dollár P, Girshick R, et al. Accurate, Large Minibatch SGD: Training Imagenet in 1 Hour [J]. arXiv preprint arXiv:1706.02677, 2017.
- [13] Jin P H, Yuan Q, Iandola F, et al. How to Scale Distributed Deep Learning? [J]. arXiv preprint arXiv: Arxiv-1611.04581, 2016.
- [14] Mitliagkas I, Zhang C, Hadjis S, et al. Asynchrony Begets Momentum, with An Application to Deep Learning [J]. arXiv preprint arXiv: Arxiv-1605.09774, 2016.
- [15] Dean J, Corrado G S, Monga R, et al. Large Scale Distributed Deep Networks [J]. 2012.
- [16] Seide F, Fu H, Droppo J, et al. On Parallelizability of Stochastic Gradient Descent for Speech DNNs [C]//2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2014: 235-239.

- [17] Keskar N S, Mudigere D, Nocedal J, et al. On Large-batch Training for Deep Learning: Generalization Gap and Sharp Minima [J]. arXiv preprint arXiv:1609.04836, 2016.
- [18] You Y, Gitman I, Ginsburg B. Large Batch Training of Convolutional Networks [J]. arXiv preprint arXiv:1708.03888, 2017.
- [19] You Y, Li J, Reddi S, et al. Large Batch Optimization for Deep Learning: Training Bert in 76 Minutes [J]. arXiv preprint arXiv:1904.00962, 2019.
- [20] Martens J, Grosse R. Optimizing Neural Networks with Kronecker-Factored Approximate Curvature [C]//International conference on machine learning. PMLR, 2015: 2408-2417.
- [21] Yao Z, Gholami A, Lei Q, et al. Hessian-based Analysis of Large Batch Training and Robustness to Adversaries [J]. arXiv preprint arXiv:1802.08241, 2018.
- [22] Bottou L, Curtis F E, Nocedal J. Optimization Methods for Large-scale Machine Learning [J]. Siam Review, 2018, 60(2): 223-311.
- [23] Chen T, Li M, Li Y, et al. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems [J]. 2015.
- [24] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-efficient Neural Network Design [C/OL]//2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2016: 1-13. DOI: 10.1109/MICRO.2016.7783721.
- [25] Wang L, Ye J, Zhao Y, et al. Superneurons: Dynamic GPU Memory Management for Training Deep Neural Networks [C/OL]//PPoPP '18: Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. New York, NY, USA: Association for Computing Machinery, 2018: 41-53. <https://doi.org/10.1145/3178487.3178491>.
- [26] Peng X, Shi X, Dai H, et al. Capuchin: Tensor-Based GPU Memory Management for Deep Learning [C/OL]//ASPLOS '20: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. New York, NY, USA: Association for Computing Machinery, 2020: 891-905. <https://doi.org/10.1145/3373376.3378505>.
- [27] Ito Y, Imai H, Duc T L, et al. Profiling Based Out-of-core Hybrid Method for Large Neural Networks [J/OL]. CoRR, 2019, abs/1907.05013. <http://arxiv.org/abs/1907.05013>.
- [28] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [Z]. 2020.
- [29] Huang C C, Jin G, Li J. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping [C/OL]//ASPLOS '20: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems.

- New York, NY, USA: Association for Computing Machinery, 2020: 1341–1355. <https://doi.org/10.1145/3373376.3378530>.
- [30] Shazeer N, Stern M. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost [C/OL]//Dy J, Krause A. Proceedings of Machine Learning Research: volume 80 Proceedings of the 35th International Conference on Machine Learning. PMLR, 2018: 4596-4604. <http://proceedings.mlr.press/v80/shazeer18a.html>.
 - [31] Jain P, Jain A, Nrusimha A, et al. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization [C/OL]//Dhillon I, Papailiopoulos D, Sze V. Proceedings of Machine Learning and Systems: volume 2. 2020: 497-511. <https://proceedings.mlsys.org/paper/2020/file/084b6fbb10729ed4da8c3d3f5a3ae7c9-Paper.pdf>.
 - [32] Huang Y, Cheng Y, Chen D, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism [J/OL]. CoRR, 2018, abs/1811.06965. <http://arxiv.org/abs/1811.06965>.
 - [33] Jia Z, Zaharia M, Aiken A. Beyond Data and Model Parallelism for Deep Neural Networks. [C/OL]//Talwalkar A, Smith V, Zaharia M. Proceedings of Machine Learning and Systems: volume 1. 2019: 1-13. <https://proceedings.mlsys.org/paper/2019/file/c74d97b01eae257e44aa9d5bade97baf-Paper.pdf>.
 - [34] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimization Towards Training A Trillion Parameter Models [J/OL]. CoRR, 2019, abs/1910.02054. <http://arxiv.org/abs/1910.02054>.
 - [35] Lepikhin D, Lee H, Xu Y, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding [Z]. 2020.
 - [36] Wahib M, Zhang H, Nguyen T T, et al. Scaling Distributed Deep Learning Workloads beyond the Memory Capacity with KARMA [C]//SC '20: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. Atlanta, Georgia: IEEE Press, 2020.
 - [37] Krizhevsky A. One Weird Trick for Parallelizing Convolutional Neural Networks [J/OL]. CoRR, 2014, abs/1404.5997. <http://arxiv.org/abs/1404.5997>.
 - [38] Mirhoseini A, Goldie A, Pham H, et al. A Hierarchical Model for Device Placement [C]// International Conference on Learning Representations. 2018.
 - [39] Jia Z, Lin S, Qi C R, et al. Exploring Hidden Dimensions in Parallelizing Convolutional Neural Networks [J/OL]. CoRR, 2018, abs/1802.04924. <http://arxiv.org/abs/1802.04924>.
 - [40] Xu Y, Lee H, Chen D, et al. GSPMD: General and Scalable Parallelization for ML Computation Graphs [J]. arXiv preprint arXiv:2105.04663, 2021.
 - [41] Jia X, Jiang L, Wang A, et al. Whale: Scaling Deep Learning Model Training to the Trillions [Z]. 2021.

- [42] Mirhoseini A, Pham H, Le Q V, et al. Device Placement Optimization with Reinforcement Learning [C]//ICML'17: Proceedings of the 34th International Conference on Machine Learning - Volume 70. Sydney, NSW, Australia: JMLR.org, 2017: 2430–2439.
- [43] Zhang X, Zhou X, Lin M, et al. Shufflenet: An Extremely Efficient Convolutional Neural Network for Mobile Devices [C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 6848-6856.
- [44] Wan R, Zhu Z, Zhang X, et al. Spherical Motion Dynamics of Deep Neural Networks with Batch Normalization and Weight Decay [J]. arXiv preprint arXiv:2006.08419, 2020.
- [45] Robbins H, Monro S. A Stochastic Approximation Method [J]. The annals of mathematical statistics, 1951: 400-407.
- [46] Qian N. On the Momentum Term in Gradient Descent Learning Algorithms [J]. Neural networks, 1999, 12(1): 145-151.
- [47] Ioffe S, Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift [C]//International conference on machine learning. PMLR, 2015: 448-456.
- [48] Deng J, Dong W, Socher R, et al. ImageNet: A Large-Scale Hierarchical Image Database [C]//2009 IEEE conference on computer vision and pattern recognition. Ieee, 2009: 248-255.
- [49] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition [J/OL]. CoRR, 2015, abs/1512.03385. <http://arxiv.org/abs/1512.03385>.
- [50] MLCommons. mlperf benchmark [EB/OL]. 2022. <https://mlcommons.org/en/>.
- [51] You Y, Wang Y, Zhang H, et al. The Limit of the Batch Size [J]. arXiv preprint arXiv: Arxiv-2006.08517, 2020.
- [52] Pauloski J G, Zhang Z, Huang L, et al. Convolutional Neural Network Training with Distributed K-FAC [C]//SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2020: 1-12.
- [53] Haruki K, Suzuki T, Hamakawa Y, et al. Gradient Noise Convolution (GNC): Smoothing Loss Function for Distributed Large-Batch SGD [J]. arXiv preprint arXiv:1906.10822, 2019.
- [54] Hoffer E, Hubara I, Soudry D. Train Longer, Generalize Better: Closing the Generalization Gap in Large Batch Training of Neural Networks [C]//NIPS. 2017.
- [55] Wen Y, Luk K, Gazeau M, et al. Interplay Between Optimization and Generalization of Stochastic Gradient Descent with Covariance Noise [J]. arXiv preprint arXiv:1902.08234, 2019.
- [56] Lin T, Kong L, Stich S, et al. Extrapolation for Large-batch Training in Deep Learning [C]//International Conference on Machine Learning. PMLR, 2020: 6094-6104.

- [57] Jastrzębski S, Kenton Z, Ballas N, et al. On the Relation Between the Sharpest Directions of DNN Loss and the SGD Step Length [J]. arXiv preprint arXiv:1807.05031, 2018.
- [58] Osawa K, Tsuji Y, Ueno Y, et al. Large-Scale Distributed Second-Order Optimization Using Kronecker-Factored Approximate Curvature for Deep Convolutional Neural Networks [C]// Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 12359-12367.
- [59] Ma L, Montague G, Ye J, et al. Inefficiency of K-FAC for Large Batch Size Training [C]// Proceedings of the AAAI Conference on Artificial Intelligence: volume 34. 2020: 5053-5060.
- [60] Li M, Zhang T, Chen Y, et al. Efficient Mini-batch Training for Stochastic Optimization [C]// Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 661-670.
- [61] Li Q, Tai C, Weinan E. Stochastic Modified Equations and Adaptive Stochastic Gradient Algorithms [C]//International Conference on Machine Learning. PMLR, 2017: 2101-2110.
- [62] Smith S L, Kindermans P J, Ying C, et al. Don't Decay the Learning Rate, Increase the Batch Size [C]//International Conference on Learning Representations. 2018.
- [63] Devarakonda A, Naumov M, Garland M. AdaBatch: Adaptive Batch Sizes for Training Deep Neural Networks [J]. arXiv preprint arXiv: Arxiv-1712.02029, 2017.
- [64] McCandlish S, Kaplan J, Amodei D, et al. An Empirical Model of Large-Batch Training [J]. arXiv preprint arXiv: Arxiv-1812.06162, 2018.
- [65] Lin H, Zhang H, Ma Y, et al. Dynamic Mini-batch SGD for Elastic Distributed Training: Learning in the Limbo of Resources [J]. arXiv preprint arXiv:1904.12043, 2019.
- [66] Huo Z, Gu B, Huang H. Large Batch Optimization for Deep Learning Using New Complete Layer-Wise Adaptive Rate Scaling [C]//Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI. 2021: 2-9.
- [67] Akiba T, Suzuki S, Fukuda K. Extremely Large Minibatch SGD: Training ResNet-50 on ImageNet in 15 Minutes [J]. arXiv preprint arXiv:1711.04325, 2017.
- [68] Jia X, Song S, He W, et al. Highly Scalable Deep Learning Training System with Mixed-Precision: Training ImageNet in Four Minutes [J]. arXiv preprint arXiv:1807.11205, 2018.
- [69] Sun P, Wen Y, Han R, et al. GradientFlow: Optimizing Network Performance for Large-Scale Distributed DNN Training [J]. IEEE Transactions on Big Data, 2019.
- [70] Osawa K, Tsuji Y, Ueno Y, et al. Scalable and Practical Natural Gradient for Large-Scale Deep Learning [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.
- [71] Dryden N, Maruyama N, Moon T, et al. Channel and Filter Parallelism for Large-Scale CNN Training [C/OL]//SC '19: Proceedings of the International Conference for High Performance

- Computing, Networking, Storage and Analysis. New York, NY, USA: Association for Computing Machinery, 2019. <https://doi.org/10.1145/3295500.3356207>.
- [72] Shoenybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [J/OL]. CoRR, 2019, abs/1909.08053. <http://arxiv.org/abs/1909.08053>.
 - [73] Gholami A, Azad A, Keutzer K, et al. Integrated Model and Data Parallelism in Training Neural Networks [J/OL]. CoRR, 2017, abs/1712.04432. <http://arxiv.org/abs/1712.04432>.
 - [74] Shazeer N, Cheng Y, Parmar N, et al. Mesh-TensorFlow: Deep Learning for Supercomputers [C/OL]//Bengio S, Wallach H, Larochelle H, et al. Advances in Neural Information Processing Systems: volume 31. Curran Associates, Inc., 2018. <https://proceedings.neurips.cc/paper/2018/file/3a37abdeef1dab1b30f7c5c7e581b93-Paper.pdf>.
 - [75] Basu D, Data D, Karakus C, et al. Qsparse-local-SGD: Distributed SGD with Quantization, Sparsification, and Local Computations [J]. arXiv preprint arXiv:1906.02367, 2019.
 - [76] Shi S, Chu X, Cheung K C, et al. Understanding Top-k Sparsification in Distributed Deep Learning [J]. arXiv preprint arXiv:1911.08772, 2019.
 - [77] Rasley J, Rajbhandari S, Ruwase O, et al. Deepspeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters [C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020: 3505-3506.
 - [78] Wahib M, Zhang H, Nguyen T T, et al. Scaling Distributed Deep Learning Workloads beyond the Memory Capacity with KARMA [Z]. 2020.
 - [79] Wang M, Huang C c, Li J. Supporting Very Large Models using Automatic Dataflow Graph Partitioning [J/OL]. Proceedings of the Fourteenth EuroSys Conference 2019, 2019. <http://dx.doi.org/10.1145/3302424.3303953>.
 - [80] Greenberg A, Hamilton J R, Jain N, et al. VL2: A Scalable and Flexible Data Center Network [C]//Proceedings of the ACM SIGCOMM 2009 conference on Data communication. 2009: 51-62.
 - [81] Nightingale E B, Elson J, Fan J, et al. Flat Datacenter Storage [C]//10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12). 2012: 1-15.
 - [82] MegEngine is A Fast, Scalable and Easy-to-use Deep Learning Framework, with Auto-differentiation. [EB/OL]. <https://github.com/MegEngine/MegEngine>.
 - [83] Zhang X, Zhou X, Lin M, et al. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices [J/OL]. CoRR, 2017, abs/1707.01083. <http://arxiv.org/abs/1707.01083>.
 - [84] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for

- Language Understanding [C/OL]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Minneapolis, Minnesota: Association for Computational Linguistics, 2019: 4171-4186. <https://www.aclweb.org/anthology/N19-1423>. DOI: 10.18653/v1/N19-1423.
- [85] Guo Z, Zhang X, Mu H, et al. Single Path One-Shot Neural Architecture Search with Uniform Sampling [J/OL]. CoRR, 2019, abs/1904.00420. <http://arxiv.org/abs/1904.00420>.
- [86] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners [Z]. 2020.
- [87] Abadi M, Barham P, Chen J, et al. TensorFlow: A System for Large-Scale Machine Learning [C]//OSDI'16: Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation. USA: USENIX Association, 2016: 265-283.
- [88] Paszke A, Gross S, Massa F, et al. Pytorch: An Imperative Style, High-performance Deep Learning Library [J]. Advances in neural information processing systems, 2019, 32.
- [89] Wang M, chin Huang C, Li J. Unifying Data, Model and Hybrid Parallelism in Deep Learning via Tensor Tiling [Z]. 2018.
- [90] Yuan J, Li X, Cheng C, et al. OneFlow: Redesign the Distributed Deep Learning Framework from Scratch [J]. arXiv preprint arXiv: Arxiv-2110.15032, 2021.
- [91] Sergeev A, Del Balso M. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow [J]. arXiv preprint arXiv:1802.05799, 2018.
- [92] Jiang Y, Zhu Y, Lan C, et al. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters [C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020: 463-479.
- [93] Zhang H, Wu P, Deng Z, et al. Autodist: Acomposable and Automated Synchronization System for Distributed Deep Learning [J].
- [94] Lu W, Yan G, Li J, et al. Flexflow: A Flexible Dataflow Accelerator Architecture for Convolutional Neural Networks [C]//2017 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2017: 553-564.
- [95] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning [C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018: 578-594.
- [96] Jia Z, Padon O, Thomas J, et al. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions [C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 47-62.

作者简历及攻读学位期间发表的学术论文与研究成果

作者简历:

姓名: 户忠哲 性别: 男 出生年月: 1993 年 2 月

2017.9-2022.7 中国科学院计算技术研究所, 计算机系统结构, 博士研究生

2014.9-2017.7 中国科学院计算机网络信息中心, 计算机技术, 硕士研究生

已发表(或正式接受)的学术论文:

1. Hu Z, Xiao J, Tian Z, et al. A Variable Batch Size Strategy for Large Scale Distributed DNN Training[C]//2019 IEEE Intl Conf on Parallel Distributed Processing with Applications, Big Data Cloud Computing, Sustainable Computing Communications, Social Computing Networking (ISPA/BDCLOUD/SocialCom/SustainCom). IEEE, ISPA 2019(Best Paper Award)
2. Hu Z, Xiao J, Tan G, et al. MegTaiChi: Dynamic Tensor-based Memory Management Optimization for DNN Training. International Conference on Supercomputing(ICS 2022).
3. Hu Z, Xiao J, Sun N, Tan G, et al. Fast and Accurate Variable Batch Size Strategy for Large Scale Distributed DNN Training[J]. Concurrency and Computation: Practice and Experience. (SCI 收录)
4. ZhuHongrui, YuanGuojun, YaoChengji, Huzhongzhe, et al. Survey on network of distributed deep learning training[J]. Journal of Computer Research and Development 58.1 (2021): 98.
5. Zhang X, Xiao J, Zhang X, Hu Z, et al. Tensor Layout Optimization of Convolution for Inference on Digital Signal Processor[C]//2019 IEEE Intl Conf on Parallel Distributed Processing with Applications, Big Data Cloud Computing, Sustainable Computing Communications, Social Computing Networking (ISPA/BDCLOUD/SocialCom/SustainCom). IEEE, ISPA.

申请或已获得的专利：

1. “数据整合方法、分布式计算节点及分布式深度学习训练系统”，申请号 CN201910741235.9
2. “数据整合方法、分布式计算节点及分布式深度学习训练系统”，申请号 CN201910741733.3

攻读博士学位期间的获奖情况：

2020.11, 获中国科学院计算技术研究所所长优秀奖学金

致 谢

转眼五年间，博生涯就要结束了，回顾这五年来的点点滴滴，感慨颇多。在博士阶段艰辛的求学过程中，我深刻的认识到了自己能力的局限性，明白了自己的不足，自己知识是如此贫瘠。但幸运的是，我遇到了一群优秀的计算所人，他们就像一盏明灯，指引着我前进，在我困顿的时候给予我温暖。在我的博士论文即将完成之际，谨在此向多年来关心和指导我的老师、同学、朋友和家人致谢！

首先谨以最诚挚的敬意感谢我的导师孙凝晖研究员，感谢他五年来对我们学生的严格要求和悉心指导。仍记忆犹新第一次在讨论班向您汇报自己工作时，您那博大精深的学术功底、一针见血的犀利点评、敏锐洞察的科研眼光以及严谨求实的工作态度，深深地震撼了刚刚步入科研之路的我们。经过您的点拨和指导，不仅让我们快速发现自身的不足和缺陷，还独辟蹊径地引导迷茫的我们选择最佳努力方向。工作中给与我们提醒和鞭策的您在私底下却非常平易近人和风趣幽默，经常给我们分享您的所见所感，无形之中从另外一个角度教会我们生活中很多道理。恩师对我们学生各个方面的启发和教导，是我们取之不尽、用之不竭的宝贵财富，将会在未来伴随我一路不断成长！永远感谢您！

感谢科学计算组的肖俊敏老师，入门之初，在您耐心的教导下，让我慢慢学会以科研思维看待所研究的问题，以及如何迎接和克服工作中各种各样的挑战和困难。在您的指导下，我也完成了第一次学术论文的编写，其中不管是论文选题、创新思路、方法设计、实验构建甚至连英文写作都在您的帮助下完成，不仅使得论文成功发表，更为我后续的科研道路打下坚实的基础。您那日复一日、勤劳不倦的工作作风也是我不断努力的方向。

在日常学习和科研工作中，我还要感谢高性能中心主任谭光明研究员，您让我增长了见识开拓了眼界，同时也去不断学习新的知识或技能来填充自己，谭老师给予了我们充分的学术自由，可以根据自己的兴趣选择课题方向，鼓励我们去探索新的方向，并在新方向的探索上尽可能的给我们寻找有利于我们学习的科研条件和环境。在探索的过程中我也找到了自己喜欢的方向，并打算在工作后仍然坚定的做下去，我觉得这是十分宝贵的财富。

我还要感谢科学算法组组长何鑫老师。何老师总是和学生们打成一片，并在

科研和生活上悉心指导我们，他乐观幽默的态度，也给了我很多鼓舞。感谢贾伟乐老师，同时也是我的师兄，对科研的的执著与坚持对我有很大的鼓励。感谢王银山老师，杨少峰老师，汪云婷师妹，感谢你们给予无微不至的帮助。感谢实验室的老师们的指导和激励！在博士求学期间，实验室也了许多珍贵的记忆，同时也遇到很多一起奋斗的伙伴们，太多欢声笑语，太多激情辩论。感谢同级的张晓扬，朱弘睿同学，怀念我们一起工作的日子。我们在科研学习路上相互陪伴，相互帮助。感谢实验室的刘军红师姐，王元戎师兄，李雪琦师兄，曾平师兄，王彪，还有水超洋，于献智，张珂炜，李鸣一，景宗飞，胡思雨，刘扬，包晗等众多师弟和师妹，感谢你们在我科研道路上的帮助。我会永远怀念我们在一起的日子。另外还要感谢实验室的学习和生活秘书，感谢研究生处的周世佳、张平老师还有李丹老师，对待学生态度和蔼可亲，默默无闻地送走一代又一代的毕业生。

当然了，我还要特别感谢父母对我含辛茹苦的养育之恩。感谢你们对我的支持，感谢我的女朋友对我的科研道路的鼓励和支持，我日后必将报答这份恩情。论文搁笔之际，我二十多年的学生岁月也快要结束了，感谢所有曾经帮助过我的人，世界很大，总有一天我们会在世界的某个角落相遇。祝愿你们幸福安康，事业有成。

向所有帮助过我的老师、同学、朋友以及本论文的评阅老师表示深深地感谢！未来我将会保持更加热情的态度，面对生活和工作中各个挑战，迎接人生的下一关。